Vergleich von generierten und manuell erstellten MCP-Servern

Comparison between auto-generated and manually built MCP servers

Bachelorarbeit

Tim Tomczek
Fachhochschule Südwestfalen
21. März 2026

Autor: Tim Tomczek
Referent: Prof. Dr. Heiner Giefers
Korreferent: Prof. Dr.-Ing. Doga Arinir
Eingereicht: 21. März 2026

Zusammenfassung

Durch Software Architekturen wie Micro Services, die auf vielen kleinen Services basieren, stieg die Anzahl an Schnittstellen zwischen den Services. Eine beliebte Technologie für die Kommunikation zwischen den Services ist HTTP und das darauf aufbauende REST. Um diese Schnittstellen zu dokumentieren, hat sich die OpenAPI Spezifikation etabliert. Sie ermöglicht es, die Schnittstellen in einem maschinenlesbaren Format zu beschreiben, um daraus Dokumentationen oder Code zu generieren. Nach dem Aufkommen von großen Sprachmodellen (LLMs), die in der Lage sind, natürlichsprachliche Anfragen zu verstehen und darauf zu antworten, stellt sich die Frage, ob diese Modelle in der Lage sind, die Schnittstellen von REST APIs zu verstehen und damit zu interagieren. Dafür wurden die Modelle mit der 'Tool Calling' Funktion ausgestattet, welche Ende 2024 von Anthropic mit dem Model Context Protocol (MCP) vereinheitlicht werden soll. Diese Kombination von Schnittstellendokumentation und LLMs brachte die Idee hervor, aus den API Beschreibungen automatisch MCP Server generieren zu lassen. Diese Arbeit untersucht sechs solcher Generatoren, um zu evaluieren wie gut sie die daraus generierten MCP Server sind und ob sich der Aufwand der Erstellung eines manuell entwickelten MCP Servers lohnt. Dafür wurden die Generatoren anhand von vier Schnittstellen evaluiert. Untersucht wurden die unterstützten Funktionen aus den OpenAPI Spezifikationen, die unterstützten MCP Funktionen, die Qualität der generierten Server und die Ergebnisse die LLMs mit diesen Generatoren erzielen können. Als Vergleich wurde für jede der vier Schnittstellen ein manuell entwickelter MCP Server erstellt, um die Ergebnisse der Generatoren damit zu vergleichen. Die Ergebnisse zeigten große Unterschiede zwischen den Generatoren. So konnten einige Generatoren einige APIs erst gar nicht in einen nutzbaren MCP Server umwandeln. Bei den erzeugten MCP Servern zeigte sich das in den meisten der getesteten MCP Servern die 'Best Practices' welche in der MCP Specifikation beschrieben sind, garnicht oder nur teilweise umgesetzt wurden. Auch waren die Ergebnisse mit den LLMs je nach Generator und API sehr unterschiedlich und bewegten sich im Bereich um die 30% Erfolgsrate. Mit dem manuellen MCP Server konnten deutlich bessere Ergebnisse erzielt werden. Dennoch zeigt der Vergleich, dass die Generatoren aufgrund der schnellen Erstellung eines MCP Servers für umfangreiche Schnittstellen durchaus eine Alternative zum manuellen Erstellen eines MCP Servers sein können, wenn die gewünschte Nutzung es erlaubt, mit den Einschränkungen der Generatoren zu leben.

Inhaltsverzeichnis

Zusammenfassung **iii**

1 Einleitung **1**
1.1 Motivation und Problemstellung 1
1.2 Zielsetzung der Arbeit und Forschungsfragen 2

2 Stand der Technik **5**

3 Methodik **7**

4 Durchführung **9**
4.1 Verwendete Schnittstellen 9
4.2 Versuchsumgebung und -aufbau 13
4.3 MCP-Server Generatoren 15
4.3.1 MCPLink 15
4.3.2 ConvertMCP 16
4.3.3 OpenAPI-to-MCP-Generator 18
4.3.4 FastMCP 20
4.3.5 Tyk API to MCP 22
4.3.6 Stainless 24
4.4 Manuelle MCP-Server Implementierung 29

5 Vergleich und Fazit **47**
5.1 Vergleich der Ergebnisse 47
5.2 Fazit 52

Literatur **55**

Abbildungsverzeichnis **61**

Tabellenverzeichnis **63**

Listingverzeichnis **65**

1 Einleitung

Diese Arbeit entsteht im Rahmen des Bachelor Verbundstudiengangs Angewandte Informatik an der Fachhochschule Südwestfalen am Standort Iserlohn. Sie befasst sich mit der Analyse und Evaluierung von automatisch generierten Model Context Protocol (MCP) Servern aus OpenAPI Beschreibungen im Vergleich zu manuell erstellten Implementierungen und zielt darauf ab, zu untersuchen, wie gut diese automatisch generierten MCP-Server im Vergleich zu manuell erstellten Implementierungen abschneiden und ob sie für den produktiven Einsatz geeignet sind oder ob sich der Mehraufwand einer manuellen Implementierung lohnt.

1.1 Motivation und Problemstellung

Durch Paradigmen wie die Microservice Architektur benötigt man eine große Anzahl an Schnittstellen, da die Software modular aufgebaut ist und einzelne Komponenten über mehrere Computer, VMs oder Container verteilt sein können. Diese Schnittstellen werden meist als RESTful APIs implementiert, da sie eine flexible und skalierbare Möglichkeit bieten, um mit den verschiedenen Komponenten zu kommunizieren. Damit hat sich REST als Standard für Web-APIs etabliert [1] Diese Schnittstellen sollten möglichst gut dokumentiert sein, damit sie einfach genutzt und beim Auswechseln einer Komponente schnell und einfach implementiert werden können. Hierfür hat sich der OpenAPI Specification (OAS) Standard [2] etabliert, der es ermöglicht Schnittstellen standardisiert zu dokumentieren. OAS ermöglicht es Entwicklern, RESTful HTTP APIs zu beschreiben, einschließlich der verfügbaren Endpunkte, HTTP-Methoden, Anforderungs- und Antwortformate sowie Authentifizierungsmechanismen. Durch diesen standardisierten und maschinenlesbaren Ansatz können mittels einer Vielzahl von Generatoren automatisch Implementierungen für verschiedene Programmiersprachen und Frameworks erstellt werden, was den Entwicklungsprozess erheblich beschleunigt und die Konsistenz der API-Implementierungen gewährleistet. Diese Eigenschaften führten dazu, dass viele Anwendungen ihre Schnittstellen über OAS dokumentieren, um von den Vorteilen der Standardisierung und Automatisierung zu profitieren. Damit wurde die OpenAPI Specification, früher Swagger, nach eigenen Angaben von SmartBear, dem Unternehmen hinter Swagger, zum de-facto-Standard[3] für die Dokumentation von REST-Schnittstellen.

Mit der Verbreitung von großen Sprachmodellen (Large Language Models, LLM), wie die GPT-Reihe von OpenAI [4], gab es Ideen und Ansätze, diese Modelle mit bestehenden Anwendungen zu verknüpfen, um deren Funktionalität und Wissen zu erweitern. Ein erster Ansatz um den Wissensstand der Modelle dynamisch zu erweitern war das Retrieval-Augmented Generation, bei dem während der Anfrage bereitgestellte Informationen vom LLM durchsucht werden können um damit aktuellere Informationen zu erhalten und Halluzinationen zu reduzieren [5] . Nach der Erweiterung des Wissensstands der Modelle gelang es Forscher-/innen und Entwickler-/innen die Modelle mit auch mit der Außenwelt interagieren zu lassen. Dabei entstanden verschiedene Ansätze für die Integration von externen Werkzeugen und Anwendungen [6] [7] [8] . Im weiteren Verlauf wurden zunehmend zusätzliche Schnittstellen integriert, um die Fähigkeiten der LLMs zu erweitern und ihnen den Zugriff auf eine immer größere Anzahl von Anwendungen zu ermöglichen [9] [10] [11] . Allerdings blieben die Ansätze meist auf eine begrenzte Anzahl von Anwendungen oder Anwendungsbereichen beschränkt. Mit dem Function Calling der OpenAI Modelle, wie GPT-4, oder RestGPT [12] gab es erste Ansätze, um eine generische Schnittstelle für die Interaktion

zwischen LLMs und externen Anwendungen zu schaffen [12] [13] [14] . Mit diesen Ansätzen konnten LLMs mit einer großen Anzahlvon Anwendungen interagieren. Es fehlte jedoch ein einheitlicher Standard, der die Interaktion zwischen LLMs und externen Anwendungen definiert und standardisiert.

Genau dieses Problem wurde Ende 2024 mit der Veröffentlichung des Model Context Protocols (MCP), durch Anthropic, im November 2024 [15]addressiert. Das Model Context Protocol ist ein offener Standard, der eine einheitliche Schnittstelle für die Interaktion zwischen großen Sprachmodellen und externen Anwendungen definiert. MCP ermöglicht es LLMs, auf eine Vielzahl von Anwendungen zuzugreifen und mit ihnen zu interagieren, ohne dass spezifische Anpassungen für jede Anwendung erforderlich sind. Dies wird erreicht, indem die anwendungsspezifische Logik in einer als MCP-Server bezeichneten Schicht gekapselt wird. Der MCP-Server kann dabei eine eigenständige Anwendung sein oder auch in eine bestehende Anwendung integriert werden. Der MCP-Server stellt dem LLM dann über das Model Context Protocol die verschiedenen Interaktionsmöglichkeiten zur Verfügung, während der Server die Kommunikation mit der eigentlichen Anwendung übernimmt [16]. Für MCP steht eine Auswahl an Server Development Kits (SDKs) zur Verfügung [17], die es Entwicklern erleichtern, MCP-Server in verschiedenen Programmiersprachen zu implementieren. Diese SDKs bieten vorgefertigte Funktionen und Klassen, die die Entwicklung von MCP-Servern vereinfachen und beschleunigen.

Unternehmen wie Atlassian veröffentlichten schnell eigene MCP-Server für ihre Produkte [18], um deren Funktionalität LLMs zugänglich zu machen. Gerade kleineren Unternehmen kann es jedoch schwerfallen, eigene MCP-Server zu implementieren, da dies Entwicklungskapazitäten erfordert die nicht immer vorhanden sind. Zwar hilft es, dass man mithilfe der API Definition schnell und einfach eine neue Schnittstellenimplementierung erstellen kann und diese mithilfe eines MCP SDKs in einen MCP-Server verwandeln kann, jedoch bleibt der Aufwand je nach Umfang der API und der Komplexität der Anwendung nicht unerheblich. Da allerdings die Implementierung eines MCP-Servers jedoch einem gewissen Grad an sich wiederholendem Code bedarf, kamen schnell erste Projekte auf, die es ermöglichen, MCP-Server automatisch aus OpenAPI Beschreibungen zu generieren [19] [20]. Diese Projekte nutzen die maschinenlesbare Definition der Schnittstellen, um automatisch den notwendigen Code für den MCP-Server zu erstellen. Dadurch kann der Entwicklungsaufwand für Unternehmen deutlich reduziert werden, da keine (vollständig) manuelle Implementierung notwendig ist. Jedoch stellt sich die Frage, wie gut ein generierter MCP-Server im Vergleich zu einer manuellen Implementierung abschneidet und ob es sich lohnt den Mehraufwand einer manuellen Implementierung in Kauf zu nehmen.

1.2 Zielsetzung der Arbeit und Forschungsfragen

Wie im vorangegangenen Abschnitt beschrieben bieten generierte MCP-Server eine vielversprechende Möglichkeit, den Entwicklungsaufwand für die Erstellung eines eigenen MCP-Server zu reduzieren. Es stellt sich damit die Frage inwiefern und ob sich der Mehraufwand einer manuellen Implementierung lohnt oder ob eventuelle Nachteile durch ein Zeitersparnis ausgeglichen werden können. Diese Frage soll im Rahmen dieser Arbeit untersucht werden. Dafür werden anhand von verschiedenen Schnittstellen sowohl manuelle als auch automatisch generierte MCP-Server Implementierungen erstellt und diese anschließend miteinander verglichen und bewertet. Die zentrale Forschungsfrage dieser Arbeit lautet daher:

> *„Lohnt sich die manuelle Erstellung eines MCP-Servers gegenüber einem generierten basierend auf Qualität, Effizienz und Ressourceneinsatz?“*

Das Ziel der Beantwortung der Forschungsfrage ist es, Entwicklern und Unternehmen eine fundierte Entscheidungsgrundlage zu bieten, ob sie den Mehraufwand einer manuellen Implementierung in

Kauf nehmen sollten oder ob die automatisch generierten MCP-Server für ihre Zwecke ausreichend sind. Um diese Frage beantworten zu können, werden während der Durchführung verschiedene Unterziele verfolgt. Drei davon sind für eine aussagekräftige Beantwortung der Forschungsfrage relevant. Das vierte Ziel ist optional und wird nur verfolgt, sofern die Zeit es zulässt. Diese umfassen zunächst die Untersuchung der durch die Generatoren erstellten MCP-Server in Bezug auf Codequalität, Wartbarkeit (sofern verfügbar), Funktionsumfang, sowohl von Model Context Protocol als auch aus den Schnittstellen, und der Nutzung der aus den Schnittstellendefinitionen verwendeten Informationen. Das zweite Ziel ist die Analyse der Leistungsfähigkeit der generierten MCP-Servern anhand von verschiedene Metriken wie Antwortzeiten und Ressourcennutzung. Das letzte benötigte Ziel ist die Analyse der Leistungsfähigkeit der manuell erstellten MCP-Server im Vergleich zu den generierten Implementierungen wobei hier zusätzlich der Entwicklungsaufwand betrachtet wird. Sollte die Zeit es zulassen, wird zusätzlich untersucht, inwiefern die generierten MCP-Server durch gezielte Optimierungen verbessert werden können und wie sich diese dann im Vergleich zu den manuellen Implementierungen schlagen.

Ziel dieser Arbeit ist es nicht, eine Empfehlung für einen spezifischen MCP-Server Generator auszusprechen, sondern herauszufinden, ob der generelle Ansatz der automatischen Generierung von MCP-Servern aus OpenAPI Beschreibungen gegenüber einer manuellen Implementierung ausreichend Vorteile bietet, um den Einsatz zu rechtfertigen oder nicht. Außerdem soll die Arbeit aufzeigen, welche Aspekte bei der Entscheidung für oder gegen eine automatische Generierung berücksichtigt werden sollten.

2 Stand der Technik

Seit der Möglichkeit von LLMs externe Werkzeuge und Anwendungen zu nutzen, gibt es auch Benchmarks um die Leistungsfähigkeit der verschiedenen Modelle auf ihre Fähigkeit zur Nutzung von Werkzeugen zu testen [21] [22]. Die Ergebnisse wiesen auf eine eingeschränkte Fähigkeit der Modelle hin, die Tools richtig zu nutzen. Im APIBank Benchmark erreichte das damalig neueste OpenAI Modell GPT-4 eine Erfolgsrate von ca. 60% bei der Nutzung von APIs [21]. Paper wie ToolLLM [11] oder RestGPT [12] versuchten den Modellen den Umgang von APIs beizubringen, um sie flexibel mit externen Anwendungen interagieren zu lassen. Erst mit der Einführung des Model Context Protocols gab es einen einheitlichen Standard für die Interaktion zwischen LLMs und externen Anwendungen, der es ermöglicht, eine große Anzahl von Anwendungen über eine einheitliche Schnittstelle zugänglich zu machen. Die offizielle MCP Registry [23] enthält aktuell, Stand März 2026, über 10.000 MCP-Server [24]. Wie auch zuvor brachte die Veröffentlichung schnell erste Projekte hervor, die untersuchten, wie gut die aktuellsten Modelle mit MCP-Servern interagieren können [25] [26] [27] [28] . Jeder neue Ansatz zur Bewertung der Toolnutzung versprach dabei mehr Realitätsnähe und eine bessere Abbildung der tatsächlichen Nutzung von Tools durch LLMs, um damit die Leistungsfähigkeit der Modelle in realen Szenarien besser bewerten zu können. Die meisten Ansätze konzentrieren sich dabei auf die Bewertung der Fähigkeiten der Modelle die verwendeten Tools richtig zu nutzen. Ein bislang wenig erforschter Bereich, die Generierung von MCP-Servern aus OpenAPI Beschreibungen, wird jedoch nur von wenigen Projekten untersucht. Während der Recherche für diese Arbeit wurde nur ein Paper gefunden welches sich mit diesem Thema beschäftigt. In dem Paper 'Making REST APIs Agent-Ready: From OpenAPI to Model Context Protocol Servers for Tool-Augmented LLMs' [29] wird behauptet, dass die Erstellung von MCP-Servern viel redundante Arbeit enthält und das diese automatisiert werden könnte. Dafür wurde ein Python basierter MCP-Server Generator entwickelt, der aus OpenAPI Beschreibungen automatisch MCP-Server generieren kann. Mit diesem Generator (AutoMCP) wurde versucht für fünfzig APIs einen MCP-Server zu erstellen. Von den insgesamt 1011 Endpunkten konnte AutoMCP für 783 Endpunkte ein funktionsfähiges Tool erstellen. Dies entspricht einer Erfolgsrate von ca. 77,5%. Für 17 APIs boten die Anbieter selbst einen MCP-Server an. In 14 Fällen konnte AutoMCP mehr Endpunkte unterstützen als der offizielle MCP-Server. Bei anschließenden Versuchen mit den erzeugten Tools scheiterten 240 von 1023 Aufrufen. Eine Untersuchung der Fehler zeigte dabei fünf grundlegende Probleme auf: Ein falsches oder fehlendes SecurityScheme für die Authentifizierung, nicht dokumentierte Header, falsche Basis-Urls, fehlende Authentifizierung auf Endpunktebene und nicht passende Parametertypen. Mit durchschnittlich 19 Zeilen Anpassung pro API konnte die Erfolgsrate von AutoMCP von 76% auf 99% erhöht werden. Dieses Paper demonstriert die prinzipielle Möglichkeit sowie Leistungsfähigkeit, MCP-Server generieren zu lassen.

Ein Blog Eintrag der Firma Stainless Software Inc., eine Firma die eine Plattform auf die Erstellung von SDKs aus API Beschreibungen spezialisiert hat, beschreibt die Effizienz von einem sogenannten 'Code-Mode'. Bei diesem Modus erhält der MCP-Server keine direkten Tools zur Erledigung seiner Aufgabe, sondern erhält Zugriff auf die API Dokumentation und die Möglichkeit, Code auszuführen, um damit die benötigten Informationen zu erhalten und die Aufgabe zu erledigen [30]. Dieser Ansatz zeigte hohe Ergebnisse in Sachen Richtigkeit der Antwort und Effizienz bei der Einholung der Antworten. In diesen Kategorien konnten Ergebnisse von

über 90% erzielt werden. Auch Cloudflare und Anthropic veröffentlichten ähnliche Blog Beitrage [31] [32].

Ein anderes Unternehmen für die Erstellung von SDKs, Speakeasy, veröffentlichte eine Blog Beitrag in dem andere MCP Generatoren mit dem Speakeasy Produkt verglichen wurden [33]. Dort werden die Produkte von Speakeasy, Gram [34], FastMCP [19] und der openapi-mcp-generator [20] miteinander verglichen. Als Ergebnis wird angegeben, dass die Produkte von Speakeasy und Gram deutlich mehr Funktionen unterstützen als die anderen beiden Generatoren. Zu den untersuchten Punkten gehören unter anderem der Einrichtungsaufwand, die Anpassungsmöglichkeiten, Typensicherheit, Authentifizierungsmechanismen, Toolkuration und das Vorhandensein eines Test Clients.

Ein letzter Blog Beitrag der hier erwähnt werden soll stammt von Thiago Mendes der Firma TM Dev Lab [35]. In diesem werden MCP-Server in vier unterschiedlichen Programmiersprachen, Java, Go, JavaScript (Node.js) und Python, miteinander verglichen. Untersucht wurden Latenzen, Durchsatz, Ressourcennutzung und Zuverlässigkeit. Dafür wurden vier MCP-Server mit gleichen Funktionen implementiert und mittels eines Lasttestframeworks getestet. Die Vergleiche zeigten, dass die Java Implementierung die niedrigste Latenz und den höchsten Durchsatz aufwies, während die Go Implementierung die geringste Ressourcennutzung aufwies. Alle vier Implementierungen zeigten eine hohe Zuverlässigkeit mit einer Fehlerquote von 0%.

3 Methodik

In diesem Kapitel wird das methodische Vorgehen zur Beantwortung der Forschungsfrage beschrieben. Das Projekt gliedert sich in drei Hauptphasen: die Auswahl und Analyse der Test-APIs, die Implementierung der MCP-Server (sowohl automatisch generiert als auch manuell) und die anschließende Bewertung und der Vergleich der verschiedenen Implementierungen. Zuerst werden geeignete Test-APIs ausgewählt, die verschiedene Anwendungsfälle und Komplexitätsgrade abdecken, um eine umfassende Analyse zu ermöglichen. Diese sollen möglichst realitätsnah sein und möglichst unterschiedliche Anforderungen an die MCP-Server stellen, um ein möglichst aussagekräftiges Ergebnis zu erzielen. Des Weiteren wird bei der Auswahl der Test-APIs darauf geachtet, dass diese kostenfrei zugänglich sind. Danach werden für die ausgewählten APIs jeweils drei bis vier realitätsnahe Nutzungsszenarien definiert. Zusätzlich zu den für das ausgewählte Szenario benötigten Endpunkten werden die benötigten Parameter und Antworten festgehalten. Diese Informationen sollen für die spätere Bewertung der MCP-Server Ergebnisse als 'Ground Truth' (Referenzdaten) dienen.

Der nächste Schritt ist die Auswahl geeigneter MCP-Server Generatoren, die in der Lage sind, aus den OpenAPI Beschreibungen der ausgewählten APIs funktionsfähige MCP-Server zu generieren. Dabei wird darauf geachtet, dass die Generatoren eigenständige Implementierungen sind und nicht einen anderen Generator als Abhängigkeit nutzen. Außerdem sollen die Generatoren nach Möglichkeit noch aktiv gepflegt werden, eine gute Dokumentation besitzen und eine freie Lizenz verwenden. Um ein möglichst breites Spektrum abzudecken sollen die Generatoren unterschiedliche Programmiersprachen und Frameworks unterstützen.

Nun folgt die Generierung und Analyse der MCP-Server von den ausgewählten Generatoren. Die Generatoren werden mit den OpenAPI Beschreibungen der ausgewählten APIs ausgeführt, um die MCP-Server zu erstellen. Um die Vergleichbarkeit der Ergebnisse zu gewährleisten werden alle Schnittstellen im selben Format (JSON oder YAML) und in derselben Version (OpenAPI 3.x) bereitgestellt. Dabei wird das, von den Test-APIs, am häufigsten verwendete Format als Basis genutzt. Sollte eine Schnittstelle nur im anderen Format vorliegen, wird diese entsprechend konvertiert. Um zu untersuchen wie sich der Umfang der API beschreibung auf die Qualität der generierten MCP-Server auswirkt, werden zwei Varianten der OpenAPI Beschreibungen genutzt: eine minimalistische Version, die nur die unbedingt notwendigen Informationen für eine funktionsfähige OpenAPI Definition enthält, und eine normale Version, die alle verfügbaren Informationen nutzt. Sofern es sich bei den ausgewählten um statische Generatoren handelt, wird der generierte Code auf verschiedene Qualitätsmerkmale hin untersucht. Dabei wird die Codequalität anhand von Metriken wie Code-Komplexität, Lesbarkeit, Wartbarkeit und Erweiterbarkeit bewertet. Sollte es sich um dynamische Generatoren handeln, werden stattdessen nur die Möglichkeiten zur Anpassung untersucht. Bei allen generierten MCP-Servern wird der Funktionsumfang überprüft. Dies beinhaltet sowohl die Funktionen des Model Context Protocols als auch die Endpunkte der API. Um eine belastbare Bewertung über, für den MCP-Server, wichtigsten Informationen treffen zu können, wird noch dokumentiert welche Informationen aus der OpenAPI Beschreibung wo und wie verwendet werden. Nach der Analyse werden die MCP-Server mithilfe von mehreren LLMs an den zuvor definierten Nutzungsszenarien getestet. Dabei werden verschiedene Metriken wie die Antwortzeiten (vom MCP-Server) und die Richtigkeit der Antworten (im Vergleich zu den Referenzdaten) erfasst.

Nach der Analyse der generierten MCP-Server folgt die manuelle Implementierung der MCP-Server für die ausgewählten APIs. Aufgrund des zeitlichen Rahmens und des möglichen Umfangs der APIs werden bei der Implementierung der MCP-Server nur Funktionen für die Nutzungsszenarien implementiert. Für die Implementierung der MCP-Server werden die für die Nutzungsszenarien benötigten Endpunkte der APIs untersucht, ob und wie sich diese für die Nutzung durch ein LLM optimieren lassen. Die Optimierungen könnten beispielsweise die Reduzierung der Antwortgröße durch weglassen überflüssiger Informationen, die Anpassung der Antwortformate oder das zusammenfassen von mehreren Aufrufen umfassen. Für alle Implementierungen wird das offizielle Python MCP SDK [36] verwendet, um eine einheitliche Basis zu gewährleisten. Für eine Auswertung des Aufwands der manuellen Implementierung wird die aufgewendete Zeit für die Entwicklung der einzelnen MCP-Server dokumentiert , um sie später für den Vergleich von Kosten heranziehen zu können. Nach der Fertigstellung der manuellen MCP-Server werden diese ebenfalls mithilfe der zuvor definierten Nutzungsszenarien und LLMs getestet. Dabei werden die gleichen Metriken wie bei den generierten MCP-Servern erfasst.

Abschließend werden die Ergebnisse der generierten und manuellen MCP-Server verglichen und bewertet. Für die Bewertung werden verschiedene Kriterien herangezogen, die für die Nutzung eines MCP-Server im produktiven Einsatz relevant sind. Nachfolgend ist eine nicht abgeschlossene Liste der Bewertungskriterien:

- **Antwortzeit:** Die Zeit in Millisekunden die der MCP-Server für die Verarbeitung einer Anfrage benötigt.
- **Ressourcennutzung:** Analyse der Arbeitsspeichernutzung des MCP-Servers im ungenutzten Zustand.
- **Größe der Antwort:** Die Größe der Antwort des MCP-Servers in Zeichen.
- **Unterstützter Funktionsumfang:** Der Umfang der unterstützten MCP- und OAS-Funktionen und API-Endpunkte.
- **Parameternutzung:** Die Anzahl und der Inhalt der von den LLMs verwendeten Parameter im Vergleich zu den notwendigen Parametern für das jeweilige Nutzungsszenario.
- **Anzahl der verwendeten Tool-Aufrufe:** Die Anzahl der ausgeführten MCP Tool-Aufrufe im Vergleich mit der minimal notwendigen Anzahl an Tool-Aufrufen für das jeweilige Nutzungsszenario.
- **Richtigkeit der Antworten:** Bewertung der Korrektheit der Antworten im Vergleich zu den Referenzdaten.
- **Entwicklungsaufwand in Stunden:** Analyse des Zeitaufwands für die manuelle Implementierung im Vergleich zur Nutzung der Generatoren.

Anhand dieser Kriterien wird eine umfassende Bewertung der generierten und manuellen MCP-Server durchgeführt, um die Forschungsfrage dieser Arbeit zu beantworten und fundierte Empfehlungen für Entwickler und Unternehmen zu geben.

Sollte die Zeit es zulassen, wird zusätzlich untersucht, inwiefern die generierten MCP-Server durch gezielte Optimierungen verbessert werden können. Dabei werden mögliche Schwachstellen identifiziert und entsprechende Anpassungen vorgenommen, um die Leistungsfähigkeit und Funktionalität der generierten MCP-Server zu erhöhen. Anschließend werden die optimierten generierten MCP-Server erneut getestet und mit den manuellen Implementierungen verglichen, um zu sehen, ob die Optimierungen zu einer signifikanten Verbesserung geführt haben.

4 Durchführung

Nach dem das Vorgehen im vorherigen Kapitel beschrieben wurde, wird in diesem Kapitel die eigentliche Durchführung der Arbeit dargestellt. Dabei werden die einzelnen Schritte der Durchführung und der zugehörige Versuchsaufbau beschrieben.

4.1 Verwendete Schnittstellen

Die Suche nach geeigneten Schnittstellen erfolgte mittels systematischer Internetrecherche. Hilfreich bei der Suche waren dabei Listen mit öffentlichen APIs wie z.B. Public APIs [37] oder Free APIs [38]. Diese enthalten eine Vielzahl an APIs aus unterschiedlichen Bereichen, die frei verwendet werden können. Leider boten viele der der dort gelisteten APIs keine OpenAPI Beschreibungen an oder hatten keine aussagekräftigen Funktionalitäten um für dieses Projekt genutzt zu werden. Als Kriterien für die Auswahl der Schnittstellen wurde festgelegt, dass die APIs eine OpenAPI Beschreibung in Version 3.x besitzen müssen und kostenfrei zugänglich sein sollen. Desweiteren sollten die APIs unterschiedliche Anwendungsbereiche abdecken und verschiedene Komplexitätsgrade aufweisen, um eine umfassende Analyse zu ermöglichen. Außerdem sollte ein möglichst realitätsnaher Anwendungsfall gegeben sein, um die Ergebnisse der Analyse auf reale Szenarien übertragen zu können. Um den Umgang der MCP-Server Generatoren mit unterschiedlichen OpenAPI Beschreibungen zu untersuchen, wurde darauf geachtet, dass die APIs unterschiedliche Merkmale und Besonderheiten in ihren Beschreibungen aufweisen, wie z.B. verschiedene Authentifizierungsmechanismen, unterschiedliche Datenformate, komplexe Endpunktstrukturen oder weitere Funktionen der OpenAPI Spezifikation wie Pflichtparameter, Standardwerte oder verschiedene Antworttypen.

Es wurden drei öffentliche Schnittstellen aus dem Umfeld des Autors ausgewählt, die unterschiedliche Anwendungsbereiche abdecken und verschiedene Komplexitätsgrade aufweisen. Zusätzlich wurde eine selbst erstellte Schnittstelle ausgewählt. Diese Schnittstelle wurde in einem früheren Modul während des Studiums entwickelt und dient der Verwaltung eines Rechnungsmanagement Systems. Die Anwendung trägt den Namen *Invoice Manager* [39]. Sie bietet grundlegende Funktionen zur Erstellung, Verwaltung und Abfrage (CRUD) von Rechnungen und Kundeninformationen. Sie soll als realitätsnahe Schnittstelle dienen, die typische Informationsverwaltungssysteme abbildet. Außerdem bietet sich dadurch die Möglichkeit, die OpenAPI Beschreibung und das dahinterliegende System gezielt anzupassen und zu erweitern, um verschiedene Szenarien zu testen. Für die öffentlichen Schnittstellen wurde die API von der Quellcodeverwaltung GitHub [40], die API der Kommunikationsplattform Discord [41] und die API des Massive Multiplayer Online Role-Playing Games (MMORPG) EVE Online [42] ausgewählt.

Die GitHub API eignet sich gut für dieses Projekt, da GitHub und GitLab weit verbreitete Plattformen für die Versionsverwaltung und Zusammenarbeit an Softwareprojekten sind. Viele Entwickler und Teams nutzen diese Plattformen, um ihren Code zu hosten, zu verwalten und gemeinsam daran zu arbeiten. Durch die Bereitstellung einer umfassenden API ermöglicht GitHub es Entwicklern, auf eine Vielzahl von Funktionen und Daten zuzugreifen, die für die Verwaltung von Repositories, Issues, Pull Requests und anderen Aspekten der Softwareentwicklung relevant sind. Dadurch bietet die GitHub API eine realitätsnahe Schnittstelle, die in vielen Anwendungsfällen genutzt werden kann. Für die Analyse der unterstützten Funktionen von OAS durch die MCP-Server bietet die GitHub API einige interessante Punkte, zum Beispiel die

Definition eines Servers der die Basis-URL der API angibt, die Verwendung von Standardwerten für Parameter und die Nutzung von Kombinationsschemas wie oneOf, allOf und anyOf für die Definition von komplexen Datentypen. Sie bietet außerdem noch eine ausführliche Beschreibung der HTTP-Statuscodes, die von den verschiedenen Endpunkten bei verschiedenen Szenarien zurückgegeben werden können. Die API bietet keine Definition von Authentifizierungsmechanismen über securitySchemes in der OpenAPI Beschreibung an, sondern verweist in der Beschreibung der Endpunkte auf die Authentifizierung über OAuth2 oder Personal Access Tokens (PATs).

Die Discord API bietet Zugang zu den Funktionen und Daten der Discord Plattform, die von Millionen von Nutzern weltweit genutzt wird. Discord ist eine beliebte Kommunikationsplattform, die hauptsächlich von Gaming-Communities genutzt wird, aber auch in anderen Bereichen wie Bildung, Unternehmen und sozialen Gruppen an Bedeutung gewonnen hat. Die API ermöglicht es Entwicklern, Bots und Anwendungen zu erstellen, die mit Discord interagieren können, um Nachrichten zu senden, Server zu verwalten, Benutzerinformationen abzurufen und vieles mehr. Sie wurde ausgewählt weil sie verschiedene Authentifizierungsmechanismen unterstützt, darunter OAuth2 und Bot-Token, die in der OpenAPI Beschreibung über securitySchemes definiert sind. Dadurch bietet die Discord API eine gute Möglichkeit, den Umgang der MCP-Server Generatoren mit unterschiedlichen Authentifizierungsmechanismen zu untersuchen. Besonders interessant ist dabei die Verwendung von verschiedenen Authentifizierungsabläufen mit OAuth2, wie dem Authorization Code Flow und dem Client Credentials Flow. Zudem bietet auch diese API eine Beschreibung der HTTP-Statuscodes, und die Definition von Kombinationsschemas.

Die EVE Online API bietet Zugang zu den Daten und Funktionen des beliebten MMORPGs EVE Online das bekannt ist für sein komplexes Wirtschaftssystem und das damit verbundene Ökosystem aus Drittanbieter-Diensten die sich die API des Spiels zu nutze machen. Die API ermöglicht es Entwicklern, auf eine Vielzahl von Informationen zuzugreifen, darunter Charakterdaten, Marktdaten, Schiffsinformationen und vieles mehr. Sie zeichnet sich durch eine Standardantwort für Fehlerfälle aus, die bei vielen Endpunkten verwendet wird, und durch die Übermittlung von Meta-Daten in den Antwort-Headern, wie z.B. die Sprache der Antwort. Dadurch bietet die EVE Online API eine interessante Möglichkeit, den Umgang der MCP-Server Generatoren mit solchen Standardantworten zu untersuchen.

Die Invoice Manager API wurde ausgewählt, um eine realitätsnahe Schnittstelle zu bieten, die typische Informationsverwaltungssysteme abbildet. Da für diese API sowohl der Quellcode für das zugrundliegende System als auch die OpenAPI Beschreibung vorliegen, besteht die Möglichkeit, die API zu verändern, um verschiedene Szenarien zu testen, die in den öffentlichen APIs nicht abgebildet sind. Ein Beispiel wäre die Nutzung von XML als Datenformat für Anfragen und Antworten, was in den anderen APIs nicht der Fall ist. Die API stellt CRUD Operationen für Rechnungen, Kunden und weitere zugehörige Ressourcen bereit und bietet damit eine gute Grundlage, um die Funktionalität und Leistungsfähigkeit der MCP-Server an gängigen Unternehmensanwendungen zu testen.

Mit diesen vier Schnittstellen steht eine gute Grundlage zur Verfügung, um die Forschungsfrage dieser Arbeit zu untersuchen und fundierte Erkenntnisse über die Leistungsfähigkeit und Funktionalität von automatisch generierten MCP-Servern im Vergleich zu manuellen Implementierungen zu gewinnen. Um für den Vergleich eine Grundlage zu schaffen, wurden für jede der vier Schnittstellen drei bis vier realitätsnahe Nutzungsszenarien definiert. Diese Szenarien decken verschiedene Anwendungsfälle ab und wurden mit Fokus auf die Besonderheiten der jeweiligen APIs ausgewählt. Diese Nutzungsszenarien und die zugehörigen Endpunkte, Parameter und Antworten werden nachfolgend beschrieben. Basierend auf diesen Informationen werden in den folgenden Kapiteln die MCP-Server bewertet.

Für die Nutzungsszenarien der GitHub API wurde ein vom Autor erstelltes Repository auf GitHub verwenden. Das Repository ist unter *ttomczek/studygroupmap* [43] zu finden und dient als Beispielrepository für die Nutzungsszenarien. Das Repository enthält zwei Branches,

drei Tags und insgesamt 3 Issues. Eines davon ist geschlossen, die verbleibenden beiden sind offen. Die beiden Branches werden jeweils durch eine Regel geschützt, die es nur bestimmten Benutzern erlaubt, Änderungen daran vorzunehmen. Basierend auf diesem Repository wurden die folgenden vier Nutzungsszenarien definiert. Szenario eins soll die Anzahl der offenen Issues im Repository Abfragen. Dabei sollte der Endpunkt *GET /repos/owner/repo/issues* genutzt werden, um die Issues des Repositories abzurufen und anschließend die offenen Issues zu zählen. Die notwendigen Parameter für den Endpunkt sind der Besitzername des Repositories (owner), der Name des Repositories (repo) und der Filterstatus (state), der auf öpen"gesetzt werden soll. Die erwartete Antwort ist zwei. Dieses Szenario dient als allgemeiner Testfall für die Funktionalität der MCP-Server. Ein Szenario dieser Art wird für jede der vier Schnittstellen definiert. Szenario zwei soll ein neues Issue erstellen. Dafür soll der Endpunkt *POST /repos/owner/repo/issues* genutzt werden. Die notwendigen Parameter sind der Besitzername des Repositories (owner), der Name des Repositories (repo), der Titel des Issues (title) und die Beschreibung des Issues (body). Als Antwort wird ein Link zum erstellten Issue erwartet. Hier soll der Umgang der MCP-Server mit POST Anfragen und dem Übermitteln von Daten im Anfragekörper untersucht werden. ...Szenario drei soll die Anzahl der Regeln (protection) aller Branches im Repository abfragen. Dafür soll der Endpunkt *GET /repos/owner/repo/branches* genutzt werden. Die notwendigen Parameter sind der Besitzername des Repositories (owner) und der Name des Repositories (repo). Als Antwort wird die Anzahl der Branches mit einer Schutzregel erwartet, in diesem Fall zwei. Dieses Szenario soll den Umgang der MCP-Server mit verschachtelten Datenstrukturen in den Antworten untersuchen.... Szenario vier soll eine Liste aller vom Nutzer mit Stern versehener Repositories abrufen. Dafür soll der Endpunkt *GET /users/username/starred* genutzt werden. Der notwendige Parameter ist der Benutzername (username). Als Antwort wird die Anzahl der mit Stern versehenen Repositories erwartet, in diesem Fall fünf. Dieses Szenario soll die Behandlung des Union-Schemas in der Antwort in der OpenAPI Beschreibung untersuchen.

Für die Discord API wurde ein eigener Discord Server erstellt, um die Nutzungsszenarien zu testen. Der Server enthält zwei Textkanäle, einen Sprachkanal, ein Mitglied und einen Bot der dem Server hinzugefügt wurde. Einer der Textkanäle hat einen offenen Thread. Basierend auf diesem Server wurden die folgenden drei Nutzungsszenarien definiert. Das erste Szenario dient wieder als allgemeiner Testfall und soll die Anzahl der Textnachrichten auf einem bestimmten Kanal abfragen. Dafür soll der Endpunkt *GET /channels/channel.id/messages* genutzt werden. Dieser Endpunkt benötigt den Parameter der Kanal-ID (channel.id). Dieser Parameter muss erst ermittelt werden, indem der Endpunkt *GET /guilds/guild.id/channels* genutzt wird, um die Kanal-ID des gewünschten Textkanals zu erhalten. Dieser Endpunkt wiederum benötigt den Parameter der Server-ID (guild.id), die ebenfalls zuvor ermittelt werden muss, indem der Endpunkt *GET /users/@me/guilds* genutzt wird, um die Liste der Server abzurufen, auf denen sich der Bot befindet. Als Antwort wird die Anzahl der Nachrichten auf dem Kanal erwartet, in diesem Fall drei. Das zweite Szenario soll einen neuen Thread auf einem Textkanal erstellen. Dafür soll der Endpunkt *POST /channels/channel.id/threads* genutzt werden. Der notwendige Parameter ist die Kanal-ID (channel.id) des Textkanals, auf dem der Thread erstellt werden soll. Dieser muss erst wieder ermittelt werden. Zusätzlich müssen im Anfragekörper der Name des Threads (name) und die Art des Threads (type) angegeben werden. Als Antwort wird die ID des erstellten Threads erwartet. Hier können die Anfragedaten im Anfragekörper auf drei verschiedene Arten übermittelt werden: als JSON, als URL-kodierte Formulardaten oder als Multipart-Formulardaten. Dieses Szenario soll den Umgang der MCP-Server mit verschiedenen Datenformaten im Anfragekörper untersuchen. Das dritte Szenario soll die Anzahl der offenen Einladungen des Servers abfragen. Dafür soll der Endpunkt *GET /guilds/guild.id/invites* genutzt werden. Der notwendige Parameter ist die Server-ID (guild.id). Als Antwort wird die Anzahl der offenen Einladungen erwartet, in diesem Fall eine. Für diese API wird für jedes Szenario eine Authentifizierung benötigt. Da es hier die Möglichkeit der Nutzung von Bot-Tokens und OAuth2

gibt, soll untersucht werden, wie die MCP-Server Generatoren mit diesen unterschiedlichen Authentifizierungsmechanismen umgehen.

Die dritte Schnittstelle ist die EVE Online API. Für die Nutzungsszenarien wurde ein eigener kostenloser Account erstellt, um auf die API zugreifen zu können. Basierend auf diesem Account wurden die folgenden zwei Nutzungsszenarien definiert. Das erste Szenario soll die Anzahl aller im Spiel verfügbaren Raumschiffe abfragen. Dafür soll der Endpunkt *GET /universe/groups/group.id* genutzt werden. Dieser Endpunkt benötigt eine Gruppen-ID als Parameter. Allerdings muss dafür erst die Kategorie-ID (category_id) für Raumschiffe ermittelt werden, indem der Endpunkt *GET /universe/categories/* genutzt wird. Mit dieser ID lassen sich dann die Gruppen und damit der Types (Entitäten) herausfinden. Als Antwort wird die Anzahl der Raumschiffe erwartet, in diesem Fall 512. Dieses Szenario benötigt keine Authentifizierung. Hier kann der Umgang mit Headern in der OpenAPI Beschreibung untersucht werden, da der Endpunkt die Sprache der Antwort über den *Accept-Language* Header steuert. Dieser Header ist in der OpenAPI Beschreibung als optionaler Parameter definiert, da er einen Standardwert besitzt. Das zweite Szenario soll den Umgang mit Fehlern untersuchen. Dafür sollen Informationen über ein nicht vorhandenes Raumschiff abgefragt werden. Dafür soll der Endpunkt *GET /universe/types/type.id* genutzt werden. Der notwendige Parameter ist die Type-ID (type.id) des Raumschiffs. Als Type-ID wird eine ungültige ID verwendet, um einen Fehler zu provozieren. Als Antwort wird ein Fehlerobjekt erwartet, das den HTTP-Statuscode 404 und eine Fehlermeldung enthält. Dieses Szenario soll den Umgang der MCP-Server mit Fehlerantworten untersuchen.

Die letzte Schnittstelle ist die Invoice Manager API. Für diese Nutzungsszenarien wird eine Instanz der Anwendung [44] lokal auf dem Computer des Autors betrieben. Als Testdaten stehen zwei Steuersätze, eine Rechnungsvorlage, zwei Kunden, drei Ansprechpartner und sechs Rechnungen mit bis zu drei Positionen zur Verfügung. Zwei der Rechnungen sind als bezahlt markiert. Das erste Szenario soll die Anzahl der Rechnungen abfragen. Dafür soll der Endpunkt *GET /invoices* genutzt werden. Als Antwort wird die Anzahl der Rechnungen erwartet, in diesem Fall sechs. Parameter sind für diesen Endpunkt nicht notwendig. Dieses Szenario dient wieder als allgemeiner Testfall. Das zweite Szenario soll die Nutzung der Parameter testen, indem die Anzahl der bezahlten Rechnungen abgefragt wird. Dafür soll der Endpunkt *GET /invoices* genutzt werden, wobei der Parameter *paid* auf "true"gesetzt werden soll. Als Antwort wird die Anzahl der bezahlten Rechnungen erwartet, in diesem Fall zwei. Als drittes Szenario sollen alle Rechnungen eines bestimmten Kunden abgefragt werden. Dafür soll der Endpunkt *GET /invoices* genutzt werden, wobei der Parameter *customerNumber* auf die ID des gewünschten Kunden gesetzt werden soll. Diese muss vorher erst über den Endpunkt *GET /business-partners* ermittelt werden. Als Antwort wird die Anzahl der Rechnungen dieses Kunden erwartet, in diesem Fall drei. In diesem Szenario soll die Verwendung von Abhängigkeiten zwischen Endpunkten untersucht werden, da hier zuerst die Kunden-ID ermittelt werden muss, um die Rechnungen dieses Kunden abfragen zu können. Szenario vier soll eine neue Rechnung erstellen. Dafür soll der Endpunkt *POST /invoices* genutzt werden. Im Anfragekörper müssen die notwendigen Daten für die Rechnung angegeben werden, darunter der Kunde, das Rechnungsdatum, die Positionen und weitere Informationen. Als Antwort wird die ID der erstellten Rechnung erwartet. Dieses Szenario testet wie gut der Umgang mit einer größeren Anzahl von notwendigen Tool-Aufrufen funktioniert, da zuerst die Rechnung und dann die Positionen erstellt werden müssen. Das fünfte und letzte Szenario soll eine Rechnung als PDF herunterladen. Dafür soll der Endpunkt *GET /invoices/invoiceId/pdf* genutzt werden. Der notwendige Parameter ist die ID der Rechnung (invoiceId), die vorher ermittelt werden muss. Als Antwort wird die ID der PDF-Datei erwartet. Mit der ID soll dann die PDF-Datei über den Endpunkt *GET /files/fileId* heruntergeladen werden. Dieses Szenario soll den Umgang mit verschiedenen Datenformaten in den Antworten untersuchen, da die Antwort hier als PDF-Datei und nicht als JSON zurückgegeben wird. Zusätzlich kann bei der Erstellung der PDF Datei mit dem ersten Aufruf untersucht werden, wie der MCP-Server

mit längeren Antwortzeiten umgeht, da die Erstellung der PDF Datei einige Zeit in Anspruch nehmen kann. Für diesen Fall bietet MCP die Möglichkeit den Fortschritt eines Tool-Aufrufs zu übermitteln. Im Rahmen der Nutzungsszenarien der Invoice Manager API kann auch der Umgang mit XML als Datenformat untersucht werden. Dafür wird der Endpunkt *GET /invoices* zusätzlich so angepasst, dass er auf Anfrage die Antwort im XML-Format zurückgibt. Dazu wird der *Accept* Header in der Anfrage auf äpplication/xml"gesetzt. Dieses Szenario soll den Umgang der MCP-Server mit verschiedenen Datenformaten in den Antworten untersuchen.

In Tabelle 4.1 findet sich ein Überblick über die definierten Nutzungsszenarien der vier ausgewählten Schnittstellen dargestellt.

4.2 Versuchsumgebung und -aufbau

Die Versuche des Projekts werden auf einem Windows 11 Computer durchgeführt. Die MCP-Server werden, soweit möglich, lokal auf dem Computer betrieben. Für die Invoice Manager API wird außerdem eine lokale Instanz der Anwendung betrieben. Die LLMs werden von GitHub Models [45] bereitgestellt, da dies eine einfache Möglichkeit bietet, verschiedene LLMs über eine einheitliche Schnittstelle zu verwenden. Die ausgewählten LLMs müssen in der Lage sein, mit MCP-Servern zu interagieren. Daher werden nur LLMs ausgewählt, die die Nutzung von Tool-Aufrufen unterstützen. Zusätzlich wurde darauf geachtet, dass die LLMs unterschiedliche Größen und Architekturen aufweisen, um ein breites Spektrum an Leistungsfähigkeiten abzudecken. Ein weiteres Kriterium bei der Auswahl der LLMs war die Verfügbarkeit und Zugänglichkeit über GitHub Models, um eine einfache Integration in den Testaufbau zu gewährleisten. Daher wurde die Auswahl der LLMs auf Modelle mit der Nutzungslimitierung 'Low' beschränkt, um die Kosten im Rahmen zu halten. Basierend auf diesen Kriterien wurden die folgenden zwei LLMs ausgewählt: *OpenAI GPT-4.1 mini*[46] und *Mistral Medium 3 (25.53)*[47]. Die Modelle haben unterschiedlich große Kontextfenster. Das GPT-4.1 mini Modell kann bis zu 1.048.576 Token verarbeiten, während das Mistral Medium 3 Modell ein Kontextfenster von 128.000 Token besitzt. Diese Information ist relevant, da die Größe des Kontextfensters die Menge an Informationen beeinflusst, die das Modell bei der Interaktion mit den MCP-Servern berücksichtigen kann. Dies betrifft unteranderem auch die Beschreibungen der MCP-Server Tools. Um den Einfluss der LLMs durch eigenes Wissen auf die Ergebnisse zu minimieren, wurden nicht die großen Versionen der LLMs ausgewählt, sondern kleinere Varianten, die weniger Parameter besitzen. Dadurch wird die Abhängigkeit von dem Wissen der LLMs reduziert und der Fokus stärker auf die Interaktion mit den MCP-Servern gelegt. Die Modelle werden mit den Standardeinstellungen von GitHub Models verwendet, um eine einheitliche Basis für die Tests zu gewährleisten. Dabei werden keine speziellen Anpassungen oder Feinabstimmungen vorgenommen.

Um die relevanten Metriken während der Versuche zu erfassen, wird eine eigene MCP-Host Anwendung entwickelt [48]. Diese nutzt die Spring AI Komponenten, welche auf dem offiziellen Java MCP SDK [49] basieren, um mit den MCP-Servern und LLMs zu kommunizieren. Sie bietet eine einfache Oberfläche, auf der die, vom MCP-Server bereitgestellten, Tools und deren Beschreibung angezeigt werden. Sie erlaubt außerdem die Auswahl der dem LLM zur Verfügung gestellten Tools, um gezielt nur die für das jeweilige Nutzungsszenario notwendigen Tools bereitzustellen. Mittels verschiedener Buttons kann der Prompt an die konfigurierten LLMs gesendet werden. Durch ein Wrapping der Tools in der Anwendung können, neben der Protokollierung der Tool-Aufrufe, Metriken wie die verwendeten Parameter, die Antwortzeiten und die Größe der Antworten erfasst werden. Diese Anwendung wurde mithilfe von KI-Tools entwickelt. Die KI half bei der Erstellung der HTML-Seite und bei der Extraktion der Header Daten aus den Anfragen und Antworten an das KI-Modell, welches für die Tests verwendet wird. Als Alternative für die eigene MCP-Host Anwendung wird die GitHub Copilot Erweiterung

Nr.	Beschreibung	Anzahl Aufrufe	Eingabeparameter	Erwartetes Ergebnis
GitHub				
1	Ermittlung der Anzahl der offenen Issues eines Repositories	1	owner: ttomczek, repo: studygroupmap, state: open	2
2	Erstellung eines neuen Issues in einem Repository	1	owner: ttomczek, repo: studygroupmap, title: "Neues Issue"	Link
3	Ermittlung der Anzahl der Branch Regeln in einen Repository	3	owner: ttomczek, repo: studygroupmap	2
4	Ermittlung der favorisierten Repositories des Nutzers	1	username: ttomczek	5
Discord				
1	Ermittlung der Anzahl der Nachrichten eines Channels	3	Server: Benachrichtigungen, Channel: testszenario-1	3
2	Erstellung eines neuen Threads in einem Channel	4	Server: Benachrichtigungen, Channel: allgemein	-
3	Ermittlung der Anzahl offener Einladungen eines Servers	2	Server: Benachrichtigungen	1
EVE Online				
1	Ermittlung der Anzahl existierender Schiffe	48	-	550
2	Ermittlung der Strukturpunkte eines nicht existierenden Schiffs	1	TypeId: 12345	Fehler
Invoice Manager				
1	Ermittlung der Anzahl der Rechnungen	1	-	6
2	Ermittlung der Anzahl der bezahlten Rechnungen	1	paid: true	2
3	Ermittlung der Anzahl der Rechnungen eines Kunden	2	Kundenname: Musterfirma	3
4	Erstellung einer Rechnung	5	Kunde: Musterfirma, Position: Material	-
5	Download einer Rechnung	3	Rechnungsnummer: 12345	-

Tabelle 4.1: Überblick über die definierten Nutzungsszenarien der vier ausgewählten Schnittstellen.

für IntelliJ verwendet, um bei Problemen mit der MCP Nutzung Fehler in der selbsterstellten Anwendung auszuschließen. Damit könnten die MCP-Server ebenfalls getestet werden, allerdings bietet die Erweiterung keine Möglichkeit zur Erfassung der relevanten Metriken.

4.3 MCP-Server Generatoren

Die Auswahl der MCP-Server Generatoren erfolgte durch eine Internetrecherche. Dabei wurden verschiedene Quellen wie GitHub, offizielle Dokumentationen und Foren durchsucht, um geeignete Generatoren zu finden. Die ausgewählten Generatoren beinhalten sowohl Open-Source Projekte als auch kommerzielle Lösungen, um ein breites Spektrum an Ansätzen abzudecken. Bei der Auswahl der kommerziellen Generatoren wurde darauf geachtet, dass diese eine kostenlose Testversion oder eine freie Lizenz für den akademischen Gebrauch anbieten, um die Nutzung im Rahmen dieses Projekts zu ermöglichen. Bei der Auswahl der Open-Source Generatoren wurde darauf geachtet, dass diese aktiv gepflegt werden, eine gute Dokumentation besitzen, eine einfache Nutzung versprechen oder eine größere Menge an GitHub-Sternen aufweisen, um die Verbreitung und Nutzung zu gewährleisten. Anhand dieser Kriterien wurden die folgenden Generatoren ausgewählt: *ConvertMCP* [50], *OpenAPI to MCP Generator* [20] und *MCPLink* [51], *FastMCP* [19], *Stainless MCP* [52] und *Tyk API to MCP* [53]. Diese MCP-Generatoren werden nachfolgend für die Generierung der MCP-Server der ausgewählten Schnittstellen verwendet und die generierten MCP-Server analysiert und getestet. Die Analyse und Testergebnisse werden nachfolgend beschrieben. Die genauen Daten zu den Tests werden im Anhang dieser Arbeit dokumentiert [48].

4.3.1 MCPLink

Der erste getestete Generator ist *MCPLink* [51]. *MCPLink* ist ein in GO geschriebener dynamischer Generator der auf GitHub unter der MIT Lizenz verfügbar ist. Er besitzt zum aktuellen Zeitpunkt (29.12.2025) 588 Sterne, wurde aber zuletzt im März 2025 aktualisiert. Die Anwendung wird für die Nutzung separat gestartet und bietet selbst keine MCP-Server Funktionalität an. Über die Verbindungs-URL können dem Generator eine URL zu einer OpenAPI Beschreibung im YAML oder JSON Format, die Basis-URL der API, einen Authentifizierungs-Header und ein Filter für die zu verwendenden Endpunkte übergeben werden. Der Generator liest dann die OpenAPI Beschreibung ein und stellt die darin definierten Endpunkte als MCP-Tools zur Verfügung. Dabei werden nur die Endpunkte berücksichtigt, die dem angegebenen Filter entsprechen. Als MCP Transport unterstützt *MCPLink* nur die ab MCP Version 2025-03-26 als veraltet markierte HTTP mit Server Sent Events (SSE) Variante [54]. Diese wurde in folgenden Versionen des Model Context Protokolls durch Streamable HTTP ersetzt. Die Verbindung von mehreren MCP Clients mit unterschiedlichen Schnittstellen auf eine MCPLink Instanz ist möglich. Während der Analyse wurde dies mit der GitHub API und der Discord API getestet. *MCPLink* nutzt als Toolnamen als Toolnamen eine Kombination aus dem Prefix "mcplink_", dem Namen der Schnittstelle, wie sie in der OpenAPI Beschreibung definiert ist, und dem Pfad des Endpunkts. Dabei werden Sonderzeichen im Pfad durch Unterstriche ersetzt. So wird beispielsweise der Endpunkt *GET /repos/owner/repo/issues* der GitHub API zum Toolnamen *mcplink_GitHub_repos_owner_repo_issues_get*. Bei der Discord API gab es beim Aufruf einiger Tools Probleme, da die Toolnamen zu lang waren. Die MCP Spezifikation sieht für den Toolnamen eine Länge von bis zu 128 Zeichen vor [55]. Diese wurden dann beim Aufruf über die GitHub Models API mit einer entsprechenden Fehlermeldung abgelehnt. Daher wurde der Code des Generators angepasst, sodass nur noch der Endpunkt-Pfad als Toolname verwendet wird. Somit würde der Toolname für den zuvor genannten Endpunkt nur noch *repos_owner_repo_issues_get* lauten. Die Beschreibung des Tools setzt sich aus der OperationId des Endpunkts und der, optionalen,

Beschreibung des Endpunkts zusammen. Als Beispiel dient die Discord API, da diese keine Beschreibung für die Endpunkte bereitstellt. Der Endpunkt *GET /channels/channel_id/messages* besitzt die OperationId *list_messages*. Somit lautet die Beschreibung des zugehörigen Tools nur *list_messages*. Bei der Betrachtung des, für die Tools, generierten InputSchemas fiel zunächst auf, dass die Parameter die im Pfad enthalten sind, wie die *channel_id* aus dem vorherigen Beispiel, in einen Block 'pathNames' zusammengefasst werden, Query-Parameter in einen Block 'searchParams' und der Requestbody in einen Block 'requestBody'. Query-Parameter die mit einer Referenz einer Operation hinzugefügt werden, wie es bei der GitHub oder EVE Online API der Fall ist, um sich wiederholdende Parameter wie die zu liefernde Sprache zu definieren, werden nicht erkannt und fehlen somit vollständig im InputSchema. Grundsätzlich scheint dieser Generator Probleme mit Referenzen zu haben die in der OpenAPI Beschreibung verwendet werden. So werden beispielsweise auch Datentypsformate die über Referenzen definiert sind, nicht korrekt abgebildet oder auch Requestbodies die über Referenzen eingebunden werden, werden zwar erkannt, sind im InputSchema aber leer. Auch Beschränkungen wie Minimum- und Maximum-Werte für numerische Parameter werden nicht berücksichtigt. Auch erweiterte Kombinationsschemas wie OneOf, AnyOf und AllOf werden ignoriert und nicht korrekt im InputSchema abgebildet. Enum-Werte für Stringtypen werden korrekt erkannt und als Liste im InputSchema dargestellt und als zusätzliches Property 'enum' hinzugefügt. Die Inbetriebnahme aller vier Schnittstellen dauerte insgesamt vier Stunden. Mit einem verbundenen MCP-Client verbraucht der Generator ohne laufende Aktion im Durchschnitt ca 86,25 MB Arbeitsspeicher laut dem Windows 11 Taskmanager. Die Ergebnisse der Nutzungsszenarien von *MCPLink* fallen mit 4 von 56 erfolgreich abgeschlossenen Szenarien schlecht aus. Das entspricht einer Erfolgsrate von ungefähr 7,14 %. Das einzige erfolgreich abgeschlossene Szenario ist das vierte der GitHub API, bei dem die Anzahl der vom Nutzer favorisierten Repositories abgefragt wird. Hierbei konnten sowohl das GPT Modell als auch das Mistral Modell mit einem Toolaufruf die korrekte Antwort liefern. Die Tools lieferten dabei jeweils 29944 Zeichen. Alle anderen Szenarien schlugen aufgrund von Fehlern bei übergebenen Parametern fehl. Bei der GitHub API gab es Probleme mit dem Sort Parameter als Enum Wert, die Discord API hatte Probleme bei der Abfrage der Server, der EVE Online API fehlten notwendige Parameter und bei der Invoice Manager API verursachten auch nicht vorhandene Parameter wie in Szenario eins, Abfrage aller Rechnungen, Fehler.

4.3.2 ConvertMCP

Als zweites wurde der Generator *ConvertMCP* [50] untersucht. *ConvertMCP* ist ein mit Typescript geschriebener statischer Generator. Mit einem Alter von knapp zwei Monaten zum Zeitpunkt der Analyse (29.12.2025) ist es das jüngste Projekt der ausgewählten Generatoren und besitzt nur einen Stern auf GitHub (Stand 29.12.2025), wurde aber aufgrund der Aktualität und der einfachen Generierung ausgewählt. Das Projekt steht unter der MIT Lizenz. Dieser Generator ist interessant durch sein einfaches Nutzungskonzept. Er bietet eine Webseite auf der die OpenAPI Beschreibung als Datei hochgeladen, als URL zu einer Online Beschreibung angegeben oder direkt als Text eingegeben werden kann. Nach dem Einfügen der API Beschreibung bietet die Weboberfläche die Möglichkeit anzugeben, welche Endpunkte in den generierten MCP-Server aufgenommen werden sollen. Als Ausgabesprache bietet der Generator alle offiziellen von MCP mit SDKs unterstützen Sprachen an: Typescript, Python, Go, Kotlin, Swift, Java, C#, Ruby, Rust und PHP [17]. Nach der Auswahl der Endpunkte und der Programmiersprache kann der MCP-Server generiert und als ZIP-Datei heruntergeladen werden. Der Quellcode wird dabei, nach eigenen Angaben, rein im Browser generiert, es findet keine Kommunikation mit einem Server statt. Für diese und die folgenden Analysen wird immer die beliebteste verfügbare Programmiersprache des jeweiligen Generators verwendet. Die Beliebtheit wird anhand der diesjährigen (2025) StackOverflow

Developer Survey, anhand der Kategorie 'Most popular technologies: Programming, scripting, and markup languages' über alle Teilnehmenden, bestimmt [56]. Anhand dieser sind die drei beliebtesten Sprachen JavaScript, Python, TypeScript und Java in dieser Reihenfolge. Da *ConvertMCP* nur TypeScript und kein reines JavaScript anbietet, wird die nachfolgende Sprache, Python, für die Analyse verwendet. Die Projektstruktur in der heruntergeladenen ZIP-Datei ist einfach gehalten und besteht aus nur drei Dateien. Einer README Datei mit Informationen zur Installation und Nutzung, einer requirements.txt Datei mit den notwendigen Abhängigkeiten und einer server.py Datei die den MCP-Server implementiert. In der requirements.txt sind zwei Anhängigkeiten definiert: *MCP* in einer Version 0.9.0 oder höher und *httpx* in einer Version 0.26.0 oder höher. Diese können einfach mit dem Python Paketmanager pip installiert werden. Die server.py Datei enthält den gesamten Quellcode des MCP-Servers. In der server.py werden ein MCP-Server Objekt und ein httpx AsyncClient Objekt erstellt. Der MCP-Server stellt zwei Funktionen bereit. Eine die eine Liste aller Tools zurückgibt und eine die Tool-Aufrufe entgegennimmt und verarbeitet. Die Liste der Tools besteht aus einer Liste von Toolobjekten, wobei jeder Endpunkt der OpenAPI Beschreibung als ein Tool dargestellt wird. Als Toolname wird wieder die OperationId verwendet. Die Beschreibung besteht entweder aus der Summary des Endpunkts oder, falls diese nicht vorhanden ist, aus der der HTTP-Methode und dem Pfad des Endpunkts. Die Parameter des Endpunkts werden im InputSchema des Tools dargestellt. Dabei werden Pfad-Parameter und Datentypsformate wie Datum, RegEx oder Enums nicht berücksichtigt. Bei mehrzeiligen Beschreibungen der Parameter wird ein falscher Pythonquelltext generiert, da die Zeilenumbrüche direkt in den String übernommen werden. Dies führte bei der GitHub API dazu, dass der generierte Quellcode nicht ausführbar war. Der Generator hat auch Probleme mit referenzierten Parametern. Diese werden zwar erkannt aber mit dem Namen und der Beschreibung 'undefined' als String Parameter im InputSchema eingefügt. Ein Requestbody wird im InputSchema als ein zusätzliches Property 'body' hinzugefügt. Dieser enthält aber keine weiteren Informationen über dessen Struktur. Sondern nur eine Beschreibung 'Request body'. Notwendige Parameter werden über ein 'required' Property an dem jeweiligen Property markiert. Dies entspricht nicht der MCP Spezifikation. Die zweite Funktion des MCP-Servers verarbeitet die Tool-Aufrufe. Diese nimmt den Namen des Tools und die übergebenen Parameter entgegen. Anschließend wird eine HTTP-Anfrage an die Basis-URL der API gesendet, die in der OpenAPI Beschreibung definiert ist. Die Antwort der API wird dann als Antwort des Tool-Aufrufs zurückgegeben oder im Falle eines Fehlers wird die Fehlermeldung zurückgegeben. Die Auswahl des zu nutzendes Endpunkts erfolgt über eine if-else Verzweigung, die für jeden Endpunkt einen eigenen Block besitzt. Dies führt dazu, dass der generierte Quellcode bei einer größeren Anzahl von Endpunkten sehr lang wird und die Übersichtlichkeit leidet. Außerdem ist auch hier einer Syntaxfehler im generierten Quellcode aufgetreten, da in dem elif Schlüsselwort ein Tabulator-Zeichen enthalten ist. Eine Unterstützung von Authentifizierungsmechanismen ist nicht implementiert. Um eine Authentifizierung zu ermöglichen, muss der generierte Quellcode manuell angepasst werden. Dafür wurde dem Skript ein möglicher Parameter für einen Authentifizierungs-Token hinzugefügt, der dann in den Headern der HTTP-Anfragen verwendet wird. Dieser einfache Mechanismus war nur möglich, da *ConvertMCP* nur STDIO als MCP Transport unterstützt und damit im Startkommando des MCP-Clients der Token übergeben werden kann. Die versuchte Inbetriebnahme der Schnittstellen dauerte in Summe ca. 4,5 Stunden. Aufgrund der beschrieben Probleme und nach manuellen Korrekturen könnte jedoch nur die Invoice Manager API erfolgreich betrieben werden, da die Korrektur der anderen Schnittstellen den zeitlichen Rahmen gesprengt hätte. Mit einem verbundenen MCP-Client verbraucht der Generator ohne laufende Aktion ca. 48,8 MB Arbeitsspeicher laut dem Windows 11 Taskmanager. Bei den Tests der Nutzungsszenarien der Invoice Manager API wurden 11 von 20 Versuchen richtig beantwortet. Das entspricht einer Erfolgsquote von 55 %. Insgesamt liegt die Erfolgsquote bei 19,6 % über alle vier Schnittstellen. Das erste Szenario zur Ermittlung der Anzahl aller Rechnung wurde wurde von beiden Modellen

korrekt mit einem Toolaufruf ausgeführt, der jeweils 1997 Zeichen lieferte, jedoch gab nur das GPT Modell die korrekte Antwort. Beim zweiten Szenario, der Ermittlung der Anzahl der bezahlten Rechnungen, wurde das Szenario ebenfalls von beiden Modellen korrekt mit einem Toolaufruf und dem richtigen Parameter ausgeführt, der jeweils 688 Zeichen lieferte. Hier lieferten beide Modelle die korrekte Antwort. Das dritte Szenario wurde nur vom GPT Modell korrekt ausgeführt. Es fragte anhand des Kundennamen die Kunden-Id ab um diese dann zur Ermittlung der Rechnungen weiter zu verwenden. Da aber das Kundenobjekt auch die Ids der zugehörigen Rechnungen enthält, konnte das GPT Modell die Anzahl der Rechnungen direkt aus dem Kundenobjekt ermitteln, ohne einen weiteren Toolaufruf tätigen zu müssen. Das Mistral Modell versuchte hingegen direkt die Rechnungen anhand einer erfundenen Kunden-Id abzufragen, was zu einem Fehler führte. Der Toolaufruf des GPT Modells lieferte 270 Zeichen, der des Mistral Modells 15 Zeichen. Wie zu erwarten war führte Nutzungsszenario vier, zur Erstellung einer neuen Rechnung, mit beiden Modellen, zu einem Fehler, da der Requestbody nicht korrekt im InputSchema dargestellt wurde und somit keine Daten übergeben werden konnten. Es wurden zwar korrekt die erforderlichen vorbereitenden Toolaufrufe zur Ermittlung der Kunden-Id, einer Rechnungsvorlagen-Id und einer Steuer-Id gettätigt, jedoch konnte die Rechnung nicht erstellt werden, da der Requestbody leer blieb. Die Toolaufrufe des GPT Modells lieferten zusammen jeweils 839 Zeichen, die des Mistral Modells 1617 beziehungsweise 1231 Zeichen. Das letzte Szenario, Rechung herunterladen, wurde vom GPT Modell korrekt mit den vier erforderlichen Toolaufrufen ausgeführt und nutzte jeweils 4807 Zeichen als Antwort. Das Mistral Modell scheiterte jedoch bereits am ersten Toolaufruf zur Ermittlung der Rechnungs-Id, da es versuchte eine erfundene Rechnungs-Id zu verwenden, anstatt die Id aus dem vorherigen Toolaufruf zu nutzen. Der Aufruf lieferte 15 Zeichen als Antwort. Erst beim zweiten Versuch konnte das Mistral Modell die Rechnung-Id korrekt ermitteln und anschließend die Rechnung herunterladen. Insgesamt nutzte es dabei 3829 Zeichen als Antwort.

4.3.3 OpenAPI-to-MCP-Generator

Der *OpenAPI-to-MCP-Generator* [20] (OASMCP) ist ein weiterer statischer MCP-Server Generator, der mit TypeScript und Node.js erstellt wurde. Das Projekt ist auf GitHub unter der MIT Lizenz verfügbar, besitzt zum Zeitpunkt der Analyse (31.12.2025) 496 Sterne und wurde zuletzt im Oktober 2025 aktualisiert. Generiert wird der MCP-Server über die Kommandozeile mittels des *npx* Befehls. Dafür werden eine OpenAPI Beschreibung, im YAML oder JSON Format, als Pfad oder URL und ein Ausgabeordner als Option übergeben um einen MCP-Server mit dem STDIO Transport zu generieren. Mit weiteren Optionen kann, unter anderem, der MCP Transport auf SSE oder Streamable-http geändert werden oder eine Basis-URL für die API angegeben werden. Zusätzlich kann der Generator programmatisch in einer Node.js Anwendung verwendet werden, um ihn zum Beispiel in eine eigene Anwendung zu integrieren. Für die Transports SSE und Streamable-http stellt der Generator einen eigenen Testclient bereit, um den generierten MCP-Server zu testen. Dieser kann im Browser ausgerufen werden. Für die Filterung von Endpunkten bietet der Generator das Flag 'x-mcp'. Dieses kann an Pfaden, Operationen oder an der Wurzel der OpenAPI Beschreibung gesetzt werden, um anzugeben, ob der Endpunkt in den generierten MCP-Server aufgenommen werden soll oder nicht. Für die Analyse wurden die vier Schnittstellen mit der Transport Option Streamable-http generiert. Mit dieser Variante war eine Verbindung mit der Testanwendung aufgrund eines falschen Mediatype Headers mit dem Wert plain/text nicht möglich. Mit dem GitHUb Copilot Plugin für IntelliJ konnte eine Verbindung hergestellt werden. Derselbe Fehler trat auch bei der Nutzung mit SSE auf. Deshalb wurde für die weitere Analyse der STDIO Transport verwendet. Der Generator erstellt unter dem angegebenen Ausgabepfad ein vollständiges Node.js Projekt mit einer package.json Datei, einer Beispiel .env Datei für Umgebungsvariablen und einem src Ordner mit dem Quellcode des MCP-Servers. Der src Ordner enthält eine index.ts Datei und im Falle von SSE

oder Streamable-http eine weitere Datei für den Transport. In der index.ts Datei sind alle Tools als eine Map von Toolnamen zu einer Tooldefinition definiert. Außerdem enthält sie eine Funktion zur Ausführung der Tools. Diese übernimmt auch die Validierung der übergebenen Parameter anhand des InputSchemas, das hinzufügen der Authentifizierungsinformationen, falls notwendig, das Senden der HTTP-Anfrage an die API und die Rückgabe der Antwort oder des Fehlers. Falls der STDIO Trasnport verwendet wird, wird in der Datei auch das MCP-Server Objekt erstellt, verwaltet und gestartet. Falls ein anderer Transport verwendet wird, wird dies in der bereits erwähnten separaten Datei gehandhabt. Als externe Anhängigkeiten werden in der package.json *@modelcontextprotocol/sdk*, *axios*, *dotenv*, *zod*, *json-schema-to-zod*, *hono*, *@hono/node-server*, *uuid* und *fetch-zo-node* verwendet. Nach der Installation der Abhängigkeiten mit dem Node.js Paketmanager npm und dem bauen mit dem Befehl *npm run build*, kann der MCP-Server mit dem Befehl *npm run start* gestartet werden. Die Authentifizierung wird über Umgebungsvariablen gehandhabt, die in der .env Datei definiert werden können. Der Generator unterstützt dabei verschiedene Authentifizierungsmechanismen wie API-Keys, Bearer Tokens und Basic Auth und auch den OAuth2 Client Credentials Flow. Diese werden dann automatisch in den HTTP-Anfragen verwendet, sofern die OpenAPI Beschreibung diese Mechanismen definiert. Da die GitHub API keine explizite SecurityScheme Definition in der OpenAPI Beschreibung besitzt, war eine Authentifizierung nicht möglich. Als Toolnamen wird wieder die OperationId des Endpunkts verwendet. Die Beschreibung des Tools wird entweder aus der Description des Endpunkts oder, falls diese nicht vorhanden ist, aus wie folgt zusammengesetzt: *EXECUTES [HTTP-Methode] [Pfad]*. Dieser Generator hat mit mehrzeiligen Beschreibungen der Endpunkte keine Probleme. Das InputSchema wird wird korrekt generiert, wobei auch referenzierte Parameter und Datentypsformate wie zum Beispiel das *Snowflake* Format der GitHub API korrekt abgebildet werden. Enums werden als Strings mit dem Zusatzattribut 'enum' dargestellt und Standardwerte werden über das Attribut 'default' angegeben. Kombinationsschemas wie OneOf, AnyOf und AllOf werden als Liste von möglichen Schemas im InputSchema dargestellt. Minumun, Maximum und Beispielwerte werden ebenfalls im InputSchema berücksichtigt. Notwendige Parameter werden über eine Liste von Property-Namen im 'required' Attribut des InputSchemas angegeben und auch Requestbodies werden korrekt im InputSchema dargestellt. Die Inbetriebnhame dauerte insgesamt ca 6,5 Stunden. Der MCP-Server benötigt mit einem verbundenen MCP-Client ohne laufende Aktion durchschnittlich 33,45 MB Arbeitsspeicher laut dem Windows 11 Taskmanager. Trotz dieser ausführlichen Darstellung konnten nur 19 von 56 (33,9%) Tests erfolgreich abgeschlossen werden. Davon sind zwei Szenarien, acht Tests, von der GitHub API, welche aufgrund der fehlenden Authentifizierung nicht durchgeführt werden konnten. Bei den GitHub Szenarien konnte das Mistral Modell kein Test erfolgreich abschließen. Es wurde auch kein einziger Toolaufruf getätigt. Das GPT Modell konnte beim ersten Szenario im ersten Versuch eine richtige Antwort liefern, indem es die Anzahl der offenen Issues korrekt ermittelte. Dabei wurden 8608 Zeichen genutzt. Im zweiten Versuch wurde der Paging Parameter *per_page* auf den Wert '1' gesetzt wodurch das Ergebnis verfälscht wurde. Beim dritten Szenario führte das Mistral Modell wieder keine Toolaufrufe auf. Das GPT Modell konnte aber in beiden Versuchen die Anzahl richtig bestimmten. Es nutzte dabei jeweils 592 Zeichen. Szenario zwei und vier konnten aufgrund der fehlenden Authentifizierung nicht durchgeführt werden. Bei der Discord API konnte kein einziges Szenario erfolgreich abgeschlossen werden. Das Mistral Modell führte bei Szenario eins und drei keine Toolaufrufe aus und hatte beim zweiten Szenario Probleme mit zu vielen Inputtoken. Das GPT Modell führte zwar Toolaufrufe aus. Diese schlugen aber fehl, da die *guild_id* nicht übergeben wurde. Diese ist auch nicht im InputSchema definiert. Die Disocrd API liegt im JSON Format vor. Hier wurden die Pfadparameter nicht korrekt im InputSchema abgebildet. Query-Parameter hingegen schon. Bei der Invoice Manager API, welche im YAML Format vorliegt, wurden die Pfadparameter korrekt im InputSchema dargestellt. Das Mistral Modell führte beim ersten Szenario der EVE Online API wieder keine Toolaufrufe aus.

Das GPT Modell führte zaw die notwendige Anzahl an Aufrufen mit den richtigen Parametern aus, konnte aber die korrekte Antwort nicht liefern. Es nutzte dabei einlal 15202 und einmal 14991 Zeichen. Beim zweiten Szenario lieferte das GPT Modell beide male die korrekte Antwort mit jeweils einem Toolaufruf, der 147 Zeichen nutzte. Das Mistral Modell konnte erst beim zweiten Versuch die korrekte Antwort liefern. Dabei benötigte es drei Aufrufe entgegen dem einen notwendigen und nutze damit 382 Zeichen. Die Versuche der Invoice Manager API verliefen deutlich besser als bei den anderen Schnittstellen. Hier konnten 14 von 20 (70%) Szenarien erfolgreich abgeschlossen werden. Im ersten Szenario riefen beide Moedlle jeweils ein Tool auf und nutzen dabei immer 3103 Zeichen. Jedoch antwortete das Mistral Modell mit einer falschen Anzahl an Rechnungen. Im zweiten Szenario konnten beide Modelle die korrekte Antwort mit einem Toolaufruf liefern der jeweils 1113 Zeichen nutzte. Das dritte Szenario konnte wieder nur vom GPT Modell erfolgreich abgeschlossen werden. Es nutzte dabei einen Toolaufruf und 496 Zeichen. Das Mistral Modell scheiterte wieder an mangelnder Toolnutzung. Szenario vier zur Erstellung einer neuen Rechnung konnte von beiden Modellen erfolgreich abgeschlossen werden. Das GPT Modell benötigte dabei fünf Toolaufrufe und nutzte 1577 und 1590 Zeichen. Das Mistral Modell benötigte ebenfalls fünf Toolaufrufe, nutzte dabei aber 1604 und 2056 Zeichen. Beide Modelle verwendeten einen Toolaufruf weniger als bei der Szenariodefinition angegeben, da sie die Rechnung nicht um die zuvor erzeugte Positions-Id erweiterten. Das letzte Szenario, das Herunterladen einer Rechnung, konnte wieder nur vom GPT Modell erfolgreich abgeschlossen werden. Es benötigte dafür vier Toolaufrufe und nutzte 5660 und 5659 Zeichen. Das Mistral Modell scheiterte wieder an mangelnder Toolnutzung.

4.3.4 FastMCP

Der letzte Open Source Generator ist *FastMCP* [19]. *FastMCP* ist ein Framework für die Entwicklung von MCP-Servern in Python. Es ist auf GitHub unter der Apache 2.0 Lizenz verfügbar, besitzt zum Zeitpunkt der Analyse (31.12.2025) 21.600 Sterne und wird aktiv weiter entwickelt. Die erste Version von *FastMCP* wurde bereits 2024 in das offizielle MCP SDK integriert [57]. Die aktuelle Version ist 2.0. Diese wird zum Beispiel auch für den OAS MCP Generator von Amazon Web Services (AWS) [58] verwendet. Die Nutzung des MCP Generators erfolgt über die Implementierung einer eigenen Python Anwendung, die die *from_openapi* Funktion des Frameworks verwendet, um daraus dynamisch die MCP-Tools zu generieren. Ein Minimalbeispiel anhand der Invoice Manager API ist in Listing 4.1 zu finden.

Listing 4.1: Minimalbeispiel für die Nutzung von FastMCP zur Generierung eines MCP-Servers aus einer OpenAPI Beschreibung der Invoice Manager API.

```
import httpx
import yaml
from fastmcp import FastMCP


spec = yaml.safe_load(httpx.get("https://raw.githubusercontent.com/
   TTomczek/apis/refs/heads/main/invoice-manager/openapi.yaml").text)

api_client = httpx.AsyncClient(
    base_url=spec["servers"][0]["url"]
)

mcp = FastMCP.from_openapi(
    openapi_spec=spec,
    client=api_client,
    timeout=30.0
```

```
)

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=3000)
```

In dem Beispiel wird die OpenAPI Beschreibung der Invoice Manager API von einer URL eingelesen und daraus die MCP-Tools generiert. Man kann sehen das für die Kommunikation mit der API ein *httpx.AsyncClient* Objekt erstellt wird, welches dann an die *from_openapi* Funktion übergeben wird. Die *base_url* des Clients muss dabei manuell gesetzt werden, da diese nicht aus der OpenAPI Beschreibung gelesen wird. Diese wird im Beispiel aus der zuvor eingelesenen OpenAPI Beschreibung extrahiert. Am Ende des Beispiels wird der MCP-Server mit dem Transport 'Streamable-http' gestartet unter Angabe der Hostadresse und des Ports gestartet. STTDIO und SSE werden ebenfalls unterstützt. Authentifizierungsmechanismen werden nicht automatisch erkannt. Diese müssen auch manuell implementiert werden, indem der Client entsprechend konfiguriert wird. Im Falle der Discord und GitHub API wird dafür der 'Authorization' Header dem Client hinzugefügt. Damit ist die Nutzung von *FastMCP* aufwändiger als bei den zuvor beschriebenen Generatoren, da eine eigene Anwendung erstellt und gepflegt werden muss. Beim verbinden des MCP-Clients mit dem MCP-Server fiel auf, dass das abrufen der Toolliste bei der GitHub, Discord und EVE API deutlich länger dauerste als bei den anderen Generatoren und zunächst zu einem Timeoutfehler in der MCP-Host Anwendung führte. Nach der Erhöhung des Timeouts für MCP Client Anfragen in der Anwendung von den standardmäßigen zwanzig Sekunden auf zwei Minuten konnte die Toolliste aber erfolgreich abgerufen werden. Auch bei diesem Generator wird wieder die OperationId des Endpunkts als Toolname verwendet. Die Beschreibung des Tools wird entweder aus der Description des Endpunkts oder, falls diese nicht vorhanden wird fast das gleiche Schema wie beim *OpenAPI-to-MCP-Generator* verwendet: *Executes [HTTP-Methode] [Pfad]*. Enthält der Endpunkt einen oder mehrere Pfadparameter wird eine kurze Dokumentation dieses Parameters an die Beschreibung angehängt. Auch *FastMCP* generiert für die Tools ein ausführliches InputSchema. Referenzierte Parameter und Datentypsformate werden abgebildet, Enums und Kombinationsschemas werden, wie beim vorherigen Generator, als Listen dargestellt. Defaultwerte, Minimum, Maximum und die notwendigen Parameter werden ebenfalls berücksichtigt und Requestbodies werden im InputSchema dargestellt. Header Parameter werden jedoch nicht im InputSchema dargestellt. Zusätzlich zum InputSchema wird für jedes Tool ein OutputSchema generiert, welches die Struktur des 'structuredContent' Blocks der Antwort des Tools beschreibt. Das 'OutputSchema' wurde in der MPC Version '2025-06-18' eingeführt [59]. Dieses Schema basiert auf den in der OpenAPI Beschreibung definierten Antwortschemas der ersten '200' Antwort des Endpunkts. Auch in diesem Schema werden referenzierte Attribute und Datentypsformate, Enums, Kombinationsschemas und notwendige Parameter berücksichtigt. Darüber hinaus werden auch Beispielwerte im Schema dargestellt. Dieser Generator erlaubt es, neben dem Mapping der Endpunkte zu MCP-Tools, auch die Anpassung des Mappings zur MCP Komponente [60]. So kann ein Endpunkt beispielsweise nicht nur als Tool sondern auch als 'Resource' oder 'ResourceTemplate' bereitgestellt werden. Dies ermöglicht eine flexiblere Einbindung der API Endpunkte. Diese Funktion wurde in dieser Arbeit nicht weiter untersucht, da die Generatoren möglichst ohne Anpassungen verglichen werden sollten. Für *FastMCP* dauerte die Inbetriebnahme der vier Schnittstellen insgesamt ca. 5,5 Stunden. Der MCP-Server benötigt mit einem verbundenen MCP-Client ohne laufende Aktion durchschnittlich 114,275 MB Arbeitsspeicher laut dem Windows 11 Taskmanager, wobei die GitHub API mit 340 MB am meisten benötigte. Bei der Analyse der erstellten Tools ist aufgefallen, dass bei der Discord API das Schema für den Requestbody bei dem Tool zur Erzeugung eines neuen Threads fehlerhaft ist. Hier wird anstatt dem Schema für den Requestbody, das Schema der Antwort verwendet. Die mit *FastMCP* erstellten MCP-Server konnten in 15 von 56 (26,8%) Fällen die Nutzungsszenarien erfolgreich abschließen. Das erste Szenario der GitHub API konnte nur das GPT Modell

erfolgreich abschließen. Es nutzte dabei einen Toolaufruf und 7036 Zeichen. Das Mistral Modell führte keine Toolaufrufe aus. Das zweite Szenario konnten beide Modelle lösen. Wobei beide zwei Toolaufrufe tätigten. Beim zweiten Versuch nutzte das Mistral Modell nur einen Aufruf, da es das Label für das zu erstellende Issue geraten hat. Die Modelle nutzten dabei um die 5376 zeichen bzw. 2852 Zeichen. Bei den Tests zum dritten Szenario führte das Mistral Modell wieder keine Toolaufrufe aus. Das GPT Modell konnte die korrekte Antwort liefern, benötigte dafür jeweils einen Toolaufruf und 919 Zeichen. Das Mistral Modell führte bei Szenario vier wieder keine Toolaufrufe aus und das GPT Modell konnte mit einem Toolaufruf und 299957 Zeichen die korrekte Antwort liefern. Die Versuche zur Discord API verliefen alle erfolglos. Das GPT Modell versuchte in Szenario eins mit jeweils drei Toolfaufrufen und 191 Zeichen die Lösung zu finden, scheiterte aber bei der Abfrage der Nachrichten an einem falschen Parameterschema. Das Mistral Modell führte wieder keine Toolaufrufe aus. Wie zu erwarten führten die Versuche zur Erstellung eines Threads in Szenraio zwei aufgrund des falschen Requestbody Schemas zu Fehlern bei beiden Modellen. Das Mistral Modell versuchte dabei einen Server mit einer zufälligen Id zu verwenden. Szenario drei verhält sich wie Szenario eins. Das GPT Modell führte zwei Toolaufrufe mit jeweils 191 Zeichen, der Fehlermeldung, durch, scheiterte aber wieder am falschen Parameterschema. Das Mistral Modell führte keine Toolaufrufe aus. Die Abfrage der Anzahl der Schiffe konnten beide Modelle nicht richtig beantworten. Das GPT Modell führte zawr die richtigen Toolaufrufe mit den richtigen Parametern durch, konnte aber die korrekte Antwort nicht liefern. Es nutzte dabei jeweils 8635 Zeichen und 49 Toolaufrufe. Das Mistral Modell führte beim ersten Versuch einen Toolaufruf durch, nutzte dabei 279 Zeichen, und führte dann keine weiteren suchen mehr durch. Der zweite Versuch führte wieder keine Toolaufrufe aus. Das zweite Szenario konnten beide Modelle richtig beantworten. Das GPT Modell nutzte dabe jeweils 77 Zeichen und einen Toolaufruf. Das Mistral Modell nutzte auch 77 Zeichen aber jeweils zwei Tools. Bei den Tests zu den ersten drei Szenarien der Invoice Manager API konnte kein Modell eine Antwort liefern, da es bei den Toolfaufrufen zu Fehlern bei den Datumsangaben der gelieferten Antworten kam. Es wurden zwar die richtigen Toolaufrufe mit den richtigen Parametern durchgeführt, aber durch die Fehler wurden keine korrekten Antworten geliefert. Das vierte Szenario wurde teilweise richtig gelöst. Das GPT Modell konnte die erforderlichen Informationen für die Erstellung einer Rechnung zusammentragen, scheiterte aber an der Erstellung der Rechnung selbst. Die Erstellung der Rechnung wurde dann mehrfach wiederholt, was zu insgesamt 11 bzw. 12 Toolaufrufen und 442 bzw. 448 genutzten Zeichen führte. Das Mistral Modell verhielt sich ähnlich. Es konnte die notwendigen Informationen zusammentragen, scheiterte aber ebenfalls an der Erstellung der Rechnung, wiederholte diese aber nicht. Es führte dabei insgesamt 5 bzw. 6 Toolaufrufe durch und nutzte 939 bzw. 944 Zeichen. Das letzte Szenario schlug beim GPT Modell wieder aufgrund des Datumsfehlers fehl. Es führte dabei die notwendigen vier Toolaufrufe durch und nutzte dabei jeweils 392 Zeichen. Das Mistral Modell führte wieder keine Toolaufrufe aus.

4.3.5 Tyk API to MCP

Der nächste Generator ist *Tyk API to MCP* (Tyk) [53]. Dieser Generator ist Teil der kommerziellen API Management Plattform Tyk von Tyk Technologies. Der Generator ist in die Plattform integriert und ermöglicht die Generierung von MCP-Servern aus OpenAPI Beschreibungen, die in der Plattform verwaltet werden. Der Generator kann aber auch separat genutzt werden. Der Quellcode des Generators ist auf GitHub öffentlich zugägnglich und steht dort unter der MIT Lizenz. Das Projekt hat 34 Sterne (Stand 03.01.2026) und wurde zuletzt im Juli 2025 aktualisiert. Obwohl das Projekt öffentlich zugägnlich ist, wird es aufgrund der zugehörigen kommerziellen Plattform hier als kommerzieller Generator betrachtet. Bei Tyk handlt es sich um einen dynamischen Generator. Die Nutzung erfolgt wie beim *OAS to MCP Generator* über die Kommandozeile mit dem Befehl *npx @tyk-technologies/api-to-mcp*. Dieser bekommt als

Option eine URL oder einen Pfad zu einer API Beschreibung und startet damit einen STDIO MCP-Server. Andere Trasnports werden nicht unterstützt. Über weitere Optionen kann der MCP-Server weiter konfiguriert werden. Zu den möglichen Optionen gehört das Setzen einer Basis-URL für die API, eine White- und Blacklist für Endpunkte, die Angabe einer Overlay Datei zur Anpassung der API Beschreibung, weitere Header und die Angabe des zu verwendenden Authentifizierungsmechanismus und dessen Token. Wahlweise kann die Konfiguration über Umgebungsvariablen, eine JSON-Datei oder direkt als Optionen im Startkommando angegeben werden. Als Toolname dient wieder die OperationId der Endpunkte. Bei diesem Generator wird auch die Beschreibung des Tools aus der OperationId genutzt. Dabei wird eine camelCase OperationId bei den Großbuchstaben durch ein Leerzeichen getrennt. Dies tritt auch bei APIs auf, die eine Beschreibung (description) und eine Zusammenfassung (summary) für Endpunkte bereitstellen. Im gegensatz zu zu den anderen Generatoren wird hier im InputSchema ein zusätzliches Property 'additionalProperties' mit dem Wert false hinzugefügt. Dies bedeutet, dass keine zusätzlichen Parameter übergeben werden dürfen, die nicht im InputSchema definiert sind. Pfad- und Queryparameter, auch referenzierte, werden im InputSchema dargestellt. Die Parameter enthalten einen Datentyp und die Beschreibung des Parameters. Darüber hinausgehende Informationen wie Datentypsformate, Enums, Minimum, Maximum und Defaultwerte werden jedoch nicht berücksichtigt. Die Berücksichtigung von Kombinationsschemas kann nicht beurteilt werden, da diese in den analysierten OpenAPI Beschreibungen nur in den RequestBodies verwendet werden. Der Requestbody wird im InputSchema aber nur minimal dargestellt, indem ein zusätzliches Property 'requestBody' vom Typ 'string' und mit der Beschreibung 'Parameter: requestBody' hinzugefügt wird. Dieses Property wird den anderen Parametern in der Liste der Pflichtparameter hinzugefügt. SecuritySchemes werden erkannt und können wie bereits erläutert über Optionen oder Umgebungsvariablen konfiguriert werden. So werden bei der Discord API zwei Security-Schemes erkannt: BotToken und OAuth2. Für die nachfolgenden Tests wurde der BotToken Mechanismus verwendet, indem der Token als Option übergeben wurde. Problematisch ist die GitHub API, welche keine SecurityScheme Definition in der OpenAPI Beschreibung besitzt. Eine manuelles hinzufügen eines 'Authorization' Headers funktionierte auch nicht. Daher konnte auch hier keine Authentifizierung durchgeführt werden.

Tyk benötigt für die Inbetriebnahme der vier Schnittstellen insgesamt ca. 4,5 Stunden und verbraucht laut dem Windows 11 Taskamanager ca 76 MB Arbeitsspeicher. Mit *Tyk* konnten 17 von 56 (30,3%) Nutzungsszenarien erfolgreich abgeschlossen werden. Das erste Szenario konnte keines der Modelle abschließen. Das Mistral Modell führte keine Toolfaufrufe aus und das GPT Modell führte das richtige Tool aus verwendete aber den Paging Parameter *per_page* mit dem Wert '1' was zu einer falschen Antwort führte. Es nutzte dabei jeweils 2532 Zeichen. Beim zweiten Szenario, erstellen eines Issues, führten die Modelle zwar das Tool zur Erstellung aus. Dieses schlug aber aufgrund der fehlenden Authentifizierung fehl. Das dritte Szenario wurde korrekt vom GPT Modell beantwortet. Es führte dabei einen Toolaufruf aus und nutzte jeweils 455 Zeichen. Das Mistral Modell führte wieder keine Toolaufrufe aus. Das vierte Szenario scheiterte wieder an der fehlenden Authentifizierung. Auch beim ersten Szenario der Discord API konnte das Mistral Modell keine Toolaufrufe ausführen. Das GPT Modell führte im ersten Versuch drei Tools aus und konnte mit 3671 Zeichen das richtige Ergebnis liefern. Im zweiten Versuch scheiterte es an einer falschen guild_id. Beim zweiten Szenario führten beide Modelle die notwendigen Tools aus konnten aber aufgrund des fehlerhaften Requestbody Schemas keinen Thread erstellen. Das GPT Modell nutzte 2329 Zeichen in beiden Versuchen, das Mistral Modell 2308 und 2327 Zeichen. Beim dritten und letzten Szenario der Discord API konnte das GPT Modell in beiden Versuchen das richtige Ergebnis liefern. Es führte dafür jeweils zwei Toolaufrufe aus, die 1155 Zeichen nutzten. Das Mistral Modell führte wieder keine Toolaufrufe aus. Im ersten Szenario der EVE Online API führte das Modell 50 Toolfaufrufe aus, da es den ersten Aufruf wiederholte. Es konnte mit den jeweils 8635 Zeichen kein richtiges Ergebnis liefern. Auch das Mistral Modell führte den

ersten Toolfaufruf doppelt aus, führte aber keine weiteren Toolaufrufe durch, wodurch es kein richtiges Ergebnis liefern konnte. Es nutzte dabei 279 Zeichen. Im zweiten Szenario konnte das GPT Modell kein Ergnis erzielen, da es bei der Ausführung einen Fehler mit einem Parameter hatte. Das Mistral Modell konnte zwar einen Toolaufruf durchführen, nutzte aber das falsche Tool und wiederholte den Aufruf zweimal. Dafür wurden 4908 Zeichen genutzt. Szenario eins der Invoice Manager API konnte beide Modelle mit einem Toolaufruf und jeweils 2022 Zeichen korrekt lösen. Dasselbe gilt für das zweite Szeanrio zur Ermittlung der Anzahl der bezahlten Rechnungen. Beide Modelle nutzten jeweils einen Toolaufruf und 662 Zeichen und nutzten den Parameter 'paid' mit dem Wert 'true'. Das dritte Szenario konnte wieder nur das GPT Modell lösen. Es nutzte in beiden Tests zwei Toolaufrufe mit 889 Zeichen. Das Mistral Modell führte wieder keine Toolaufrufe aus. Szenario vier wurde wie zu erwarten Aufgrund der fehlenden Requestbody Definition von beiden Modellen nicht gelöst. Beide Modelle nutzten dabei nicht alle notwendigen Tools und versuchten die Rechnung mehrfach zu erstellen. Beim herunterladen der Rechung, Szenario fünf, führte das Mistral Modell wieder keine Toolaufrufe aus. Das GPT Modell konnte die Rechnung in beiden Versuchen korrekt herunterladen. Es führte dafür vier Toolaufrufe durch und nutzte 4844 und 4843 Zeichen.

4.3.6 Stainless

Der letzte untersuchte MCP-Server Generator ist wieder ein kommerzieller Generator von der Firma Stainless Software, Inc (Stainless) [61]. Stainless ist ein Service zur Erstellung von Server Development Kits (SDKs) aus OpenAPI Beschreibungen. Neben SDKs für verschiedene Programmiersprachen bietet Stainless auch die Möglichkeit an, MCP-Server aus OpenAPI Beschreibungen zu generieren. Dies ist jedoch nur für die Sprache TypeScript möglich. Stainless bietet eine kostenfreie Version für bis zu fünf SDKs an, was für die Analyse der vier APIs ausreicht [62]. Die Nutzung des Generators erfolgt über die Weboberfläche von Stainless. Nach der Aneldung über GitHub oder eines anderen SSO Providers kann dort ein neues Projekt erstellt werden. In diesem Projekt kann dann eine OpenAPI Beschreibung über eine URL oder durch Hochladen einer Datei hinzugefügt werden. Aus dieser Beschreibung wird dann eine *Stainless Config* im YAML Format erstellt. Diese enthält die Informationen zur API und wird zur Generierung der SDK verwendet. Der MCP-Server wird dann anhand der Config generiert und verwendet für den Zugriff auf die API das zuvor erstellte SDK. Die API Beschreibung und die erzeugte *Stainless Config* kann im *Stainless Studio* bearbeitet werden, um zum Beispiel Endpunkte zu entfernen oder anzupassen. Die Codegenerierung erfolgt über GitHub Actions. Der generierte Code wird in einem GitHub Repository abgelegt, in das der Nutzer eingeladen wird. Dort kann der generierte COde und auch ein fertig gebautes npm Paket, als Tarball, heruntergeladen werden. Falls konfiguriert, kann das Paket auch automatisch in die npm Registry veröffentlicht werden. Ist der MCP-Server bei npm veröffentlicht, kann er wie bei den vorherigen Generatoren einfach über die Kommandozeile mit dem Befehl *npx <paketname>* gestartet werden. Diese Veröffentlichung bei npm ist im Rahmen dieser Arbeit nicht konfiguriert worden. Für die nachfolgenden Tests und Analyse des generierten Codes wird der Quellcode aus dem GitHub Repository verwendet. Der generierte Code enthält ein Node.js Projekt mit entsprechender TypeScript Konfiguration, package.json, Markdowndateien als Dokumentation und einem src Ordner mit dem Quellcode der SDK. Dieser wird in dieser Arbeit nicht weiter betrachtet. Zusätzlich zum src Ordner enthält das Projekt ein Unterprojekt das den MCP-Server enthält. Dieser befindet sich im Ordner 'packages/mcp-server' und enthält ebenfalls eine package.json Datei und einen src Ordner mit dem Quellcode des MCP-Servers. Der src Ordner des MCP-Servers enthält verschiedene TypeScript Dateien die allgemein für den MCP-Server notwendig sind, wie zum Beispiel Dateien für die verschiedenen Transports, von denen STDIO und Streamable-http unterstützt werden. Für die Streamable-http Variant besteht neben einem HTTP-Port auch noch die Möglichkeit

den MCP-Server auf einem Unix-Socket zu starten. Außerdem enthält der Ordner Dateien für zwei der drei Betriebsmodi dieses MCP-Servers. Neben dem Standardmodus, bei dem jeder API Endpunkt als ein Tool bereitgestellt wird, gibt es noch zwei weiter Modi. Der 'Dynamic'-Modus, stellt drei allgemeine MCP-Tools bereitstellt, einen zum Abfrufen aller API Endpunkte, einen zum Abfragen des Schemas eines Endpunktes und einen zum Ausführen eines Endpunktes. Der dritte Modus ist der 'Code'-Modus. In diesem Modus kann das LLM Code zur Interaktion mit der SDK an den MCP-Server schicken, welcher dann vom MCP-Server in einer Sandboxumgebung ausgeführt wird. Das Ergebnis wird dann als Antwort an das LLM zurückgegeben. Dieser bietet nur zwei Tools an, einem zum Durchsuchen der API Dokumentation und einen zum Ausführen des Codes. Beide Modi sollen den den benötigten Context für das LLM reduzieren, indem die Anzahl der Tools reduziert wird. Beide werden später anhand eines Szenarios getestet. Der Betriebsmodus, das Transport, eine Filterung der Endpunkte und API-Keys können über Optionen im Startkommando des MCP-Servers konfiguriert werden. Neben diesen allgemeinen Dateien enthält der src Ordner des MCP-Servers auch noch einen 'tools' Ordner. Darin befinden sich nach 'Resource', dem hierarchie-Level entsprechendem Teil des Endpunktpfads, gruppierte Ordner, die wiederum TypeScript Dateien für jeden Endpunkt enthalten. So erzeugt der Pfad 'invoices/invoiceId' die Ordnerstruktur '/invoices' und der Pfad 'invoices/positions/positionId' die Ordnerstruktur '/invoices/positions'. In dem Ordner enthalten sind dann die möglichen Aktionen für diesen Endpunkt als TypeScript-Datei, welche das jeweilige Tool darstellen. Für den Toolnamen wird dieses mal nicht die OperationId verwendet, sondern eine Bezeichnung der Aktion aus der *Stainless Config*. Diese Bezeichnug beschreibt die Aktion der jeweiligen Resource, wie zum Beispiel 'list_repos_issues' für die Auflistung aller Issues eines Repository, 'retrieve_repos_issues' für das Abrufen eines einzelnen Issues oder 'create_repos_issues' für das Erstellen einer neuen Issues. Als Beschreibung dient die Description des Endpunkts aus der OpenAPI Beschreibung. Der Fall das diese fehlt kann nicht beuerteilt werden, da dies nur in der Discord API vorkommt, diese aber aufgrund von nicht bekannten Fehlern bei der Codegenerierung nicht erstellt werden konnte. Eine angepasste Variante der Invoice Manager API ohne Beschreibungen führte zu einem Toollosen MCP-Server. Das generierte InputSchema enthält Pfad- und Queryparameter, auch wenn diese in der OpenAPI Beschreibung nur referenziert sind. Die Parameter enthalten den angegebenen Datentyp, das Format, die Beschreibung und falls notwendig die Liste der möglichen Werte bei Enums. Zusätzliche Informationen wie Minimum, Maximum, Defaultwerte und Beispiele werden jedoch nicht angegeben. Kombinationsschemas werden wieder als Liste von möglichen Schemas dargestellt. Ein Requestbody wird dargestellt, indem die einzelnen Properties des Requestbodys als Properties im InputSchema hinzugefügt werden. Die notwendigen Parameter werden über die 'required' Liste im InputSchema angegeben. Zusätzlich findet sich bei vielen ein zusätzliches Property 'jq_filter'. Dieses Property ist optional und vom Typ 'string'. Es enthält eine Beschreibung die darauf hinweist, dass hiermit ein JQ Filter [63] angegeben werden kann, um die Antwort der API zu filtern. Damit kann die Antwort der API auf die für das LLM relevanten Daten reduziert werden, um den verwendeten Context zu verringern. *Stainless* verwendet als einziger getesteter Generator die 'ToolAnnotations', welche in der Version 2025-03-06 des MCP Protokolls hinzugefügt wurden [64]. Entsprechden Tools werden mit dem Hint 'ReadOnlyHint', der angibt dass das Tool keine Änderung am System vornimmt, oder 'IdempotentHint', der angibt dass das Tool mit den selben Parametern mehrfach mit dem selben Ergebnis ausgeführt werden kann, annotiert. Die Datei der jeweiligen Tools enthält zu der eigentlichen Tooldefinition auch eine Funktion zur Ausführung des Tools. In dieser Funktion ist die Verwendung der SDK zu sehen, sowie die Fehlerbehandlung bei der die Fehlermeldung als Antwort zurückgegeben wird. Aufgrund einer langen Einarbeitungsphase in die Stainlessplattform, Probleme mit der Code Generierung der Discord API und Schwierigkeiten beim bauen und starten des MCP-Servers dauerte die Inbetriebnahme der drei funktionierenden Schnittstellen insgesamt ca. 16 Stunden. Da der Buildprozess des Projektes für die Ausführung als

GitHub Workflow implementiert ist, musste die Anwendung in einem Docker Container, welcher als Devcontainer mitgeliefert wird, gebaut und gestartet werden. Daher stammen die Angaben zum Arbeitsspeicherverbrauch dieses mal von der Anwendung htop [65] im Docker Container. Der MCP-Server benötigt mit einem verbundenen MCP-Client ohne laufende Aktion durchschnittlich 93,6 MB Arbeitsspeicher. Mit dem Stainless MCP-Server konnte eine Erfolgsquote von 28 von 56 (50%), oder 63,4% ohne die Discord Szenarien, erreicht werden. Das erste Szenario der GitHub API konnte nur das GPT Modell erfolgreich abschließen. Es nutzte dabei einen Toolaufruf und 8579 Zeichen. Das Mistral Modell führte keine Toolaufrufe aus. Das zweite Szenario, für die Erstellung eines Issues, konnte beide Modelle lösen, jedoch wurde beiden Issues kein Label hinzugefügt oder welche abgefragt. Beide Modelle führten dazu einen Toolaufruf durch. Das GPT Modell nutzte dabei 3169 und 3337 Zeichen, das Mistral Modell 3246 und 3303 Zeichen. Beim dritten Szenario führten beide Modelle Toolaufrufe aus und lieferten die korrekte Antwort, jedoch konnte das Mistral Modell nicht anhand der Antwort erkennen, dass es die richtige Anzahl an Regeln ermittelt hatte. Es antwortete vermutlich nur mit der Anzahl der ermittelten Branches. Da es auch nur einen Toolaufruf durchführte, obwohl drei notwendig gewesen wären. Das GPT Modell nutzte dafür die drei Tools und jeweils 3273 Zeichen, das Mistral Modell 1172 Zeichen. Beim vierten Szenario konnte das GPT Modell wieder die erwartete Antwort liefern. Es nutzte dafür ein Tool und 34281 Zeichen. Das Mistral Modell lief auf einen Fehler mit einem zu großem Input und führte keine Toolaufrufe aus. Die Versuche der GitHub API wurden mit einem gültigen Personal Access Token (PAT) durchgeführt, welcher die notwendigen Berechtigungen für die Szenarien besitzt. Da bei den Versuchen zur EVE Online API keine Authentifizierung notwendig ist, wurde kein Token angegeben. Dies führte jedoch zu Problemem, da der MCP-Server einen leeren 'Authorization' Header an die API Anfragen anhängte. Zudem gab es Probleme mit dem Parameter 'X-Compatibility-Date'. Daher konnte kein Szenario erfolgreich abgeschlossen werden. Beim ersten Szenarion der Invoice Manager API konnte das GPT Modell beide male die korrekte Antwort liefern. Es nutzte dafür beide male einen Toolaufruf. Beim ersten mal wurde keine Parameter verwendet, was zu 3076Zeichen führte. Beim zweiten mal wurde der Parameter 'jq_filter' mit dem Wert 'length' verwendet, was zu 14 Zeichen führte. Das Mistral Modell führte keine Toolaufrufe aus und antwortete mit einer belieben Anzahl an Rechnungen. Das zweite Szenario konnten beide Modelle richtig beantworten. Sie nutzenbeide einen Toolfaufruf und den Parameter 'paid' mit dem Wert 'true' und den 'jq_filter' zur Ermittlung der Länge der Liste der Antwort. Die Modelle nutzten jeweils 14 zeichen. Auch das dritte Szeanrio konnten beide Modelle lösen. Das GPT Modell nutzte dafür jeweils zwei Toolaufrufe und den 'jq_filter' und benötigte damit nur 31 Zeichen. Das Mistral konnte durch den Einsatz des 'jq_filter' die Rechnungen direkt filtern ohne zuvor die KundenId ermitteln zu müssen und konnte dadurch einen Toolaufruf einsparen. Es nutzte damit nur 14 Zeichen. Das vierte Szenario konntem auch beide Modelle erfolgreich abschließen. Sie benötigten dafür jeweils funf Tools, das GPT Modell im zweiten Versuch sechs, und nutzten dabei 288 und 313 Zeichen beim GPT Modell und 879 und 842 beim Mistral Modell. Auch hier wurde der 'jq_filter' verwendet um zum Beispiel eine Liste von Kunden und deren Ids zu erstellen. Das letzte reguläre Szenario konnten die Modelle mit vier Aufrufen, sechs beim ersten Versuch des Mistral Modells, erfolgreich abschließen. Das GPT Modell nutzte dabei 4287 und 4095 Zeichen, das Mistral Modell 7391 und 3831 Zeichen. Zusätzlich wurden einige Szenarien der Invoice Manager API ohne 'jq_filter' Parameter oder mit einem anderen Betriebsmodus getestet. Szenario vier, Erstellung einer neuen Rechnung, benötigte ohne 'jq_filter' 1884 Zeichen beim GPT Modell was eine Steigerung von 1571 Zeichen im Vergleich zu dem Versuch mit 'jq_filter' darstellt. Im 'Dynamic' Betriebsmodus war es dem GPT Modell nicht möglich ein Tool aufzurufen, da es falsche Parameter übergab. Zuletzt wurde der 'Code' Modus wieder mit Szenario vier getestet. Dies führte zu einem unbekannten Fehler bei der Ausführung des Codes im MCP-Server. Daher konnte das Szenario nicht abgeschlossen werden. Der vom Modell erzeugte Code ist in Listing 4.2 zu sehen. Man sieht dort wie ein Datum

erzeugt, eine Rechung erstellt und mit der Id der erzeugten Rechnung eine Position erstellt werden sollte.

Listing 4.2: Beispielcode, welcher von dem GPT Modell im 'Code' Modus des Stainless MCP-Servers generiert wurde.

```
async function run(client) {
        const today = new Date().toISOString().split('T')[0];
  // Step 1: Create the invoice with default values and the given
   customer and service date
  const invoice = await client.invoices.create({
    customerNumber: 'Musterfirma', // Assuming customerNumber can take
   the name, otherwise this needs the real customer number
    serviceFrom: today
  });  // Step 2: Create the invoice position
  const position = await client.invoicePositions.create({
    invoice: invoice.id,
    pricePerUnitInCents: 1000, // $10 = 1000 cents
    quantity: 100,
    unit: 'pieces',
    description: 'Materials'
  });
  return invoice.id;
}
```

Eine Übersicht über die von den einzelnen Generatoren unterstützten OpenAPI Specification unterstützten Features ist in Tabelle 4.2 zu finden.

Die Durchführung der Nutzungsszenarien mit den verschiedenen MCP-Server Generatoren hat gezeigt, dass die Qualität der generierten MCP-Server stark variiert. Die besten Ergebnisse lieferte der *Stainless* MCP-Server mit 63,4% trotz der Probleme bei der Generierung der Discord API. Danach folgt *OpenApi-to-MCP Generator* (OASMCP) mit 33,9%, *Tyk* mit 30,3%, *FastMCP* mit 26,8%, *ConvertMCP* mit 19,6% und zuletzt *MCPLink* mit 7,14%. Nach Schnittstellen gruppiert liegt die Invoice Manager API mit einer Quote von 46,7% an erster Stelle, gefolgt von der GitHub API mit 30,2%, der EVE Online API mit 14,6% und der Discord API mit 4,2%. Bei der Invoice Manager API gab es Mit *ConvertMCP* ein Problem bei der Generierung, da hier ein falsches InputSchema erzeugt wurde. Dieser Fehler trat auch bei den anderen Schnitsstellen auf, da die Invoice Manager API aber sonst keine Probleme hatte, konnte das falsche Schema korrigiert und der Generator an zumindest einer API getestet werden. Bei zwei Szenarien wurden nicht alle erforderlichen Tools verwendet und eine fiktive Kundennummer verwendet. Drei weitere fehlgeschlagene Tests schlugen Aufgrund von Fehlern mit Parametern fehl. Diese Fehler können auf ein mangelhaftes InputSchema zurückgeführt werden. Acht weitere Tests stießen auf Probleme bei der Validierung der Antwort der API. Diese Probleme deuten auf Probleme mit dem OutputSchema hin, welches in den meisten Generatoren, außer *FastMCP* nicht berücksichtigt wird. Auch bei der GitHub API hat *ConvertMCP* ein falsches InputSchema generiert. Hierzu kamen noch Probleme mit mehrzeilugen Beschreibungen von Parametern, was die Nutzung vollständig verhinderte. Weitere Fehler bei den Tests sind auf fehlerhafte Parameter bei Enums zurückzuführen. Weitere acht Tests schlugen aufgrund der fehlenden Dokumentation der Authentifizierung fehl. Die EVE Online API hat das bereits bekannte Problem mit *ConvertMCP*. Zusätzlich wurden hier referenzierte Parameter zwar erkannt, aber nicht aufgelöst. Dies führte zu vielen Parametern mit dem Namen 'undefined'. Fünf Tests schlugen wieder aufgrund von Parametern fehl. Erwähnenswert ist noch ein Pronlem mit *Stainless*. Hier wird eine Authentifizierungsmethode erkannt und angewandt, obwohl diese für die Endpunkte nicht notwendig und auch nicht konfiguriert ist. Dies führte zu fehlerhaften Anfragen an die API. Die Discord API, welche im JSON-Format vorliegt, konnte von zwei Generatoren nicht verarbeitet werden. Einmal *ConvertMCP* wegen der bekannte Probleme

OAS Funktion	MCPLink	ConvertMCP	OASMCP	FastMCP	Tyk	Stainless
Toolbeschreibung aus Description	✓	✓	✓	✓	✗	✓
Parameterbeschreibung	✗	✓	✓	✓	✓	✓[1]
Pfadparameter	✓	✓	✓	✓	✓	✓
Queryparameter	✓	✓	✓	✓	✓	✓
Parametertypen	✓[2]	✗	✓	✓[3]	✓	✓[3]
Requestbody	✓	✗[4]	✓	✓	✗[5]	✓
Min/Max	✗	✗	✗	✓	✗	✗
Referenzen	✗	✗	✓	✓	✓	✓
Default Werte	✗	✗	✓	✗	✗	✗
Example Werte	✗	✗	✗	✓	✗	✗
Enums	✓	✗	✓	✓	✗	✓
Kombinationsschemas	-[6]	✗	✓	✓	-[6]	✓
Required	✓	✓[7]	✓	✓	✓	✓
Security	✓[8]	✗	✓[9]	✓[10]	✓	✓[11]

Tabelle 4.2: Unterstützte OAS Features der Generatoren

[1] Nicht bei Pfadparametern
[2] Vorhanden aber falsch
[3] Mit Format
[4] Nur erkannt, nicht dargestellt
[5] Nur erkannt, mit additionalProperties: true
[6] Nicht einsehbar, da nur in Body
[7] Falsche Darstellung
[8] Generisch über Header
[9] Mit OAuth
[10] Nicht automatisch, manuelle Implementierung
[11] Nur über Header

und von *Stainless*. Bei letzterem schlug die Codegenerierung aufgrund eines Fehlers bei mit einem 'Linter' zur Codeformatierung fehl. Acht Tests mit *MCPLink* und *OASMCP* schlugen fehl, da diese Generatoren die Pfadparameter nicht darstellten. Die Szenarien zur Erstellung eines Threads konnten mit *Tyk* nicht durchgeführt werden, da es hier keine Beschreibung des Requestbodys im InputSchema gab. Desweiteren gab es bei *FastMCP* ein fehlerhaftes Schema für den Requestbody bei der Erstellung eines Threads, welches das Schema der Antwort enthielt und einmal eine fehlende Toolnutzung zur Ermittlung einer Id. Für die Fehlversuche bei denen das Mistral Modell keine Toolaufrufe ausführte, kann kein direkter Zusammenhang mit den Generatoren hergestellt werden. Mit 29 fehlgeschlagenen Tests waren Fehler bei Parametern des MCP Tools die häufigste Fehlerquelle, dem folgen 24 Fehler des Mistralmodells, 12 Fehler bei der Authentifizierung, 8 Fehler bei der Validierung der Antwort fünf fehlgeschlagene Generierungen und vier Fehler bei der Nutzung von Tools.

Nach der Analyse der Testergebnisse wurden die Generatoren noch hinsichtlich der Spezifikationen des Model Context Protocol für MCP-Server untersucht. Diese Spezifikationen beschreiben Funktionen (Capabilities), die ein MCP-Server unterstützen kann und wie diese implementiert werden sollten. Die Untersuchung bezieht sich auf die momentan (Stand Januar 2026) neueste Version des MCP Protokolls 2025-11-25 [66] [67]. Für die Feststellung der Funktionen wurde, sofern vorhanden, der Quellcode und die Dokumentation der Generatoren untersucht. Die Ergebnisse der Untersuchung sind in Tabelle 4.3 zusammengefasst.

An der Tabelle 4.3 ist zu erkennen, dass die meisten Generatoren nur die grundlegendste Funktion eines MCP-Servers unterstützen. Die Ausführung von Tools. Einzig *FastMCP* bietet die Möglichkeit, über ein manuelles Mapping, API Endpunkte als Resources bereitzustellen.

Capability	MCPLink	ConvertMCP	OASMCP	FastMCP	Tyk	Stainless
Prompts	✗	✗	✗	✗	✗	✗
Resources	✗	✗	✗	✓[1]	✗	✗
Completions	✗	✗	✗	✗	✗	✗
Logging	✗	✗	✗	✓	✗	✓
Pagination	✗	✗	✗	✗	✗	✗
Cancellation	✗	✗	✗	✗	✗	✗
Ping	✗	✗	✗	✗	✗	✗
Progress	✗	✗	✗	✓	✗	✗
Tasks	✗	✗	✗	✗	✗	✗
Tools	✓	✓	✓	✓	✓	✓
Tool InputSchema	✓	✓	✓	✓	✓	✓
Tool OutputSchema	✗	✗	✗	✓	✓	✗
Tool Structured Output	✗	✗	✗	✓	✗	✗
Tool ContentTypes	✗	✗	✗	✗	✗	✗
Tool Annotations	✗	✗	✗	✗	✗	✓
Tool Security	✗	✗	✗	✗[2]	✗	✗
Tool Error Handling	✓	✗	✗	✓	✓	✓

Tabelle 4.3: Unterstützte MCP-Server Capabilities der Generatoren

1: Über manuelles Mapping
2: Nur Inputparametervalidierung, Rate-Limiting über Middleware

Besonders auffällig ist das kein Generator die von der MCP Spezifikation definierten Vorgehen für Sicherheit [68] und berücksichtigt. Die entsprechende Fehlerbehandlung [69] berücksichtigen nur vier von sechs Generatoren, wobei *ConvertMCP* und *OASMCP* komplett versagen. In der Spezifikation wird beispielsweise empfohlen, dass Eingabeparameter entsprechend validiert werden sollten oder Fehlermeldungen mit einer spezifischen JSON-RPC Struktur zurückgegeben werden sollten. Dies verschlechtert zum einen die Möglichkeit des LLM bei Parameterfehlern einen korrigierten Aufruf zu generieren, zum anderen birgt es Sicherheitsrisiken, da eventuell schadhafte Parameter nicht entsprechend behandelt werden.

4.4 Manuelle MCP-Server Implementierung

Um die Ergebnisse der Generatoren vergleichen zu können, werden im nächsten Schritt MCP-Server für die vier Schnittstellen manuell implementiert. Die Implementierung wird mit der Programmiersprache Python in der Verson 3.12 und dem offiziellen Python MCP SDK von Anthropic [36] in der Version 1.25.0 durchgeführt, in welches die erste Version von FastMCP integriert wurde. Daher können bei der Implementierung importe von FastMCP auftauchen, dass aber nicht direkt benutzt wird. Für die Anbindung der APIs werden die vorliegenden OpenAPI Beschreibungen genutzt, um mit der OpenAPI CLI Python Module für die APIs zu generieren. Diese Module erlauben dann einen komfortablen Zugriff auf die APIs über Python Funktionen, welche die Endpunkte der APIs repräsentieren. Die Generierung der Module erfolgt dabei über die OpenAPI CLI welche in einem Docker Container verwendet wird. Listing 4.3 zeigt beispielhaft den Befehl zur Generierung der GitHub API.

Listing 4.3: Befehl zur Generierung der API Module mit der OpenAPI CLI

```
docker run --rm -v "${PWD}:/local" -e JAVA_OPTS="-DmaxYamlCodePoints
   =10000000" openapitools/openapi-generator-cli generate -g python --
```

```
additional-properties=library=httpx,packageName=github_client,
generateSourceCodeOnly=true --global-property=apiTests=false,
modelTests=false,apiDocs=false,modelDocs=false -i https://raw.
githubusercontent.com/TTomczek/apis/refs/heads/main/github/api.github
.com.yaml -o /local/src
```

Um die Effektivität der einzelnen Optimierungen der manuellen MCP-Server zu testen, werden die Server stufenweise um die Optimierungen erweitert. Wobei die nächste Stufe immer auf der vorherigen aufbaut. Zunächst wird ein Ansatz analog zu den MCP-Generatoren angewandt, bei dem jeder API Endpunkt als ein Tool bereitgestellt wird. Diese Tools werden dann weiter optimiert. Zuletzt werden dann eigenständige Tools implementiert, welche nicht direkt einem API Endpunkt entsprechen, sondern mehrere Endpunkte kombinieren oder zusätzliche Funktionalitäten bieten, um die Nutzung der API durch das LLM zu vereinfachen. Folgende Stufend er Optimierung werden für die Tools getestet.

- Stufe 1a: Keine Optimierung. Reines Abbilden der API Endpunkte als Tools.
- Stufe 1b: Hinzufügen von MCP Best Practices wie Rate-Limiting, Validierung, Fehlerbehandlung und Säuberung nach MCP Spezifikation.
- Stufe 2: Anpassen und Hinzufügen der Beschreibungen der Tools, sowie deren Parametern.
- Stufe 3: Definition des OutputSchema und Rückgabe der Daten als strukturierte Daten anstatt als unstrukturierter Text.
- Stufe 4a: Hinzufügen eines zusätzlichen 'jq_filter' Parameters, welcher die Möglichkeit bietet, die Antwort der API mit einem JQ Filter zu filtern, um den Context für das LLM zu reduzieren, zunächst ohne OutputSchema.
- Stufe 4b: Erweiterung von Stufe 4a um ein OutputSchema, um die Nutzung von 'jq_filter' zu unterstützen.
- Stufe 5: Umformung der Toolantwort in das 'TOON'-Format [70], um die Antwort der API besser für die Nutzung durch das LLM zu optimieren.
- Stufe 6: Eigenständige Tools

Die definierten Optimierungen sind darauf ausgelegt dem LLM die Richtige Nutzung der Tools zu erleichtern, indem die Informationen die zu einem Tool zur Verfügung stehen verbessert werden und die Antworten an das LLM effizienter dargestellt werden, um den verwendeten Context zu reduzieren. Wie auch zuvor werden die, am Anfang der Arbeit definierten, Metriken anhand der Nutzungsszenarien für die vier Schnittstellen erhoben, um die Effektivität der Optimierungen zu bewerten. Da der zeitliche Rahmen dieser Arbeit begrenzt ist, können nicht alle Endpunkte der APIs in die Implementierung einbezogen werden, wie es bei den Generatoren möglich ist, sondern nur die Endpunkte, welche für die definierten Nutzungsszenarien notwendig sind.

Teil a der Stufe 1 ist die Basis für den MCP-Server, indem die API Endpunkte als Tools bereitgestellt werden. Dafür wird zunächst der ein *FastMCP* Objekt erzeugt das den MCP-Server repräsentiert. Dieser wird dann mit der *run* Methode und dem Parameter *transport* und dem zugehörigen Wert 'streamable-http' als Streamable HTTP Server gestartet. Für die Tools können nun Funktionen definiert werden, die die Tools darstellen. Diese können entweder mit der Funktion *add_tool* des MCP-Servers oder mit einem Dekorator definiert werden. In diesem Projekt wird, der Übersicht halber, die Dekorator Variante verwendet. Dabei wird eine Funktion definiert, welche die Logik des Tools enthält, und mit dem *@mcp.tool()* Dekorator versehen, wobei *mcp* der Name des zuvor erzeugten MCP-Server Objekts ist. Der Dekorator erzeugt

die Tooldefinition aus der Funktion. Dafür wird der Funktionsname als Toolname verwendet. Dieser wurde hier aus der OperationId der OpenAPI Beschreibung abgeleitet. Sonderzeichen wie Punkte, Bindestriche oder Schrägstriche wurden dabei durch Unterstriche ersetzt, um die Toolnamen konform mit der MCP Spezifikation zu gestalten. Der Docstring der Funktion wird als Beschreibung des Tools verwendet. Diese wurde, sofern vorhanden, aus der Description des Endpunkts in der OpenAPI Beschreibung übernommen. Das InputSchema des Tools aus den Parametern der Funktion erstellt. Die Tool-Parameter werden aus den API Funktionen der generierten Module übernommen. Handelt es sich bei einem Parameter um ein Objekt, wie zum Beispiel ein *SimpleUser* der GitHub API, wird dieses Objekt als Klasse definiert. Ausnahmen gibt es hier bei besonders komplexen Objekten, wie zum Beispiel die Objekte zum erzeugen eines Threads in der Dscord API. Hier werden dann die Objekte der generierten Module verwendet. Über das *Pydantic* [71] Modul können die Parameter annotiert werden, um weitere Informationen für die Erzeugung des InputSchemas bereitzustellen. Dazu gehören zum Beispiel die Angabe von Wertebereichen für numerische Werte, eine Beschreibung des Parameters, Beispielwerte oder auch 'Pattern' für Strings, um die erwartete Struktur eines Strings anzugeben. Dies bietet zusätzlich den Vorteil, dass die Parameter automatisch validiert werden, bevor das Tool ausgeführt wird, und bei ungültigen Parametern eine Fehlermeldung zurückgegeben wird.

Listing 4.4: Beispiel für Erstellung eines einfachen MCP-Server mit dem SDK.

```
mcp = FastMCP(port=3000)

if __name__ == "__main__":
    mcp.run(transport="streamable-http")

@mcp.tool()
async def get_issues(owner: Annotated[str, Field(description="The␣owner␣
   of␣the␣repository")], repo: Annotated[str, Field(description="The␣
   name␣of␣the␣repository")]):
    """Fetches issues from a GitHub repository."""
    issues = await api.get_issues(owner, repo)
    return issues
```

Authentifizierungsinformationen wie sie für einige Szenarien der GitHub und Discord API benötigt werden, werden über eine Umgebungsvariable an die Anwendung übergeben und bei der Initialisierung der APIs übergeben. Die Logik der Tools besteht zunächst nur darin die erhaltenen Parameter in die Datenstruktur der API Module zu überführen und die entsprechende Funktion auszurufen. Das Ergebnis wird dann als Antwort zurückgegeben. Bis zu dieser Stelle dauerte die Umsetzung insgesamt ca. 19 Stunden. Davon entfallen fünf Stunden aus die Einrichtung des Projekts und die Generierung der API Module. Die Implmentierung der GitHub Tools dauerte eine Stunde, die der Discord Tools 5,75 Stunden, die der EVE Online Tools 2,5 Stunden und die der Invoice Manager Tools 4,75 Stunden. Diese Zeiten enthalten das Testen und beheben von Fehlern der Tools. Die Auslastung des Arbeitsspeichers des MCP-Servers mit einem verbundenen MCP-Client ohne laufende Aktion beträgt durchschnittlich ca. 151 MB laut dem Windows 11 Taskmanager. Das teilt sich auf in ca. 321 MB für GitHub, 143 MB für Discord, 94,5 MB für EVE und 54 MB für die Invoice Manager API. Wie zuvor auch werden pro Nutzungsszenario jeweils zwei Tests pro Modell durchgeführt. Bei den Versuchen zur GitHub API ergab sich eine Erfolgsquote von 12 von 16 (75%), bei der Discord API 6 von 12 (50%), bei der EVE Online API 0 von 8 (0%) und bei der Invoice Manager API 13 von 20 (65%). Die Fehler der GitHub API stammen in dieser Stufe von den Tests mit dem Mistral Modell. Dieses führte bei vier Tests keine Toolaufrufe aus. Bei der Disord API wird verhält es sich genauso, hier führten die Tests mit dem Mistral Modell entweder zu einem Fehler oder zu keiner Toolnutzung. Die Fehler bei der EVE Online API stammen von beiden Modellen. Das GPT Modell konnte im ersten Szenario noch

Toolanfragen ausführen, jedoch nicht das richtige Ergebnis liefern. Bei den anderen Szenarien konnten beide Modelle keine Toolaufrufe durchführen oder es traten Fehler bei den Toolanfragen auf. Beim Invoice Manager gab im ersten Szenario trotz richtigen Toolaufruf dreimal eine falsche Antwort udn nur einmal vom GPT Modell eine korrekte Antwort. Das zweite Szenario konnte von beiden Modellen gelöst werden. Das dritte Szenario konnte vom Mistral Modell aufgrund fehlender Toolaufrufe oder falscher Parameter nicht gelöst werden. Die Erstellung einer neuen Rechnung, Szenario vier, konnte von beiden Modellen richtig durchgeführt werden. Das letzte Szenario, das Erstellen des PDF einer Rechnung, konnte vom GPT Modell gelöst werden, jedoch nicht vom Mistral Modell, da hier Fehler bei den Toolaufrufen auftraten.

Teil b von Stufe 1 soll die Best Practices der MCP Spezifikation für Tools [69][68] implementieren. Dazu gehören zum Beispiel die Validierung der Eingabeparameter, um ungültige Anfragen zu verhindern, die Rückgabe von Fehlermeldungen, um dem LLM die Möglichkeit zu geben, auf Fehler zu reagieren, die Bereinigung der Toolantwort, um potenziell Schadhafte oder vertrauenswürdige Informationen zu entfernen, oder auch die Implementierung von Rate-Limiting, um die API vor Überlastung zu schützen. Diese Funktionen sollen die Sicherheit und Zuverlässigkeit des MCP-Servers verbessern, indem sie sicherstellen, dass nur gültige Anfragen verarbeitet werden, Fehler angemessen behandelt werden und die API vor Missbrauch geschützt wird. Wie in den folgenden Stufen auch, basiert die Implementierung auf der vorherigen Stufe, indem die Tools aus Stufe 1a um die Best Practices erweitert werden. Der erste Punkt ist die Validierung der Eingabeparameter. Diese wird durch die Verwendung von Pydantic Modellen für die Parameter der Tools erreicht. Pydantic bietet eine automatische Validierung der Parameter basierend auf den angegebenen Typen und zusätzlichen Informationen wie Wertebereichen oder regulären Ausdrücken. Bei ungültigen Parametern wird automatisch eine Fehlermeldung generiert, welche dann an das LLM zurückgegeben wird. Die Rückgabe von Fehlermeldungen wird durch ein *try/except* Block in der Logik der Tools implementiert. In diesem Block wird die eigentliche Logik des Tools ausgeführt und bei einem Fehler wird die Fehlermeldung abgefangen und als Antwort zurückgegeben. Dies erlaubt eine granulare Fehlerbehandlung, da verschiedene Fehlerarten unterschiedlich behandelt werden können. Eine einfache Fehlermeldung wird auch ohne diesen Block zurückgegeben. Die letzten beiden Punkte der Best Practices, die Bereinigung der Toolantwort und die Implementierung von Rate-Limiting, werden durch die Verwendung von Dekoratoren implementiert, welche zusätzlich zur Tooldefinition hinzugefügt werden können. Die Rate-Limiting Annotation gibt an, dass die Anzahl der Aufrufe eines Tools pro Zeiteinheit begrenzt werden soll. In diesem Fall wird eine Standardbegrenzung von 3 Aufrufen pro Minute für jedes Tool festgelegt. Diese Werte können über Parameter des Dekorators für jedes Tool individuell angepasst werden. Sollte das Limit überstiegen werden, wird die Ausführung des Tools um die Zeit des festgelegten Zeitraums verzögert, um die Anzahl der Aufrufe zu begrenzen. Der Dekorator für die Bereinigung der Antwort führt mehrere Schritte durch, um die Antwort an das Modell zu bereinigen. Zunächst werden alle Kennwörter, Secrets oder andere sensible Informationen aus der Antwort entfernt, indem sie mit einem Platzhalter ersetzt werden. Die Ersetzung erfolgt dabei über reguläre Ausdrücke, um verschiedene Formate von sensiblen Informationen zu erkennen. Dieser reguläre Ausdruck wurde mithilfe von KI generiert und dann manuell angepasst, um die Erkennung von sensiblen Informationen zu verbessern. Der nächste Schritt maskiert Anführungszeichen in der Antwort, um Probleme bei der Verarbeitung der Antwort durch das LLM zu vermeiden. Der letzte Schritt entfernt Pfadangeben die auf eine voherige Dateihierarchie verweisen, um Pfadtraversal zu verhindern. Die Liste der durchzuführenden Schritte für eine effektive Bereinigung ist natürlich nicht universell und hängt von den jeweiligen Tools, Apis und Anwendungsfällen ab. Da die Dekoratoren direkt für alle Tools verwendet werden können und keine spezifische Logik in der Tooldefinition benötigen, wurde die benötigte Zeit von 4 Stunden für die Implementierung auf alle Tools verteilt. Zusätzlich kommt noch ein geringer Aufwand für die Fehlerbehandlung hinzu. Das ergibt eine Umsetzungszeit von 1,25 Stunden pro API. Die

Arbeitsspeicherauslastung hat sich durch die Implementierung nur minimal geändert, was aber auch auf Ungenauigkeiten bei der Messung zurückzuführen sein könnte. Die Tests ergaben eine Erfolgsquote von 10 von 16 (62,5%) bei der GitHub API, 8 von 12 (66,67%) bei der Discord API, 2 von 8 (25%) bei der EVE Online API und 16 von 20 (80%) bei der Invoice Manager API. Bei der GitHub API konnte im ersten Szenario das Mistral Modell keine korrekte Antwort liefern. Es führte keine Toolaufrufe durch und antwortete mit einer falschen Anzahl an Issues. Im zweiten Szenario war es dem GPT Modell nicht möglich ein Issue zu erstellen, da Probleme mit dem Labelformat gab. Das dritte Szenario konnte in allen Tests gelöst werden. Beim vierten Szenario gab das Mistral Modell wieder an nicht die notwenigen Tools zu kennen. Bei der Discord API konnte das Mistral Modell im ersten und zweiten Szenario nicht das richtige Ergebnis liefern. Einmal da es keine Toolaufrufe durchführte und einmal aufgrund von Fehlern bei den Toolaufrufen. Alle anderen Tests konnten gelöst werden. Das GPT Modell führte beim ersten Szenario der EVE Online API zwar alle notwendigen Tools aus, konnte aber nicht die korrekte Anzahl an Schiffen ermitteln. Beim zweiten Szenario konnte das GPT Modell korrekt darauf hinweisen, das es kein Schiff mit dem Namen 'Star destroyer' gibt. Das Mistral Modell führte bei keinem der Szeanrien Tools aus. Bei den Invoice Manager Tests konnte im ersten Szenario das Mistral Modell beim ersten Versuch die richtige Antwort trotz richtiger Toolaufrufe nicht liefern. Ansosnten funktionierten die Tests in diesem Szenario. Alle Tests des zweiten Szenarios konnten gelöst werden. Beim dritten Szenario führte das Mistral Modell keine Toolaufrufe durch, und lieferte eine falsche Antwort. Das GPT Modell erfolgreich lösen. Beim vierten Szenario konnten beide Modell wieder die korrekten Antworten liefern. Fehler traten im fünften Szenario nur beim ersten Versuch des GPT Modells auf, da es das PDF für eine falsche Rechnung erstellte.

Die zweite Optimierungsstufe besteht aus der Verbesserung der Beschreibungen der Tools und deren Parameter. Dies soll dem LLM helfen, die Tools besser zu verstehen und sie korrekt zu verwenden. Bisher wurden für die beschreibungen der Tools und Parameter die Informationen aus der OpenAPI Beschreibung übernommen, sofern diese vorhanden waren, was bei der Discord API nicht der Fall war. Bei der GitHub API wurden die Beschreibungen hauptsächlich um zusätzliche Inforamtionen gekürzt, die für das LLM nicht relevant sind, wie die Angabe von verschiedenen möglichen Mediatypes. Bei der Discord API wurde jedes Tool und jeder Parameter mit einer Beschreibung versehen, um die fehlende Dokumentation der API zu kompensieren. Bei den anderen APIs wurden keine Änderungen an den Beschreibungen vorgenommen, da diese bereits ausreichend Informationen enthielten. Für die Änderungen an den Beschreibungen wurden pro API ca. 30 Minuten aufgewendet. Auch in dieser Stufe gab es keine nennenswerten Änderungen bei der Arbeitsspeicherauslastung. Da bei der EVE und der Invoice Manager API keine Änderungen an den Beschreibungen vorgenommen wurden, wurden die Tests dieser APIs nicht erneut durchgeführt. Bei der GitHub API verringerte sich die Erfolgsquote auf 8 von 16 (50%), da das Mistral Modell keines der vier Szenarien lösen konnte. Im ersten Szeanrio führte es keine Toolaufrufe durch und bei den anderen drei Szenarien gab es Fehler bei den Toolaufrufen. Die Erfolgsquote bei der Discord API blieb mit 8 von 12 (66,67%) gleich. Es traten die selben Fehler wie in der vorherigen Stufe auf.

Stufe 3 soll eine Verbesserung der Tests erreichen, indem die Antworten der Tools als strukturierte Daten zurückgegeben werden, anstatt als unstrukturierter Text. Dafür werden den Toolfunktionen Type Hints hinzugefügt, aus denen das SDK die OutputSchemas für die Tools ableitet, welches die Struktur der Antwort für das LLM definiert. Dies kann dem LLM helfen, die Daten die ein Tool bietet noch vor Nutzung des Tools zu verstehen, um die richtigen Tools auszuwählen und die Antwort der Tools besser zu nutzen. Wie bei den Fehlerantworten wird jetzt auch hier die Antwort in ein *CallToolResult* Objekt aus dem MCP SDK verpackt. Die APIs lifern Pydantic Modelle, Klassen die *BaseModel* erweitern, zurück. Diese müssen mit der Funktion *model_dump()* in ein Dictionary umgewandelt werden, damit sie als strukturierte Daten zurückgegeben werden können. Aufgrund des verpackens der Antwort in ein *CallToolResult*

muss der Type Hint der Toolfunktion entsprechend angegeben werden, damit die Validierung der Antwort durch das SDK korrekt funktioniert. Daher werden die Type Hints wie folgt angegeben: *-> Annotated[CallToolResult, <Antwort Objekt>]*. Da die Antworten direkt den APIs entstammen, werden die Type Hints direkt aus den Rückgabetypen der Funktionen der generierten Module übernommen. Die Umsetzung dieser Stufe dauerte 4,25 Stunden für die GitHub API, eine Studne für die Discord API, 1,75 Stunden für die EVE Online API und 2 Stunden für die Invoice Manager API. Die Umsetzung der GitHub API dauerte etwas länger, da hier zuerst die Fehler mit der Validierung der Antwortschemas auftauchten. Der Arbeitsspeicherverbrauch stieg bei dieser Stufe minimal an, auf 313,9 MB für die GitHub API, 144,1 MB für die Discord API, 95,2 MB für die EVE Online API und 55,1 MB für die Invoice Manager API. Bei den ersten Versuchen mit dieser Stufe fiel auf, dass alle Tests aufgrund fehlender Daten vom Modell abgebrochen wurden. Bei der Untersuchung des Problems stellte sich heraus, dass das für die Testanwendung verwendete, Spring AI keine strukturierten Daten unterstützt. Die Modelle erhielten also nur den leeren unstrukturierten Text. Um dieses Problem zu lösen, wurde eine angepasste Version des *spring-ai-mcp*[72] Paketes, welches in der version 1.1.0 verwendet wird, erstellt. In dieser angepassten Version wurde in den Klassen *SyncMcpToolCallback* und *AsyncMcpToolCallback* die Rückgabe der *call* Methode von unstrukturierten *content()* auf das strukturierte *structuredContent()* geändert, um die Daten an das Sprachmodell weiterzugeben. Es ergab sich eine Erfolgsquote von 7 von 16 (43,75%) bei der GitHub API und 6 von 12 (50%) bei der Discord API, 3 von 8 (37,5%) bei der EVE Online API und 15 von 20 (75%) bei der Invoice Manager API. Bei der GitHub API konnte das Mistral Modell in beiden Versuchen kein Ergebnis liefern, da es keine Toolaufrufe durchführte. Im zweiten Szenario stieß das GPT Modell auf Probleme bei der Erstellung eines Issues. Auch das Mistral Modell hatte Probleme bei der Erstellung eines Issues. Es gab Probleme bei der Validdierung an. Im dritten Szenario führte das Mistral Modell im ersten Test keine Toolaufrufe durch und im vierten Szenario gab es Fehler bei den Toolaufrufen bei beiden Versuchen. Bei den Discord Tests meldete das Mistral Modell im ersten Szenario, dass es nicht die notwendigen Fähigkeiten für die Aufgabe hat. Das GPT Modell konnte hier die richtige Anzahl an Nachrichten ermitteln. Im zweiten Szenario hatte das GPT Modell Probleme bei der Validierung der Antwort der API und konnte daher keinen Thread erstellen. Das Mistral Modell brach die Tests mit einer Serverfehlermeldung des Modells ab. Das dritte Szenario konnte von beiden Modellen gelöst werden. Bei der EVE Online API konnte im ersten Szenario die Anzahl an Schiffen nicht korrekt vom GPT Modell ermittelt werden. Es wurden zwar die richtigen Tools ausfgerufen, die Antwort stimmte aber nicht mit der erwarteten Anzahl überein. Das Mistral Modell meldete wieder das Fehlen der notwendigen Fähigkeiten für die Aufgabe. Das zweite Szenario konnte nur im ersten Versuch des Mistral Modells nicht gelöst werden, da es wieder das Fehlen der notwendigen Fähigkeiten meldete. Alle anderen Tests konnten gelöst werden. Die ersten zwei Szenarien der Invoice Manager API konnten erfolgreich gelöst werden. Beim dritten Szenario führte das Mistral Modell keine Toolaufrufe durch und lieferte eine falsche Antwort, während das GPT Modell die Aufgabe erfolgreich lösen konnte. Beim vierten Szenario konnten beide Modell erfolgreich eine neue Rechnung erstellen. Beim fünften Szenario versuchte das GPT Modell im ersten Versuch das PDF für eine falsche Rechnung zu erstellen. Im zweiten Versuch konnte es die richtige Rechnung auswählen und das PDF erstellen. Das Mistral Modell merkte wieder das Fehlen der notwendigen Fähigkeiten an und konnte daher keinen Test lösen.

Stufe 4a soll die Möglichkeit bieten, die Antworten der Tools zu filtern, um den Context für das LLM zu reduzieren. Die Optimierungsidee dieser Stufe stammt vom MCP Generator Stainless [52]. Dieser hat einigen Tools ein zusätzlichen Parameter hinzugefügt, welcher die Möglichkeit bietet, die Antwort der Tools mit einem JQ Filter zu filtern, um den Context für das LLM zu reduzieren. JQ ist ein leichtgewichtiges und flexibles Kommandozeilenwerkzeug zur Verarbeitung von JSON-Daten [63]. Es ermöglicht die Auswahl, Filterung und Umformung von JSON-Daten auf einfache Weise. Durch die Verwendung eines JQ Filter können die Antworten

der Tools so gefiltert werden, dass nur die relevanten Informationen zurückgegeben werden, um den Context für das LLM zu reduzieren und die Effizienz der Nutzung der Tools zu verbessern. Die Implementierung dieser Stufe besteht darin, den Tools einen zusätzlichen Parameter namens 'jq_filter' hinzuzufügen, welcher die Möglichkeit bietet, einen JQ Filter als String zu übergeben. Um den Effekt des Parameters genauer zu testen, wird Teil a der vierten Stufe zunächst ohne die Definition von OutputSchemas und strukturierten Daten implementiert, um zu sehen, wie effektiv der Filter allein bei der Reduzierung des Contexts ist. Dafür setzt diese Implementierung zunächst auf der Stufe 2 auf. Um den Filter auf das API Ergebnis anwenden zu können, wird die Python Bibliothek *jq.py* [73] verwendet, welche eine Schnittstelle zur JQ Funktionalität bietet. In der Implementierung wird geprüft, ob der 'jq_filter' Parameter übergeben wurde. Wenn dies der Fall ist, wird der Filter auf die Antwort der API angewendet, ansonsten wird die Antwort unverändert zurückgegeben. Da JQ für die Anwendung auf JSON-Daten ausgelegt ist, werden die API Daten zunächst wieder mit *model_dump()* in ein Dictionary umgewandelt, bevor der Filter angewendet wird. Mit dieser Stufe sank der Arbeitsspeicherbedarf wieder auf 312,7, 142,7, 95,2 und 54,4 MB für die GitHub, Discord, EVE Online und Invoice Manager API. Die Umsetzung dieser Stufe beläuft sich auf durchschnittlich 30 Minuten pro API, da hier nur der zusätzliche Parameter und die Logik zur Anwendung des JQ Filters hinzugefügt werden muss. Bei den Tests belief sich die Erfolgsquote auf 13 von 16 (81,25%) bei der GitHub API, 4 von 12 (33,34%) bei der Discord API, 3 von 8 (37,5%) bei der EVE Online API und 15 von 20 (75%) bei der Invoice Manager API. Bei der GitHub API konnte das Mistral Modell im ersten Szenario die Anzahl der Issues nicht korrekt ermitteln, da es keine Toolaufrufe durchführte. Im dritten Szenario hatte es Probleme mit der Erstellung eines Issues, mit der Meldung von mangelnden Berechtigungen, obwohl die Authentifizierung korrekt eingerichtet war. Alle anderen Tests konnten erfolgreich gelöst werden. Bei der Discord API onnte im ersten Szenario das GPT Modell durch den falschen Einsatz des Limit Parameters nicht die richtige Anzahl an Nachrichten ermittelt werden. Das Mistrall meldete im ersten Versuch das fehlen der Fähigkeiten für diese Aufgabe, konnte aber im zweiten Versuch die Aufgabe erfolgreich lösen. Im zweiten Szenario konnte das GPT Modell den Thread nicht erstellen, da es Probleme bei der Ermittlung der Id des Kanals gab. Bei diesem Versuch, versuchte das Modell mit dem 'jq_filter' Parameter eine Liste von Kanal Ids und Kanalnamen zu erstellen, was aufgrund falscher Namen im Filterausdruck nicht gelang. Das Mistral Modell hatte Fehler bei der Toolausführung und konnte daher das Szenario nicht lösen. Beim dritten Szenario stieß das Mistral Modell wieder auf Fehler bei der Toolausführung, während das GPT Modell die Aufgabe erfolgreich lösen konnte. Das GPT Modell nutzte in diesem Szeanrio den 'jq_filter' Parameter beim ersten mal um direkt den richtigen Discord Server anhand des Namen zu ermitteln und dessen Id zu extrahieren. Beim zweiten Versuch wurde der Parameter genutzt um eine Liste von Server Ids und Servernamen zu erstellen, um so die Id des Servers zu ermitteln. Das erste Szenario der EVE Online API konnte beide Modelle nicht lösen, da sie nicht die korrekte Anzahl an Schiffen als Antwort gaben, obwoahl sie die richtigen Tools ausführten. Beim zweiten Versuch merkte das Mistral Modell wieder den Mangen an Fähigkeiten an. Das zweite Szenario konnte nur beim ersten Versuch des Mistral Modells aufgrund der meldung mangelnder Fähigkeiten nicht gelöst werden. Bei den Tests für die EVE Online API wurde der 'jq_filter' Parameter nicht genutzt. Das erste Szenario der Invoice Manager API konnte nur beim zweiten Versuch des Mistral Modells nicht gelöst werden, es führte zwar die richtigen Tools aus, aber lieferte eine falsche Antwort. Das zweite Szenario konnte von beiden Modellen gelöst werden. Beim dritten Szeanrio antwortete das Mistral Modell beime rsten Versuch mit einer falschen Anzahl an Rechnungen und führte keine Toolaufrufe durch. Beim zweiten Versuch kam wieder die Meldung über fehlende Fähigkeiten. Das vierte Szenario konnte von beiden Modellen gelöst werden. Im ersten Versuch nutzte das GPT Modell den 'jq_filter' Parameter, um direkt die Id der erstellten Rechnung zu ermitteln und eine Liste von Kundennamen und Ids zu erstellen. Das Mistral Modell machte bei diesem Szenario keinen Gebrauch von dem 'jq_filter' Parameter.

Beim letzten Szeanrio meldete das Mistral Modell wieder einen Mangel an Fähigkeiten, während das GPT Modell die Aufgabe erfolgreich lösen konnte.

Als Erweiterung der Stufe 4a soll in Stufe 4b die Möglichkeit geschaffen werden, die Nutzung des 'jq_filter' Parameters durch die Definition von OutputSchemas zu unterstützen. Dadurch, dass dem LLM die Struktur der Antowrt durch das OutputSchema bekannt ist, kann es den 'jq_filter' Parameter effektiver nutzen, um die relevanten Informationen aus der Antwort zu extrahieren und zurückzugeben, was die Effizienz der Nutzung der Tools weiter verbessern kann. Für die Umsetzung wird zunächst das Ergebnis der Tools direkt in ein *CallToolResult* Objekt verpackt, wie es bereits in Stufe 3, aber nicht Stufe 4a, der Fall war. Anschließend werden die Type Hints für die Tools aus der Stufe 3 übernommen, um die OutputSchemas zu definieren. Da durch das Hinzufügen der Type Hints die Validierung der Toolantwort aktiviert wird, diese aber aufgrund des 'jq_filter's nicht immer bekannt ist, muss die Validierung der Antwort für den Fall, dass der 'jq_filter' Parameter genutzt wird, deaktiviert werden. Um dies zu erreichen, wird der Type Hint angepasst. Anstatt der in Stufe 3 verwendeten Angabe von *-> Annotated[CallToolResult, <Antwort Objekt>]* wird jetzt die Angabe *-> Union[<Antwort Objekt>, Any]* verwendet, um einerseits die Struktur der Antwort zu definieren, aber andererseits die Validierung zu deaktivieren, indem die Möglichkeit einer beliebigen Antwort gegeben wird. Die Umsetzung dauerte pro API ca. 15 Minuten, da hier nur die Anpassung der Type Hints und die Deaktivierung der Validierung hinzugefügt werden muss. Der Arbeitsspeicherverbrauch stieg minimal an, auf 313,7 MB für die GitHub API, 144,8 MB für die Discord API, 95,0 MB für die EVE Online API und 55,2 MB für die Invoice Manager API. Die Erfolgsquote ergab 13 von 16 (81,25%) bei der GitHub API, 6 von 12 (50%) bei der Discord API, 2 von 8 (25%) bei der EVE Online API und 14 von 20 (70%) bei der Invoice Manager API. Bei den Tests zur GitHub API gab es nur drei fehlgeschlagene Tests, bei denen das Mistral Modell keine Toolaufrufe durchführte, einmal die zwei Versuche des ersten Szenarios und einmal der erste Versuch des dritten Szenarios. Szenario eins der Discord API konnte vom GPT Modell gelöst werden, jedoch nicht vom Mistral Modell, welches wieder die fehlenden Fähigkeiten meldete. Beim zweiten Szenario gab es Fehler bei Verarbeitung der Tools durch das Mistral Modell. Das GPT Modell konnte auch nach mehreren Versuchen einen Thread zu erstellen das Szeanrio nicht abschließen. Beim dritten Szenario konnte nur das GPT Modell im ersten Versuch das Szeanrio nicht lösen, da es hier bei der Verarbeitung der Einladungen einen falschen 'jq_filter' Parameter nutzte, um die Einladungen umzuformen. Bei der EVE Online API führte das GPT Modell zwar die richtigen Tools aus, konnte aber nicht die korrekte Anzahl an Schiffen ermitteln. Das Mistral Modell meldete wieder das Fehlen der notwendigen Fähigkeiten für die Aufgabe. Beim zweiten Szenario meldete das GPT Modell richtig, dass es kein Schiff mit dem Namen 'Star destroyer' gibt, während das Mistral Modell wieder die fehlenden Fähigkeiten meldete. Die ersten beiden Szenarien der Invoice Manager API konnten erfolgreich gelöst werden. Beim dritten Szenario konnte das Mistral Modell wieder keine Tools finden. Das vierte Szeanrio konnte wieder von beiden Modellen gelöst werden, wohingegen das fünfte Szenario von keinem Modell richtig gelöst werden konnte, da entweder nicht die aktuellste Rechnung ausgewählt wurde oder im zweiten Versuch das Mistral Modell wieder die fehlenden Fähigkeiten meldete. Während in Stufe 4a der 'jq'-Filter bei 34, ohne Fehler, durchgeführten Tests nur 7 mal verwendet wurde, stieg die Nutzung des 'jq'-Filters in Stufe 4b bei ebenfalls 34 Tests auf 11 Nutzungen an. Dies ist eine Steigerung von 20,58% auf 32,35%, was darauf hindeutet, dass die Unterstützung des 'jq_filter' Parameters durch die Definition von OutputSchemas die Nutzung dieses Parameters durch die Modelle erhöht hat.

Die letzte Stufe der Optimierungen, die auf den reinen API Endpunkten basieren, ist die Umformung der Toolantworten in das 'TOON'-Format [70]. Das Token-Oriented Object Notation Format wurde 2025 von Johann Schopplich erfunden und soll Daten, welche häufig als JSON vorliegen, in einem Format darstellen, welches weniger Token verbraucht, um einerseits den

Context für das LLM zu reduzieren und damit auch Kosten für die verwendeten Token zu reduzieren, aber andererseits auch die Struktur der Daten beizubehalten, um die Nutzung der Daten durch das LLM zu erleichtern. Das Format ist angelehnt an das YAML und CSV Format.

Listing 4.5: Beispiel des JSON-Formats

```
{
  "context": {
    "task": "Our favorite hikes together",
    "location": "Boulder",
    "season": "spring_2025"
  },
  "friends": ["ana", "luis", "sam"],
  "hikes": [
    {
      "id": 1,
      "name": "Blue Lake Trail",
      "distanceKm": 7.5,
      "elevationGain": 320,
      "companion": "ana",
      "wasSunny": true
    },
    {
      "id": 2,
      "name": "Ridge Overlook",
      "distanceKm": 9.2,
      "elevationGain": 540,
      "companion": "luis",
      "wasSunny": false
    }
  ]
}
```

Listing 4.6: Beispiel des YAML-Formats

```
---
context:
  task: Our favorite hikes together
  location: Boulder
  season: spring_2025
friends:
- ana
- luis
- sam
hikes:
- id: 1
  name: Blue Lake Trail
  distanceKm: 7.5
  elevationGain: 320
  companion: ana
  wasSunny: true
- id: 2
  name: Ridge Overlook
  distanceKm: 9.2
  elevationGain: 540
  companion: luis
  wasSunny: false
```

Listing 4.7: Beispiel des TOON-Formats

```
context:
  task: Our favorite hikes together
  location: Boulder
  season: spring_2025
friends[3]: ana,luis,sam
hikes[3]{id,name,distanceKm,elevationGain,companion,wasSunny}:
  1,Blue Lake Trail,7.5,320,ana,true
  2,Ridge Overlook,9.2,540,luis,false
```

Die Beispiele 4.5, 4.6 und 4.7 zeigen die Darstellung der selben Daten in den Formaten JSON, YAML und TOON. Das Beispiel stammt von der offiziellen TOON Webseite [70]. OpenAI bietet einen Online Tokenizer [74] an, mit dem die Anzahl der Token für Texte bestimmt werden kann. Verwendet man hier den Tokenizer von GPT-5 bekommt man für die Beispiele folgende Anzahl an Token: 169 Token bei 501 Zeichen für das JSON-Format, 120 Token bei 337 Zeichen für das YAML-Format und 87 Token bei 249 Zeichen für das TOON-Format. Das bedeutet, dass die Darstellung der Daten im TOON-Format im Vergleich zum JSON-Format eine Reduzierung von 51,48% der Token. Für die Umsetzung dieser Stufe werden die Antworten der Tools nun von JSON in das TOON-Format gewandelt. Dafür wird die offizielle Python Implementierung des TOON-Formats [75] verwendet, welche eine Funktion namens *encode()* bereitstellt, um JSON Daten in das TOON-Format umzuwandeln. Diese kann direkt an die Antwort der Tools angewendet werden, bevor diese an das LLM zurückgegeben wird. Die Umsetzung dieser Stufe dauerte ca. 15 Minuten pro API. Die Arbeitsspeicherauslastung änderte sich durch die Umformung der Antworten in das TOON-Format nicht nennenswert, da die Umformung direkt auf die Antwort angewendet wird, bevor sie an das LLM zurückgegeben wird. Die Erfolgsquote ergab 9 von 16 (56,25%) bei der GitHub API, 6 von 12 (50%) bei der Discord API, 4 von 8 (50%) bei der EVE Online API und 14 von 20 (70%) bei der Invoice Manager API. Bei den Tests zur GitHub API konnte das GPT Modell beim ersten Versuch des ersten Szenarios die Anzahl der Issues nicht korrekt ermitteln, da hier der 'per_page' Parameter falsch genutzt wurde. Im zweiten Versuch konnte es die Aufgabe erfolgreich lösen. Das Mistral Modell führte bei diesem Szenario keine Toolaufrufe durch und Antwortete mit einer falschen Anzahl an Issues. Im zweiten Szenario gelang es beiden Modellen ein Issue zu erstellen. Beim dritten Szenario führte nur das Mitral Modell beim ersten Versuch keine Toolaufrufe durch, während es im zweiten Versuch die Aufgabe erfolgreich lösen konnte. Das GPT Modell konnte beide Versuche erfolgreich lösen. Im vierten Szenario konnte nur das GPT Modell im ersten Versuch die Aufgabe korrekt lösen. Bei den anderen drei Versuchen würde nur das erste der fünf Repositories zurückgegeben. Bei den Discord Tests konnte das GPT Modell im ersten Szeanrio die richtige Anzahl ermitteln, das Mistral Modell meldete wieder die fehlenden Fähigkeiten für diese Aufgabe. Beim zweiten Szenario konnte das GPT Modell im ersten Versuch keinen neuen Thread anlegen und im zweiten Versuc benötigte es mehrere Toolaufrufe bis es die Aufgabe lösen konnte. Das Mistral Modell stieß wieder auf Fehler bei der Ausführung. Im dritten Szenario konnte das GPT Modell in beiden Fällen und das Mistral Modell im ersten Versuch die Aufgabe lösen, während es im zweiten Versuch wieder den Mangel an Fähigkeiten meldete. Bei der EVE Online API konnte das GPT Modell im ersten Szenario die Anzahl an Schiffen nicht korrekt ermitteln. Das Mistral Modell meldete wieder das Fehlen der notwendigen Fähigkeiten für die Aufgabe. Beim zweiten Szenario konnten beide Modelle korrekt darauf hinweisen, dass es kein Schiff mit dem Namen 'Star destroyer' gibt. Bei der Invoice Manager API konnten die ersten beiden Szenarien von beiden Modellen gelöst werden. Beim dritten Szenario konnte das Mistral Modell wieder keine Tools finden, während das GPT Modell die Aufgabe erfolgreich lösen konnte. Beim vierten Szenario konnten beide Modelle erfolgreich eine neue Rechnung erstellen. Beim fünften Szenario meldete das Mistral Modell wieder den Mangel an Fähigkeiten, während das GPT Modell in beiden Fällen eine falsche Rechnung auswählte und das PDF dafür erstellte.

Die sechste und letzte Stufe ist die Optimierung der zur Verfügung stehenden Tools selbst. In den vorherigen Stufen wurden die Tools direkt aus den API Endpunkten erstellt, ohne Änderungen an der Logik oder der Funktionalität der Tools vorzunehmen. Jeremiah Lowin, der Entwickler von FastMCP, schreibt in einem Blogpost [76], dass die direkte Umwandlung von REST API Endpunkten zu MCP Tools schlecht für die Nutzung durch LLMs ist, da der meist atomare Aufbau von REST API für LLMs nicht ideal ist. Zum einen liegt das, laut Lowin, daran, dass das Aufrufen vieler Tools eine teure Operation ist, die zeit kostet und den Context des Modells belastet, da jedes Mal die Informationen der Tools verarbeitet werden müssen. Zum anderen muss das Modell bei der Auswahl der Tools die Logik der API verstehen, um die richtigen Tools auszuwählen und sie in der richtigen Reihenfolge aufzurufen, was bei komplexeren APIs schwierig sein kann. Daher werden in dieser Stufe die Tools so optimiert, dass sie besser für die Nutzung durch LLMs geeignet sind. Das bedeutet, dass die Tools so gestaltet werden, dass sie mehr Funktionalität bieten und weniger Aufrufe erfordern, um eine Aufgabe zu lösen. Die Optimierung der Tools kann auf verschiedene Weisen erfolgen, wie zum Beispiel durch die Kombination mehrerer API Endpunkte zu einem einzigen Tool, die Implementierung von zusätzlicher Logik in den Tools, um die Nutzung der Tools zu vereinfachen, oder auch durch die Bereitstellung von vorverarbeiteten Daten, um die Notwendigkeit von mehreren Toolaufrufen zu reduzieren. Bei der GitHub API konnte die Anzahl der Tools von 6 auf 4 reduziert werden. Dafür wurden einmal die Tools für die Abfrage der Labels und die zur Erstellung eines Issues zu einem einzigen Tool kombiniert und die Tools zur Abfrage der Branches eines Repositories und zur Abfrage von Details eines Branches zu einem einzigen Tool kombiniert. Listing 4.8 zeigt die Logik des kombinierten Tools zur Abfrage der Labels und zur Erstellung eines Issues dargestellt.

Listing 4.8: Beispiel für die Kombination der Tools zur Abfrage der Labels und zur Erstellung eines Issues.

```
1  @mcp.tool()
2  @rate_limit()
3  @sanitize_output()
4  async def create_issue(
5          owner: Annotated[str, Field(description="The␣owner␣of␣the␣
       repository.")],
6          repo: Annotated[str, Field(description="The␣name␣of␣the␣
       repository.")],
7          title: Annotated[str, Field(description="The␣title␣of␣the␣issue.
       ")],
8          body: Annotated[Optional[str], Field(description="The␣contents␣
       of␣the␣issue.")] = "",
9          labels: Annotated[Optional[List[str]], Field(description="The␣
       labels␣to␣add␣to␣the␣issue.")] = None,
10         jq_filter: Annotated[Optional[str], Field(
11             description="An␣optional␣jq␣filter␣to␣apply␣to␣the␣result,␣
       to␣customize␣the␣result␣format.")] = None
12 ):
13     """Create a new issue in a repository."""
14     try:
15         if labels is None:
16             labels = []
17         else:
18             label_check = LabelCheck(labels, owner, repo)
19             await label_check.check_labels_exist()
20             if len(label_check.nonExistingLabels) > 0:
21                 return CallToolResult(content=[TextContent(type="text",
```

```
22                                                                  text=f"The␣
    following␣labels␣do␣not␣exist:␣{',␣'.join(label_check.
    nonExistingLabels)}")],
23                                      isError=True)
24
25         issue_create_request = {
26             "title": {"actual_instance": title},
27             "body": body,
28             "labels": [{"actual_instance": label for label in labels}]
29         }
30         created_issue = await issuesApi.issues_create(owner=owner, repo=
    repo, issues_create_request=issue_create_request)
31
32         if jq_filter is not None and jq_filter != "":
33             return encode(jq.compile(jq_filter).input_value(
    created_issue.model_dump(mode="json")).all())
34         else:
35             return encode(created_issue.model_dump(mode="json"))
36     except Exception as e:
37         return CallToolResult(content=[TextContent(type="text", text=str
    (e))], isError=True)
38
39
40 class LabelCheck():
41     def __init__(self, labels: List[str], repo_owner: str, repo_name:
    str):
42         self.labels = labels
43         self.repoOwner = repo_owner
44         self.repoName = repo_name
45         self.existingLabels = []
46         self.nonExistingLabels = []
47
48     async def check_labels_exist(self):
49         labels_to_check = self.labels
50
51         page = 1
52         labels_from_repo: List[Label] = await issuesApi.
    issues_list_labels_for_repo(owner=self.repoOwner,
53
                 repo=self.repoName, per_page=100,
54
                 page=page)
55         self.existingLabels.extend([label.name for label in
    labels_from_repo if label.name in labels_to_check])
56         labels_to_check = set(labels_to_check) - set(self.existingLabels
    )
57
58         while len(labels_from_repo) == 100:
59             page += 1
60             labels_from_repo = await issuesApi.
    issues_list_labels_for_repo(owner=self.repoOwner, repo=self.repoName,
61
        per_page=100, page=page)
62             self.existingLabels.extend([label.name for label in
    labels_from_repo if label.name in labels_to_check])
```

```
63            labels_to_check = set(labels_to_check) - set(self.
    existingLabels)
64
65        self.nonExistingLabels = list(labels_to_check)
```

Das Tool ist zunächst ähnlich dem voherigen Tools, außer dass es einen zusätzlichen Parameter namens 'labels' hat, welcher eine Liste von Strings erwartet. In den Zeilen 18 bis 22 sieht man die Logik in der geprüft wird, ob die gewünschten Labels existieren. Dafür wird in der Klasse *LabelCheck* der API Endpunkt zur Abfrage der Labels eines Repositories aufgerufen, um die verfügbaren Labels zu erhalten. Anschließend wird geprüft, ob die gewünschten Labels in der Liste der verfügbaren Labels enthalten sind. Sollte eines der Labels nicht verfügbar sein, wird eine entsprechende Fehlermeldung an das Modell zurückgegeben, damit es die Möglichkeit hat, darauf zu reagieren, zum Beispiel indem es zum Beispiel die gewünschten Label anpasst. An dieser Stelle werden nur die Label zurückgegeben, die nicht existieren. Eine weitere Möglichkeit der Umsetzung an dieser Stelle wäre es, die fehlenden Label automatisch zu erstellen oder ähnliche Label zu den gewünschten Label vorzuschlagen. Sollte die Prüfung der Label erfolgreich sein, wird die Logik zur Erstellung eines Issues aufgerufen, um das Issue mit den gewünschten Label zu erstellen. Das Tool zur Abfrage der Branch Informationen wurde ähnlich umgesetzt. Es holt zuerst alle verfügbaren Branches des angegebenen Repositories und ruft dann für den gewünschten Branch die Details ab, um sie gesammelt an das Modell zurückzugeben. Dadurch kann zum Beispiel die Anzahl der Toolaufrufe bei drei Branches von vier Aufrufen auf einen Aufruf reduziert werden. Da die Ergebnisse der vorherigen Stufen, die strukturierte Daten verwenden, einen leichte Verschlechterung der Ergebnisse im Vergleich zu den vorherigen Stufen zeigten, werden die Daten in dieser Stufe wieder als unstrukturierter Text zurückgegeben. Die Annotationen für das Rate-Limiting und die Bereinigung der Antworten werden weiterhin verwendet und auch der 'jq_filter' Parameter bleibt erhalten. Die Umsetzung dieser Stufe für die GitHUb API dauerte 6,25 Stunden. Der Arbeitsspeicherverbrauch ist mit 313 MB ähnlich dem der vorherigen Stufe. Mit diesen Tools konnten 10 von 16 (62,5%) Tests der GitHub API gelöst werden. Das GPT Modell konnte nur im zweiten Versuch des ersten Szenarios die Anzahl der Issues korrekt ermitteln, da es im ersten Versuch den 'per_page' Parameter falsch nutzte. Das Mistral Modell führte bei diesem Szenario keine Toolaufrufe durch. Das zweite Szenario konnte von beiden Modellen gelöst werden. Beim dritten Szenario konnte nur das GPT Modell die Aufgabe lösen, das Mistral Modell meldete wieder das Fehlen der notwendigen Fähigkeiten für die Aufgabe. Beim vierten Szenario konnte wieder beide Modelle die Aufgabe lösen.

Der MCP-Server für die Discord API konnte von 5 auf 3 Tools reduziert werden. Dies war möglich, indem die Tools die zur Abfrage der Server- und Kanalinformationen nötig waren, direkt in die Tools zur Erledigung der Aufgaben integriert wurden, um so die Anzahl der benötigten Toolaufrufe zu reduzieren. Vorher war es notwendig erst die Serverinformationen abzufragen, um die Id des Servers zu erhalten, dann die Kanalinformationen abzufragen, um die Id des Kanals zu erhalten, um dann die eigentliche Aufgabe zu erledigen. Durch die Integration der Logik zur Abfrage der Server- und Kanalinformationen in die Tools zur Erledigung der Aufgaben, können die Aufgaben jetzt direkt mit den Namen des Servers und des Kanals erledigt werden, ohne dass vorher die Ids vom Modell abgefragt werden müssen. Diese Arbeit wurde nun in die einzelnen Tools verlagert. Dafür wurde den jeweiligen Tools ein oder zwei zusätzliche Parameter hinzugefügt, um die Namen der Server und Kanäle zu übergeben.

Listing 4.9: Auszug aus einem Tool des Discord MCP-Servers. Zu sehen ist die Abfrage der Server- und Kanalid

```
1 try:
2     guild = await find_server_by_name(server_name)
3     channel = await find_channel_by_name(guild, channel_name)
```

```
4  except ValueError as e:
5      return CallToolResult(content=[TextContent(type="text", text=str(e))
       ], isError=True)
6
7  async def find_server_by_name(server_name: str):
8      guilds = await api.list_my_guilds()
9      server_name_lower = server_name.lower()
10     for guild in guilds:
11         if guild.name.lower() == server_name_lower:
12             return guild
13     raise ValueError(f"Server␣with␣name␣'{server_name}'␣not␣found")
14
15 async def find_channel_by_name(guild: MyGuildResponse, channel_name: str
       ):
16     channels = await api.list_guild_channels(guild_id=guild.id)
17     channel_name_lower = channel_name.lower()
18     for channel in channels:
19         if channel.actual_instance.name.lower() == channel_name_lower:
20             return channel.actual_instance
21     raise ValueError(f"Channel␣with␣name␣'{channel_name}'␣not␣found␣in␣
       server␣'{guild.name}'")
```

Die Logik zur Abfrage der Server- und Kanalinformationen ist in den Zeilen 1 bis 5 zu sehen. Zunächst wird die Liste der verfügbaren Server abgefragt, um die Id des gewünschten Servers zu ermitteln. Anschließend wird die Liste der verfügbaren Kanäle des Servers abgefragt, um die Id des gewünschten Kanals zu ermitteln. Sollte einer der Namen nicht gefunden werden, wird eine entsprechende Fehlermeldung zurückgegeben, damit das Modell die Möglichkeit hat, darauf zu reagieren, zum Beispiel indem es die Namen anpasst. Mit diesem Mechanismus wurden die drei notwendigen Tools für die Aufgaben ausgestattet. Übrig bleibt ein Tool zur Abfrage von Nachrichten eines Kanals, eines zur Erstellung eines Threads und eines zur Abfrage der Einladungen zu einem Server. Das Tool zur Abfrage der Einladungen zu einem Server könnte noch weiter ausgebaut werden, indem man optionale Parameter für den Kanal- oder Threadnamen hinzufügt. Dadurch könnte es noch vielseitigeres werden. Bei dem Tool zur Erstellung eines Threads wurden hauptsächlich die Parameter angepasst. Zuvor wurden große und tief verschachtelte Objekte für die Erstellung des Threads erwartet. Jetzt wird nur noch der Name des Threads und eine initiale Nachricht erwartet. Dies schränkt zwar dessen Möglichkeiten ein, aber es vereinfacht die Nutzung für durch das LLM, da es nicht mehr die komplexe Struktur der vorherigen Parameter verstehen muss, um das Tool zu nutzen. Die Umsetzung dieser Stufe für die Discord API dauerte 1,75 Stunden. Der Arbeitsspeicherverbrauch ist mit 142,3 MB etwas geringer dem der vorherigen Stufe. In dieser Stufe konnte 10 von 12 (83,34%) Tests der Discord API gelöst werden. Nur im zweiten Szenario gab es Probleme mit dem GPT Modell. Es führte das Tool zwar richtig aus, tat dies aber mehrfach, wodurch mehrere Threads erstellt wurden.

Durch die Spezialiserung der Tools des EVE Online MCP-Servers konnte die Anzahl der Tools von 4 auf 2 reduziert werden. Für das erste Szeanrio, die Abfrage der Anzahl an Schiffen, wurden zuvor zwei verschiedene Tools und 49 Toolaufrufe benötigt. Durch die Erstellung eines Tools zur Abfrage aller Types (Schiffe) einer Kategorie konnte die Anzahl der erforderlichen Toolaufrufe auf einen Aufruf reduziert werden, da alle benötigten Informationen in der Antwort des Tools enthalten sind. Dafür werden im Tool selbst alle Gruppen einer Kategorie abgefragt und dann für jede Gruppe die Types abgefragt, um so eine Liste von allen Types einer Kategorie zu erstellen, welche dann an das LLM zurückgegeben wird. Das zweite Szenario, die Abfrage von Informationen zu einem bestimmten Schiff, benötigte zuvor zwei Tools und jeweils einen Toolaufruf, um die Id eines Schiffs zu ermitteln und dann die Details des Schiffs abzufragen. Durch die Erstellung eines Tools zur Abfrage von Informationen zu einem Schiff anhand des Namens, konnte die Anzahl

der Tools auf ein Tool reduziert werden, da die Logik zur Ermittlung der Id des Schiffs direkt integriert wurde. Dafür wird im Tool der Name des Schiffs in eine Id aufgelöst und diese dann weiter verwendet. Die Umsetzung dieser Stufe für die EVE Online API dauerte eine Stunde. Der Arbeitsspeicherverbrauch ist mit 94,7 MB wieder ähnlich dem der vorherigen Stufe. In dieser Stufe konnten 3 von 8 (37,5%) Tests der EVE Online API gelöst werden. Im ersten Szenario konnte das GPT Modell zwar das Tool aufrufen, aber es konnte nicht die korrekte Anzahl an Schiffen ermitteln. Das Mistral Modell meldete wieder das Fehlen der notwendigen Fähigkeiten für die Aufgabe. Beim zweiten Szenario konnte das GPT Modell korrekt darauf hinweisen, dass es kein Schiff mit dem Namen 'Star destroyer' gibt. Das Mistral Modell meldete im ersten Versuch das ein solches Schiff nicht bekannt sei, obwohl es keine Toolaufrufe durchführte, und im zweiten Versuch konnte die Aufgabe mit entsprechendem Hinweis des Modells abgeschlossen werden.

Der Invoice Manager MCP-Server konnte von 9 auf 3 reduziert werden. Dafür wurde wie beim Discord MCP-Server die Logik zur Abfrage von Informationen direkt in die Tools zur Erledigung der Aufgaben integriert, um so die Anzahl der benötigten Toolaufrufe zu reduzieren. Damit entfallen die Tools die vier Tools zur Abfrage der Kunden-, Ansprechparter-, Steuer- und Rechnugsvorlageninformationen, da diese Informationen jetzt direkt von den Tools die diese Inforasmtionen benötigen, ausgelöst werden. Zudem wurde das Tool zur Erstellung einer neuen Rechnugsposition direkt in das Tool zur Erstellung einer neuen Rechnung integriert. Das macht den MCP-Server in diesem Fall etwas unflexibler, da es jetzt nicht mehr möglich ist, eine neue Rechnungsposition zu erstellen, ohne eine neue Rechnung zu erstellen. Außerdem wurden die Tools zur Erstellung der PDF Datei einer Rechnung und dem Herunterladen einer Rechnung zu einem einzigen Tool kombiniert.

Listing 4.10: Tool zur Erstellung einer Rechnung mit Positionen

```
1  @mcp.tool()
2  @rate_limit()
3  @sanitize_output()
4  async def create_invoice(
5          invoice: Annotated[Invoice, Field(description="The␣invoice␣to␣
       create.")],
6          jq_filter: Annotated[Optional[str], Field(
7              description="An␣optional␣jq␣filter␣to␣apply␣to␣the␣result,␣
       to␣customize␣the␣result␣format.")] = None
8  ):
9      """Create a new invoice"""
10     try:
11         customer_id = await find_customer_id_by_name(invoice.
       customer_name)
12         receiver_id = None
13         sales_tax_id = await find_sales_tax_id_by_value(invoice.
       sales_tax)
14         invoice_template_id = await find_invoice_template_id_by_name(
       invoice.invoice_template)
15
16         if invoice.receiver_name is not None:
17             receiver_id = await find_receiver_id_by_name(invoice.
       receiver_name)
18
19         invoice_dto = {
20             "id": None,
21             "description": invoice.description,
22             "viaMail": invoice.via_mail,
23             "preText": invoice.pre_text,
```

```
24              "postText": invoice.post_text,
25              "serviceFrom": invoice.service_from,
26              "serviceTo": invoice.service_to,
27              "orderNumber": invoice.order_number,
28              "customerNumber": customer_id,
29              "paid": False,
30              "positions": [],
31              "receiver": receiver_id,
32              "salesTax": sales_tax_id,
33              "invoiceTemplate": invoice_template_id,
34              "file": None
35          }
36
37          created_invoice = await invoices_api.create_invoice(invoice_dto)
38
39          created_position_ids = []
40          for position in invoice.positions:
41              position_dto = {
42                  "description": position.description,
43                  "pricePerUnitInCents": position.price_per_unit_in_cents,
44                  "quantity": position.quantity,
45                  "unit": position.unit.value,
46                  "invoice": created_invoice.id
47              }
48              created_positions = await invoice_positions_api.
    create_position(invoice_position=position_dto)
49              created_position_ids.append(created_positions.id)
50
51          created_invoice.positions = created_position_ids
52          updated_invoice = await invoices_api.update_invoice_by_id(id=
    created_invoice.id, invoice=created_invoice)
53
54          if jq_filter is not None and jq_filter != "":
55              return encode(jq.compile(jq_filter).input_value(
    updated_invoice.model_dump(mode="json")).all())
56          else:
57              return encode(updated_invoice.model_dump(mode="json"))
58      except Exception as e:
59          return CallToolResult(content=[TextContent(type="text", text=str
    (e))], isError=True)
```

Das Tool zur Erstellung einer Rechnung mit Positionen ist in Listing 4.10 zu sehen. Es erwartet nun ein neues Rechnungsobjekt, welches anstatt von Ids für die Kunden, Ansprechpartner, Steuer- und Rechnungsvorlagen, die Namen dieser Objekte enthält. Außerdem enthält es eine Liste von Positionen, welche die Beschreibung, Menge und den Preis der Positionen enthält. Das Tool löst dann die Namen der Kunden, Ansprechpartner, Steuer- und Rechnungsvorlagen in Ids auf und erstellt die Rechnung mit den entsprechenden Positionen. Nach der Erstellung der Positionen wird die zuvor erstellte Rechnung aktualisiert, um die Positionen hinzuzufügen. Dieser Schritt ist in vorherigen Versuchen nicht immer durchgeführt worden. Sollte es bei einem der Schritte zu einem Fehler kommen, wird eine entsprechende Fehlermeldung zurückgegeben. Mit diesem MCP-Server konnte eine Erfolgsquote von 13 von 20 (65%), bei einem zeitaufwand von 2,25 Stunden, erreicht werden. Das erste Szeanrio konnte, bis auf den zweiten Versuch des Mistral Modells, von beiden Modellen gelöst werden. Das zweite Szenario konnte von beiden Modellen gelöst werden. Das dritte Szenario konnte nur vom GPT Modell gelöst werden. Das Mistral Modell antwortete zwar mit der richtigen Anzahl an Rechnungen, führte aber keine Toolaufrufe

durch. Beim vierten Szenario konnte das GPT Modell in beiden Fällen eine Rechnung erstellen, während das Mistral Modell im zweiten Versuch mit einem Fehler abbrach. Im letzten Szenario konnte das GPT Modell im ersten Versuch nicht die aktuellste Rechnung ermitteln und daher auch nicht erstellen. Im zweiten Versuch konnte es die Aufgabe dann erfolgreich lösen. Das Mistral Modell brach wieder mit einer Fehlermeldung ab.

Als nächstes folgt eine Analyse der Gründe für das fehlschlagen der Tests in den verschiedenen Stufen. In Stufe 1a trat achtmal der sogenannte 'Mistralfehler' auf, bei dem das Mistral Modell die Meldung zurückgab, dass es nicht die notwendigen Fähigkeiten für die Aufgabe hat. Es gab viermal eine falsche Antwort und keine Toolaufrufe und jeweils zweimal falsche Parameterwerte oder Fehler der GitHub Models API bei der Modell ausführung. Stufe 1b hatte acht Mistralfehler, viermal falsche Antwort und keine Toolaufrufe und zweimal falsche Parameterwerte und zwei Modellfehler. Stufe 2 hatte acht Modellfehler, zwei Mistralfehler und einen Fall von fehlenden Toolaufrufen. Stufe 3 hatte neun Mistralfehler, fünfmal keine Toolaufrufe, vier Modellfehler und zweimal eine falsche Antwort und falsche Parameterwerte. In Stufe 4a traten sechs Mistralfehler auf, sechsmal gab es eine falsche Antwort, vier Modellfehler, zwei Fälle von fehlenden Toolaufrufen, einen falschen Parameterwert und einmal meldete das Modell einen Authentifizierungsfehler. Mit Stufe 4b traten neun Mistralfehler auf, fünfmal gab es eine falsche Antwort, dreimal keine Toolaufrufe, zweimal Modellfehler und die Verwendung von falschen Parametern und einen falschen Parameterwert. Stufe 5 lieferte neun falsche Antworten, neun Mistralfehler, drei fehlende Toolaufrufe, zwei Modellfehler und eine falsche Parameternutzung. Die letzte Stufe, 6, hatte fünf falsche Antworten, fünf Mistralfehler, zwei fehlende Toolaufrufe, vier fehlende Toolaufrufe, drei Modellfehler und eine falsche Parameternutzung.

5 Vergleich und Fazit

In diesem Kapitel werden die Ergebnisse der Generatoren und der verschiedenen Stufen von Optimierungen miteinander verglichen und analysiert werden, um zu sehen, welcher Ansatz die besten Ergebnisse liefert, um abschließend ein Fazit zu ziehen, ob sich der Einsatz eines MCP Generators lohnt.

5.1 Vergleich der Ergebnisse

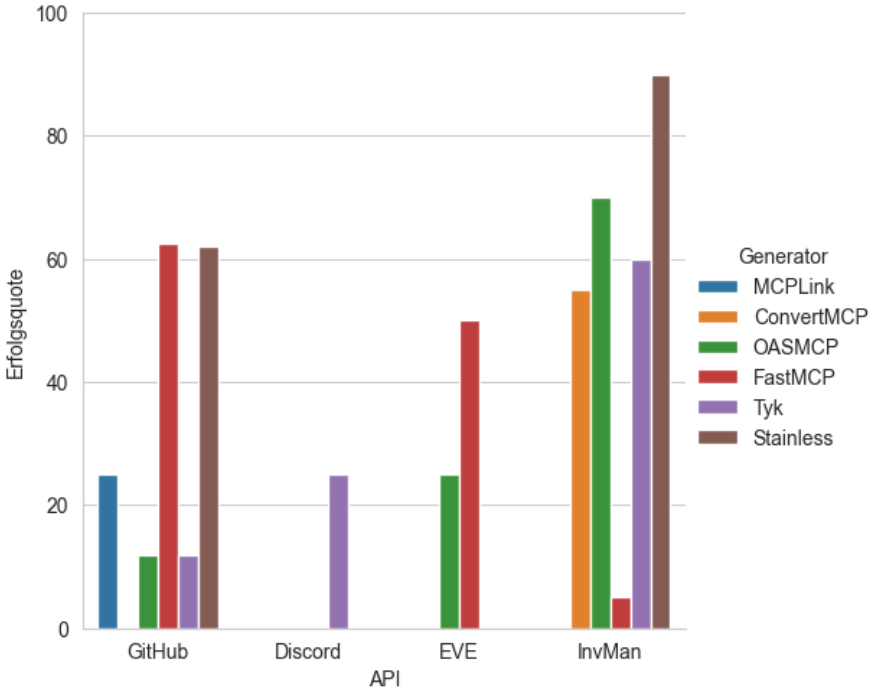

Abbildung 5.1: Erfolgsquoten der Generatoren pro API.

Abbildung 5.1 zeigt die Erfolgsquoten der Generatoren pro API. In der Abbildung ist zu erkenne, dass sich die Ergebnisse pro API je nach Generator stark unterscheiden. Besonders auffällig ist dies bei der GitHub API. Des weiteren ist zu erkennen, dass bei der Discord API nur der *Tyk* Generator erfolgreiche Tests durchführen konnte, während die anderen Generatoren hier alle Tests nicht lösen konnten. Bei der EVE Online API sehen die Ergebnisse ähnlich aus, hier konnten nur der *OpenAPI-to-MCP (OASMCP)* Generator und der *FastMCP* Generator erfolgreiche Tests durchführen. Sofern die Generatoren einen nutzbaren MCP-Server generieren konnten, befinden sich 6 von 13 (46,15%) Generatoren in einem Bereich von 50 bis 70% mit einem Ausreißer des *Stainless* Generators bei der Invoice Manager API mit einer Erfolgsquote von ca 90%. Die restlichen sechs Generatoren befinden sich im Bereich von unter 25%. Pro API ergibt dies die folgenden Erfolgsquoten: 27% bei der GitHub API, 4,1% bei der Discord API, 12,5% bei der EVE Online API und 46,6% bei der Invoice Manager API.

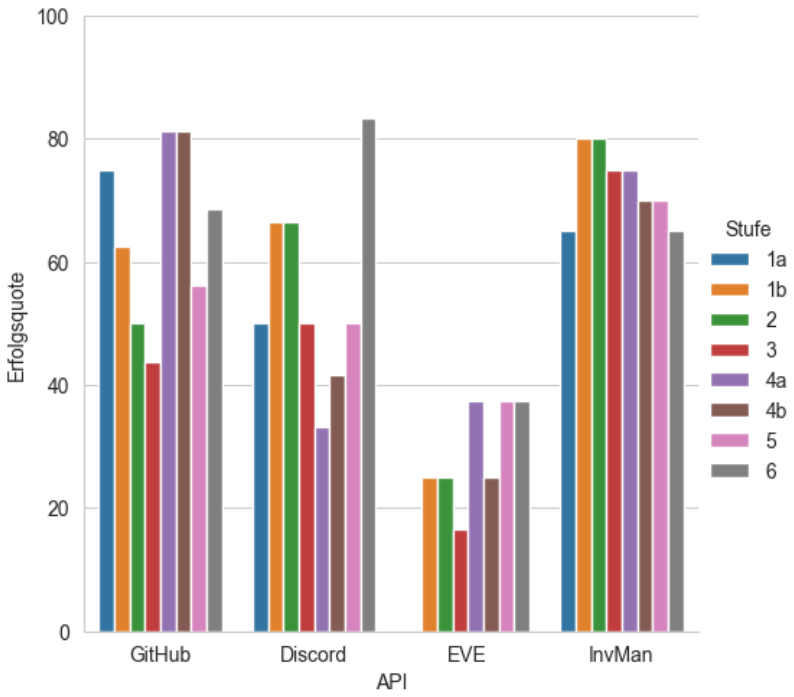

Abbildung 5.2: Erfolgsquoten der Optimierungsstufen pro API.

Im Vergleich zu den MCP Generatoren sind die Ergebnisse des manuell erstellten MCP-Servers deutlich besser. Abbildung 5.2 zeigt die Erfolgsquoten der Optimierungsstufen pro API. Hier ergeben sich pro API die folgenden Erfolgsquoten: 64,8% bei der GitHub API, 55,1% bei der Discord API, 25,5% bei der EVE Online API und 72,5% bei der Invoice Manager API. Dies ergibt eine Steigerung der Erfolgsquote von ca. 55% im schlechsten Fall, der Invoice Manager API, und im besten Fall von ca. 1243% bei der Discord API, im Vergleich zu den MCP Generatoren. Beim manuellen MCP-Server liegen die Erfolgsquoten in 21 von 32 Fällen bei genau oder über 50%. Bei den Generatoren nur in 7 von 24 Fällen. An den Ergebnissen der verschiedenen Stufen kann man sehen, dass es nicht wie vielleicht erwartet einen stetigen Anstieg der Erfolgsquote gibt, sondern dass die Erfolgsquote je nach Stufe und API unterschiedlich ausfällt. Es gibt Stufen, bei denen die Erfolgsquote im Vergleich zur vorherigen Stufe sinkt und Stufen bei denen die Erfolgsquote im Vergleich zur vorherigen Stufe steigt. Daher lässt sich nicht sagen, dass eine bestimmte Optimierung immer zu einer Verbesserung der Erfolgsquote führt, sondern es kommt auf die API und die spezifischen Optimierungen an, um die besten Ergebnisse zu erhalten. Zusätzlich zu den besseren Erfolgsquoten bietet der manuell erstellte MCP-Server noch die Möglichkeit, die Tools weiter zu verbessern, was bei den meisten Generatoren nicht möglich ist, da sie den MCP-Server dynamisch generieren und es daher nicht möglich ist, Änderungen an der Logik der Tools vorzunehmen. So kann zum Beispiel die Größe der Toolantwort durch Filterung wie mit dem 'jq_filter' Parameter reduziert werden, um den Context für das LLM zu reduzieren. Diese Optimieurng wurde in Stufe 4a und 4b umgesetzt. Die Reaktion der Tests auf diese Optimierung ist unterschiedlich ausgefallen. Bei einigen Schnittstellen führte die Nutzung zu einer Verbesserung der Erfolgsquote, bei anderen Schnittstellen sank die Quote, wie bei der Discord API zu sehen ist. Interessant bei dieser Stufe ist aber Senkung der vom Tool gelieferten Zeichen. Da die Nutzung dieses Parameters in den durchgeführten Tests nicht sehr häufig war, können an dieser Stelle keine Aussagen über die Effektivität des Parameters gemacht werden. Der 'jq_filter' Parameter wurde in Stufe 4a bei 56 durchgeführten Tests nur in 7 Tests genutzt, was einer Nutzung von 12,5% entspricht. Auch konnte in Stufe 4b, durch das Hinzufügen des

OutputSchemas, die Nutzung des Parameters nicht gesteigert werden. Dies speigelt sich auch in den Erfolgsquoten wieder, welche sich, bis auf bei der Discord API, nicht verbessert haben. Hier beläuft sich die Nutzung des 'jq_filter' Parameters bei 56 durchgeführten Tests auf 10 Tests (17,8%). Die letzte Stufe der Optimierungen die auf den API Endpunkten basieren, die Umformung der Toolantworten in das TOON-Format, ergab bei der Discord und EVE API eine Steigerung der Erfolgsquote, während sie bei der GitHub API sank und bei der Invoice Manager API gleich blieb. Interessant an dieser Stufe ist die Anzahl der verwendeten Zeichen. Um dies zu veranschaulichen wurde die Anzahl der Zeichen für das vierte Szenario der Invoice Manager API, welches die Erstellung einer neuen Rechnung behandelt, bestimmt. Dafür wurde der Durchschnitt der verwendeten Zeichen aller vier Versuche einer Stufe oder eines Generators genommen, um so die Anzahl der Zeichen pro Stufe oder Generator zu bestimmen. Das Ergebnis ist in Abbildung 5.3 zu sehen. Das Ergebnis der Zeichen Analyse ist in Abbildung 5.3 zu sehen.

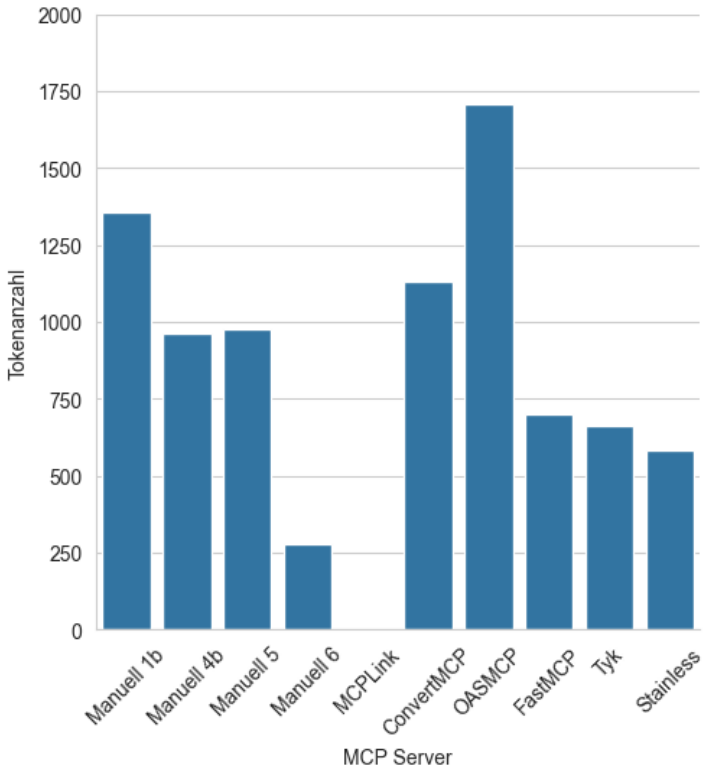

Abbildung 5.3: Anzahl der verwendeten Zeichen für das vierte Szenario der Invoice Manager API pro Stufe oder Generator.

Man kann erkennen, dass die Anzahl der Zeichen in der fünften Stufe, welche die Ausgabe in das TOON-Format umwandelt, etwas höher ist als in der vorherigen Stufe, was darauf hindeutet, dass die Umwandlung der Ausgabe in das TOON-Format nicht zur gewünschten Reduzierung der Zeichen geführt hat, im Vergleich zur Optimierungsstufe 1b wurde aber eine Reduzierung von ca. 27,8% erreicht. Die Anzahl der Zeichen bei den Generatorn *FastMCP*, emphTyk und *Stainless* liegt noch etwas unter den Zeichen der Stufe 5. Eine deutliche Reduzierung der Zeichen, 52,1% im Vergleich zum *Stainless* Generator, konnte mit der Optimierungsstufe 6 erreicht werden. Diese Steigerung lässt sich durch die Reduzierung der Anzahl der notwendigen Toolaufrufe erklären, da die Toolantworten von diversen Zwischenschritten garnicht erst das LLM erreichen. Diese Stufe hat noch zwei weitere Vorteile gegenüber den voherigen Stufen und den Generatoren. Zum einen können Aufgaben deutlich schneller erledigt werden, da weniger Toolaufrufe durchgeführt und

durch das LLM verarbeitet werden müssen. Der MCP-Server der Stufe 6 benötigt im Vergleich zum *Stainless* Generator 368,5% weniger Zeit für die Erstellung einer Rechnung. Stufe 6 benötigte durschschnittlich 195,3 ms und *Stainless* 913 ms für die Erstellung einer Rechnung. Darin ist die Zeit für die Verarbeitung der Toolergebnisse durch das LLM nicht inbegriffen. Der zweite Vorteil ist die Reduzierung des Contexts für das LLM, da weniger Tools bereitgestellt werden müssen. Während die Generatoren und die vorherigen Stufen sechs Tools bereitstellen mussten um das Szenario der Erstellung einer Rechnung zu lösen, benötigte Stufe 6 nur noch 1 Tool, um die Aufgabe zu erledigen. Dies verringert die Anzahl an Zeichen welche an das LLM mitgegeben werden müssen, im Falle des FastMCP Generators, um 15633 Zeichen pro Aufruf. laut dem OpenAI Tokenizer [74] dies einer Reduktion von 3430 Token, was bei einem Preis von 0,00001 US Dollar pro Token bei GitHub Models API [77], einer Reduktion von 0,0343 US Dollar pro Aufruf entspricht. Bei sechs notwendigen Aufrufen für dieses Szenario entspricht dies einem Ersparnis von 0,1372 US Dollar für die Ausführung dieses Szenarios. Diesem Ersparnis gegenübr steht die Zeit, die zuvor inverstiert werden musste, um den MCP-Server zu implmentieren. Die Dauer der Implementierung der einzelnen Stufen ist in Tabelle 5.4 zu sehen. Man kann erkennen, dass die

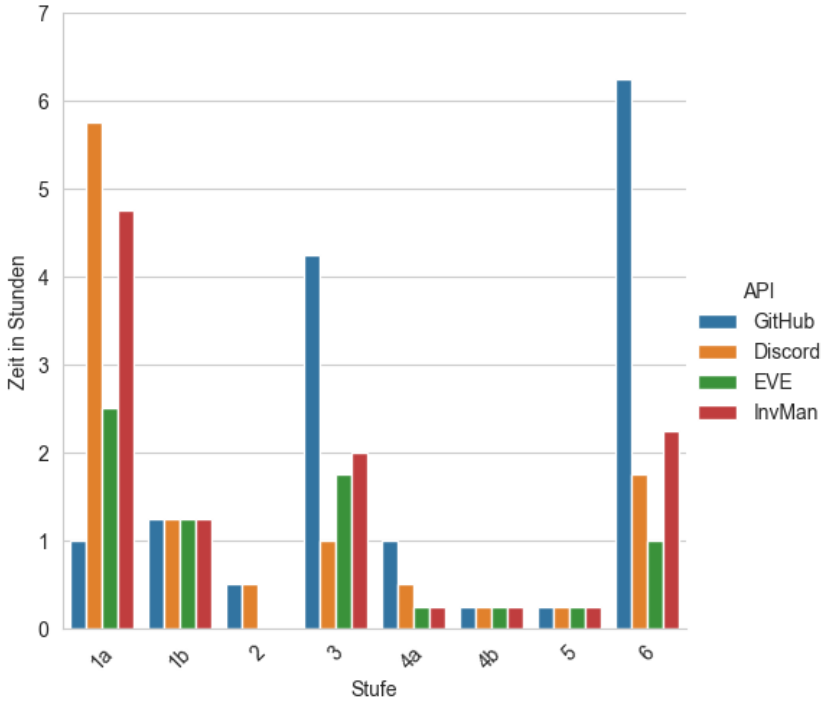

Abbildung 5.4: Dauer der Implementierung der einzelnen Stufen.

Implementierung der ersten Stufe, mit 14 Stunden, am meisten Zeit in Anspruch genommen hat. Dazu kommt noch die Zeit für die Generierung der API mit einem OpenAPI Generator. Die Dauer der Umsetzung der weiteren Stunden hält sich meist bei unter einer Stunde pro API. Eine Ausnahme bildet Stufe 3, in der das OutputSchema durch Python Type Hints definiert wurde. Die identischen Umsetzungszeiten in den Stufen 1b, 4b und 5 erklären sich durch die Art der Optimierung. Stufe 1b konnte zum Großteil durch Dekoratoren umgesetzt werden, welche dann nur an die entsprechenden Tools angehängt werden mussten. In Stufe 4b wurden nur die OutputSchemas, die bereits in Stufe 3 verwendet wurden, wieder eingefügt werden und in Stufe 5 wurde nur das Ergebnis des Tools durch die *encode* Funktion der TOON Python Implementierung umgewandelt. Diese Aufgaben konnten in wenigen Minuten erledigt werden. Da die Erfassung der Zeiten in 15 Minuten Blöcken erfolgte haben diese Stufen jeweils die gleiche Zeit von 15

Minuten pro API erhalten. Stufe 6 benötigte wieder mehr Zeit, da die Tools von Grund auf neu implementiert wurden. Die hier erfassten Zeiten zeigen die benötigte Dauer der Umsetzung von maximal 9 Tools pro API in Stufe 5 oder 7 in Stufe 6. Rechnet man die Umsetzungszeiten der Stufen 1 bis 5 für die Anzahl aller API Endpunkte am Beispiel der GitHub API hoch, welche 1111 Endpunkte umfasst, kommt man auf eine Dauer von 1728 Stunden (216 Arbeitstage), um alle Tools der GitHub API umzusetzen. Im Verlgeich dazu betrug die längste Dauer für die Generierung eines MCP-Servers durch den *Stainless* Generator 12 Stunden und erreichte damit bei den zwei APIs die erfolgreiche Versuche hatten, eine Erfolgsquote von mindestens 60%.

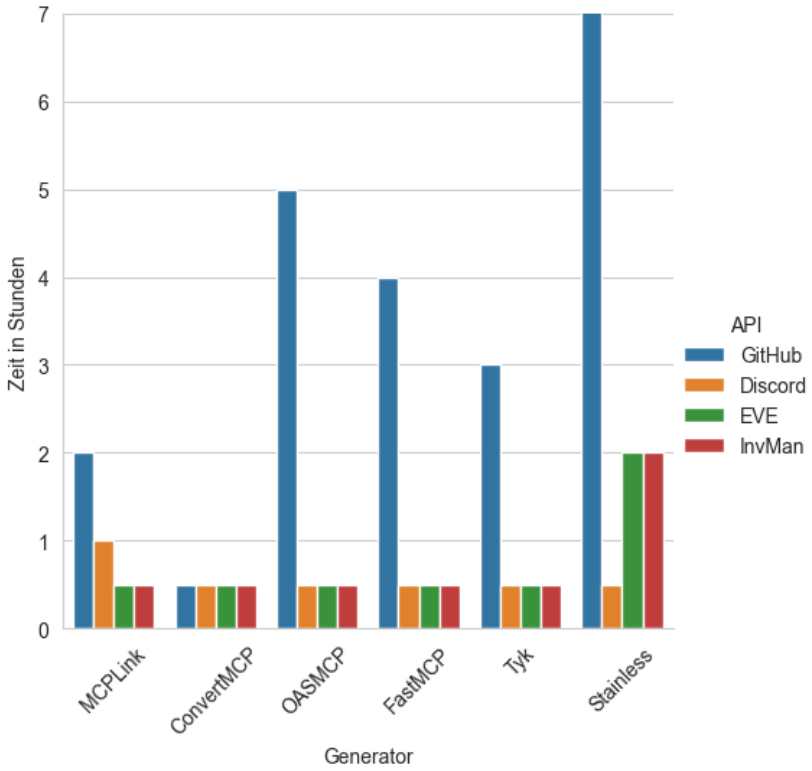

Abbildung 5.5: Dauer der Implementierung mit Generatoren.

Abbildung 5.6 zeigt die RAM-Nutzung der verschiedenen Generatoren und Optimierungsstufen. Wie zu erwarten war, hat die GitHub API die höchste RAM-Nutzung, da sie die meisten Endpunkte und damit die meisten Tools bereitstellt. Beachtenswert ist dabei, dass die RAM-Nutzung zwischen *FastMCP* und dem manuellen MCP-Server sich nur geringfügig unterscheiden, obwohl der manuelle MCP-Server deutlich weniger Tools bereitstellt als *FastMCP*. Grundsätzlich ist zu sehen, dass die Python basierten MCP Server (*ConvertMCP*, *FastMCP*, Stufe 5, Stufe 6) mehr Arbeitsspeicher benötigen als JavaScript bzw. TypeScript basierte MCP Server. Wobei der Anstieg mit der Anzahl der Tools überproportional zu sein scheint, was sich an der RAM-Nutzung der GitHub API im Vergleich zu den anderen APIs zeigt. Am wenigsten RAM benötigte der *OpenAPI-to-MCP* (OASMCP) Generator. Vergleicht man die Python basierten MCP Generatoren mit der manuellen Implmentierung ist erkennbar, dass die RAM-Nutzung der manuellen Implementierungim Vergleich zu *FastMCP* und *ConvertMCP* etwas höher ist, obwohl sie deutlich weniger Tools bereitstellt. Bei den JavaScript/TypeScript basierten Generatoren ist zu beobachten, dass die RAM-Nutzung unabhängig von der Anzahl der Tools relativ konstant bleibt. Eine Ausnahme bilder dabei die GitHub API bei dem *OASMCP* Generator.

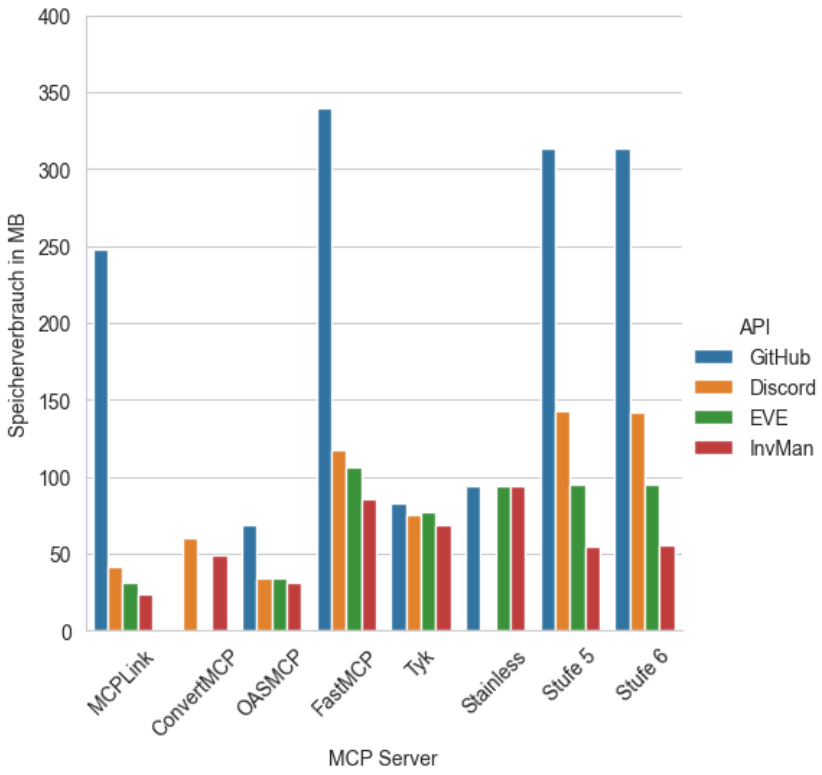

Abbildung 5.6: RAM-Nutzung der verschiedenen Stufen und Generatoren.

Als letzten Punkt werden noch die Verfügbaren Features der beiden Ansätze gegenübergestellt. Dafür wird die Tabelle der Featureübersicht 4.3 der Generatoren aus Kapitel 4.3 um den manuellen MCP-Server erweitert.

Die Tabelle 5.1 zeigt die Unterstützung der verschiedenen MCP-Server Capabilities der Generatoren und des manuellen MCP-Servers. Auch der manuelle MCP-Server unterstützt nur die grudnlegenden Capabilities die Tools betreffen. Jedoch setzt der manuelle MCP-Server wesentlich mehr Best Practices um als die Generatoren. Punkte wie die Nutzung von Annotations oder die Nutzung von ContentTypes sind bei der Erstellung eines MCP-Server auch einfach möglich. Diese beiden Punkte wurden von den Generatoren, mit Ausnahme von Annotations bei dem *Stainless* Generator, nicht umgesetzt. Auch ist eine nachträgliche Anpassung nicht möglich, da die meisten Generatoren den MCP-Server dynamisch generieren. Hier bilden *Stainless* und *ConvertMCP* die Ausnahme, da es sich bei diesen um statische Generatoren handelt.

5.2 Fazit

Die vorangegangenen Kapitel haben gezeigt, dass die Qualität und die damit erreichbaren Ergebnise der MCP Geneneratoren sich stark von einander unterscheiden, sowohl was die Erfolgsquoten als auch die unterstützten Features betrifft. Die Implmentierung des maneullen MCP-Servers hat gezeigt, dass damit bessere Ergebnisse erzielt werden können, sowohl bei dem MCP-Server der auf den API Endpunkten basiert (Stufe 1 - 5), als auch bei den optimierten Tools (Stufe 6). Wobei die Tools der letzten Stufe, weitere Vorteile gegenüber den vorherigen Stufen und den Generatoren bieten, wie die Reduzierung der Anzahl der notwendigen Toolaufrufe, die damit verbundene Reduzierung des Contexts für das LLM und die schnellere Erledigung der Aufgaben. Aufgrund ihrer Natur werden die Generatoren einen solchen Grad an Optimierung

Capability	MCPLink	ConvertMCP	OASMCP	FastMCP	Tyk	Stainless	Manuell
Prompts	✗	✗	✗	✗	✗	✗	✗
Resources	✗	✗	✗	✓[1]	✗	✗	✗
Completions	✗	✗	✗	✗	✗	✗	✗
Logging	✗	✗	✗	✓	✗	✓	✗
Pagination	✗	✗	✗	✗	✗	✗	✗
Cancellation	✗	✗	✗	✗	✗	✗	✗
Ping	✗	✗	✗	✗	✗	✗	✗
Progress	✗	✗	✗	✓	✗	✗	✗
Tasks	✗	✗	✗	✗	✗	✗	✗
Tools	✓	✓	✓	✓	✓	✓	✓
InputSchema	✓	✓	✓	✓	✓	✓	✓
OutputSchema	✗	✗	✗	✓	✓	✗	✓
Structured Output	✗	✗	✗	✓	✗	✗	✓
ContentTypes	✗	✗	✗	✗	✗	✗	✗
Annotations	✗	✗	✗	✗	✗	✓	✗
Security	✗	✗	✗	✗[2]	✗	✗	✓
Error Handling	✓	✗	✗	✓	✓	✓	✓

Tabelle 5.1: Unterstützte MCP-Server Capabilities der Generatoren und des manuellen MCP-Servers. Der Prefix Tool wurde bei den unteren sieben Punkten aufgrund der Formatierung weggelassen.

1: Über manuelles Mapping
2: Nur Inputparametervalidierung, Rate-Limiting über Middleware

nicht erreichen können. Das Rechenbeispiel im vorherigen Abschnitt zeigt aber auch, dass die Optimierung der Tools einige Zeit in Anspruch nimmt und damit Ressourcen bindet und Kosten verursacht, welche durch die Optimierung erst wieder zurückgewonnen werden müssen. Dazu kommt noch die Tatsache, dass bei dem manuellen MCP-Server nur einige Endpunkte der API als Tool bereitgestellt wurden, welche für die Erledigung der Aufgaben notwendig waren, während bei den Generatoren alle Endpunkte der API als Tools bereitgestellt wurden und damit auch der volle Funktionsumfang der API und möglichweise auch der Anwendung zur Verfügung steht. Die Entscheidung, ob sich der Einsatz eines MCP Generators lohnt, lässt sich daher nicht pauschal beantworten, sondern hängt von verschiedenen Faktoren, wie der API, der notwendigen Erfolgsquote, des notwendigen Funktionsumfangs, der verfügbaren Ressourcen und der Zeit, die für die Erstellung eines manuellen MCP-Servers benötigt wird, ab. Ein eigens erstellter MCP-Server wird, wie die Ergebnisse gezeigt haben, bessere Ergebnisse liefern. Daher ist dieser Ansatz zu favorisieren, wenn die notwendigen Ressourcen und die Zeit zur Verfügung stehen und eine hohe Erfolgsquote und Effizienz bei der Erledigung der Aufgaben notwendig ist. Ein öffentlicher MCP-Server der von einer großen Nutzeranzahl verwendet wird sollte daher die Möglichkeiten zur Einsparung von Ressourcen, wie Token und LLM Aufrufe, nutzen, um Kosten zu reduzieren. Ein MCP-Server der von einer kleineren Nutzeranzahl verwendet wird und bei dem der Erfolg einer Aufgabe nicht kritisch ist, könnte auch mit einem MCP Generator erstellt werden. Dies könnte bei einem MCP-Server der nur Daten ausliest und an das LLM weitergibt, der Fall sein, da ein falscher Parameter zwar falsche Ergebnisse liefern würde, aber keine kritischen Fehler verursachen würde, wie einer Daten verändernden Aktion.

Anhand der in dieser Arbeit erfassten Ergebnisse kann eine Kalkulation für die Kosten zur Erstellung eines manuellen MCP-Servers und der möglichen Einsparungen erstellt werden, um mithilfe der erwarteten Nutzung des MCP-Servers eine Entscheidung zu treffen, ob sich die Erstellung eines manuellen MCP-Servers lohnt. Die Forschungsfrage dieser Arbeit lautete:

"Lohnt sich die Erstellung eines manuellen MCP-Servers im Vergleich zum Einsatz eines MCP Generators?". Die Antwort auf diese Frage hängt also stark von den individuellen Anforderungen und Umständen ab. Dennoch kann man annehmen, dass ein auf das Nutzungsszenario und die Schnittstelle angepasster, MCP-Server immer bessere Ergebnisse liefern wird, als ein generierter MCP-Server.

Literatur

[1] L. Li, W. Chou, W. Zhou und M. Luo, „Design Patterns and Extensibility of REST API for Networking Applications,“ *IEEE Transactions on Network and Service Management*, Jg. 13, Nr. 1, S. 154–167, 2016, ISSN: 1932-4537. DOI: 10.1109/TNSM.2016.2516946.

[2] The Linux Foundation, *OpenAPI Specification*, 2025. Adresse: https://www.openapis.org/ (besucht am 22. 12. 2025).

[3] SmartBear Software, *OpenAPI 3.0 Tutorial*, 2025. Adresse: https://support.smartbear.com/swaggerhub/docs/en/get-started/openapi-3-0-tutorial.html (besucht am 03. 12. 2025).

[4] OpenAI, *OpenAI*, 2025. Adresse: https://openai.com (besucht am 22. 12. 2025).

[5] P. Lewis, E. Perez, A. Piktus u. a., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 22. Mai 2020. Adresse: https://arxiv.org/pdf/2005.11401.

[6] T. Schick, J. Dwivedi-Yu, R. Dessì u. a., *Toolformer: Language Models Can Teach Themselves to Use Tools*, 9. Feb. 2023. Adresse: https://arxiv.org/pdf/2302.04761.

[7] S. Yao, J. Zhao, D. Yu u. a., *ReAct: Synergizing Reasoning and Acting in Language Models*, 6. Okt. 2022. Adresse: http://arxiv.org/pdf/2210.03629v3.

[8] A. Parisi, Y. Zhao und N. Fiedel, *TALM: Tool Augmented Language Models*, 24. Mai 2022. Adresse: https://arxiv.org/pdf/2205.12255.

[9] S. G. Patil, T. Zhang, X. Wang und J. E. Gonzalez, *Gorilla: Large Language Model Connected with Massive APIs*, 24. Mai 2023. Adresse: https://arxiv.org/pdf/2305.15334.

[10] Y. Shen, K. Song, X. Tan, D. Li, W. Lu und Y. Zhuang, *HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face*, 30. März 2023. Adresse: http://arxiv.org/pdf/2303.17580v4.

[11] Y. Qin, S. Liang, Y. Ye u. a., *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*, 31. Juli 2023. Adresse: https://arxiv.org/pdf/2307.16789.

[12] Y. Song, W. Xiong, D. Zhu u. a., *RestGPT: Connecting Large Language Models with Real-World RESTful APIs*, 27. Aug. 2023. Adresse: http://arxiv.org/pdf/2306.06624.

[13] P. Lu, B. Peng, H. Cheng u. a., *Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models*, 19. Apr. 2023. Adresse: https://arxiv.org/pdf/2304.09842v3.

[14] OpenAI, J. Achiam, S. Adler u. a., *GPT-4 Technical Report*, 15. März 2023. Adresse: https://arxiv.org/pdf/2303.08774.

[15] Anthropic PBC, *Introducing the Model Context Protocol*, 2024. (besucht am 22. 12. 2025).

[16] Anthropic PBC, *Understanding MCP servers*, 2025. (besucht am 22. 12. 2025).

[17] Anthropic PBC, *MCP SDKs*, 2025. Adresse: https://modelcontextprotocol.io/docs/sdk (besucht am 22. 12. 2025).

[18] Atlassian, *Introducing Atlassian's Remote Model Context Protocol (MCP) Server*, 2025. Adresse: https://www.atlassian.com/blog/announcements/remote-mcp-server (besucht am 22. 12. 2025).

[19] Jeremiah Lowin, *FastMCP 2.0*, 2025. Adresse: https://gofastmcp.com (besucht am 22. 12. 2025).

[20] Harsha Vardhan, *OpenAPI to MCP Generator*, 2025. Adresse: https://github.com/harsha-iiiv/openapi-mcp-generator (besucht am 22. 12. 2025).

[21] M. Li, Y. Zhao, B. Yu u. a., *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*, 2023. Adresse: https://arxiv.org/pdf/2304.08244.

[22] S. Yuan, K. Song, J. Chen u. a., *EASYTOOL: Enhancing LLM-based Agents with Concise Tool Instruction*, 2024. Adresse: https://arxiv.org/pdf/2401.06201.

[23] MCP contributors, *MCP Registry*, 2025. Adresse: https://registry.modelcontextprotocol.io/ (besucht am 17. 03. 2026).

[24] Den Delimarsky, *MCP Registry Analytics*, 2026. Adresse: https://mcpgrowth.den.dev/ (besucht am 17. 03. 2026).

[25] Z. Wang, Q. Chang, H. Patel u. a., *MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers*, 28. Aug. 2025. Adresse: https://arxiv.org/pdf/2508.20453.

[26] C. Bandi, B. Hertzberg, G. Boo u. a., *MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers*, 2026. Adresse: https://arxiv.org/pdf/2602.00933.

[27] Z. Luo, Z. Shen, W. Yang u. a., *MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers*, 20. Aug. 2025. Adresse: https://arxiv.org/pdf/2508.14704.

[28] M. Yin, D. Shen, S. Xu u. a., *LiveMCP-101: Stress Testing and Diagnosing MCP-enabled Agents on Challenging Queries*, 21. Aug. 2025. Adresse: https://arxiv.org/pdf/2508.15760.

[29] M. Mastouri, E. Ksontini und W. Kessentini, *Making REST APIs Agent-Ready: From OpenAPI to Model Context Protocol Servers for Tool-Augmented LLMs*, 23. Juli 2025. DOI: 10.48550/ARXIV.2507.16044.

[30] Pierce Clark und Kevin Whinnery, *SDK code mode shows SotA accuracy and performance for agents using APIs*, Stainless Software, Inc., Hrsg., 2026. Adresse: https://www.stainless.com/blog/sdk-code-mode (besucht am 18. 03. 2026).

[31] Kenton Varda und Sunil Pai, *Code Mode: the better way to use MCP*, Cloudflare, Inc., Hrsg., 2025. Adresse: https://blog.cloudflare.com/code-mode/ (besucht am 18. 03. 2026).

[32] Adam Jones und Conor Kelly, *Code execution with MCP: Building more efficient agents*, P. B. Anthropic, Hrsg., 2025. Adresse: https://www.anthropic.com/engineering/code-execution-with-mcp (besucht am 18. 03. 2026).

[33] Nolan Sullivan, *Generate MCP servers from OpenAPI documents*, 2025. Adresse: https://www.speakeasy.com/blog/generate-mcp-from-openapi?utm_source=copilot.com#four-tools-for-generating-mcp-servers-from-openapi-documents (besucht am 15. 03. 2026).

[34] Speakeasy, *gram: The MCP platform*, Speakeasy, Hrsg., 2026. Adresse: https://www.getgram.ai/ (besucht am 18. 03. 2026).

[35] Thiago mendes, *Multi-Language MCP Server Performance Benchmark*, TM Dev Lab, Hrsg., 2026. Adresse: https://www.tmdevlab.com/mcp-server-performance-benchmark.html (besucht am 14. 03. 2026).

[36] Anthropic PBC, Open Source Contributors, *MCP Python SDK*, 2024. (besucht am 24. 12. 2025).

[37] Public APIs GitHub Contributors, *Public APIs List*, 2025. Adresse: https://github.com/public-apis/public-apis (besucht am 16. 12. 2025).

[38] Free APIs GitHub Contributors, *Free APIs List*, 2025. Adresse: https://free-apis.github.io/#/categories (besucht am 16. 12. 2025).

[39] Tim Tomczek, *Invoice Manager API*, GitHub, 2024. Adresse: https://github.com/TTomczek/invoice-manager-api/tree/main (besucht am 20. 12. 2025).

[40] I. GitHub, *GitHub OpenAPI Specification 3.9*, 2025. Adresse: https://github.com/github/rest-api-description/blob/main/descriptions/ghes-3.9/ghes-3.9.yaml (besucht am 16. 12. 2025).

[41] Discord Inc., *Discord OpenAPI Specification*, 2025. Adresse: https://github.com/discord/discord-api-spec/blob/main/specs/openapi.json (besucht am 16. 12. 2025).

[42] CCP ehf, *EVE Online OpenAPI Specification 2025-11-06*, 2025. Adresse: https://developers.eveonline.com/api-explorer (besucht am 16. 12. 2025).

[43] Tim Tomczek, *Studygroupmap Repository*, GitHub, 2023. Adresse: https://github.com/TTomczek/studygroupmap (besucht am 05. 12. 2025).

[44] Tim Tomczek, *Invoice Manager Server*, 2025. Adresse: https://github.com/TTomczek/invoice-manager-server (besucht am 15. 12. 2025).

[45] GitHub, Inc., *Introducing GitHub Models: A new generation of AI engineers building on GitHub: Blog*, 2024. Adresse: https://github.blog/news-insights/product-news/introducing-github-models/ (besucht am 15. 12. 2025).

[46] I. GitHub, *OpenAI GPT-4.1-mini: GitHub Models*. Adresse: https://github.com/marketplace/models/azure-openai/gpt-4-1-mini (besucht am 15. 12. 2025).

[47] I. GitHub, *Mistral Medium 3 (25.05): GitHub Models*. Adresse: https://github.com/marketplace/models/azureml-mistral/mistral-medium-2505 (besucht am 15. 12. 2025).

[48] Tim Tomczek, *Anhänge*, 2026. Adresse: https://github.com/TTomczek/thesis-attachments/releases/tag/1.0.0 (besucht am 05. 03. 2026).

[49] P. B. Anthropic, *Java SDK Overview*, 2024. Adresse: https://modelcontextprotocol.io/sdk/java/mcp-overview (besucht am 15. 12. 2025).

[50] Walter Domenico Vergara, *ConvertMCP*, 2025. Adresse: https://github.com/walterwootz/api-mcp-magic (besucht am 15. 12. 2025).

[51] Automation AI Labs, *MCPLink*, 2025. Adresse: https://github.com/automation-ai-labs/mcp-link (besucht am 15. 12. 2025).

[52] Stainless Software, Inc., *Stainless MCP*, 2025. Adresse: https://www.stainless.com/products/mcp (besucht am 15. 12. 2025).

[53] Tyk Technologies, *Tyk MCPs*, 2025. Adresse: https://tyk.io/docs/ai-management/mcps/overview (besucht am 15. 12. 2025).

[54] Anthropic PBC, *MCP Specifications: Transports*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-03-26/basic/transports#streamable-http (besucht am 15. 12. 2025).

[55] Model Context Protocol a Series of LF Projects, LLC, *MCP Toolname Length Specification*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-11-25/server/tools#tool-names (besucht am 14. 01. 2026).

[56] Stack Exchange Inc., *StackOverflow 2025 Developer Survey*, Stack Exchange Inc., Hrsg., 2025. Adresse: https://survey.stackoverflow.co/2025/technology/.

[57] jlowin, *FastMCP 1.0 MCP SDK*. Adresse: https://github.com/jlowin/fastmcp?tab=readme-ov-file#fastmcp-20-the-standard-framework (besucht am 02. 01. 2026).

[58] Amazon Web Services - Labs, *AWS OpenAPI MCP Server*, 2026. Adresse: https://github.com/awslabs/mcp/blob/main/src/openapi-mcp-server/pyproject.toml (besucht am 02. 01. 2026).

[59] Anthropic PBC, *MCP OutputScheme*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-06-18/changelog (besucht am 05. 01. 2026).

[60] Jeremiah Lowin, *FastMCP OpenAPI Integration Route-Mapping*, 2025. Adresse: https://gofastmcp.com/integrations/openapi#route-mapping (besucht am 05. 01. 2026).

[61] Stainless Software, Inc., *Stainless Software, Inc.* 2026. Adresse: https://www.stainless.com/ (besucht am 14. 01. 2026).

[62] Stainless Software, Inc., *Stainless Pricing*, 2026. Adresse: https://www.stainless.com/pricing (besucht am 04. 01. 2026).

[63] jqlang Organization, *JQ*. Adresse: https://jqlang.org/ (besucht am 04. 01. 2026).

[64] Anthropic PBC, *MCP ToolAnnotations*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-03-26/changelog#major-changes (besucht am 05. 01. 2026).

[65] htop, *htop - an interactive process viewer*. Adresse: https://htop.dev/ (besucht am 04. 01. 2026).

[66] Model Context Protocol a Series of LF Projects, LLC, *MCP Server Utilities*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-11-25/server/utilities/completion (besucht am 14. 01. 2026).

[67] Model Context Protocol a Series of LF Projects, LLC, *MCP Utilities*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-11-25/basic/utilities/cancellation (besucht am 14. 01. 2026).

[68] Model Context Protocol a Series of LF Projects, LLC, *MCP Tool Security Considerations*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-11-25/server/tools#security-considerations (besucht am 14. 01. 2026).

[69] Model Context Protocol a Series of LF Projects, LLC, *MCP Tool Error Handling*, 2025. Adresse: https://modelcontextprotocol.io/specification/2025-11-25/server/tools#error-handling (besucht am 14. 01. 2026).

[70] Johann Schopplich, *TOON - Token Oriented Object Notation*, 2025. Adresse: https://github.com/toon-format/toon (besucht am 16. 02. 2026).

[71] Pydantic GitHub Contributors, *Pydantic*, 2026. Adresse: https://docs.pydantic.dev.

[72] Broadcom Inc., GitHub Contributors, *Spring AI MCP*, 2026. Adresse: https://github.com/spring-projects/spring-ai/tree/main/mcp/common/src/main/java/org/springframework/ai/mcp (besucht am 22. 02. 2026).

[73] Michael Williamson, *jq.py: a lightweight and flexible JSON processor*, 2026. Adresse: https://github.com/mwilliamson/jq.py (besucht am 21. 02. 2026).

[74] OpenAI, *OpenAI Tokenizer*, 2026. Adresse: https://platform.openai.com/tokenizer (besucht am 23. 02. 2026).

[75] toon-python GitHub Contributors, *TOON Format for Python*, 2026. Adresse: https://github.com/toon-format/toon-python (besucht am 23. 02. 2026).

[76] Jeremiah Lowin, *Stop Converting Your REST APIs to MCP: Your auto-generated MCP is bad, and you should feel bad.* 2025. Adresse: `https://www.jlowin.dev/blog/stop-converting-rest-apis-to-mcp` (besucht am 24. 02. 2026).

[77] GitHub, Inc., *GitHub Models Pricing*, 2026. Adresse: `https://docs.github.com/en/billing/concepts/product-billing/github-models#token-units` (besucht am 11. 03. 2026).

Abbildungsverzeichnis

5.1 Erfolgsquoten der Generatoren pro API. 47
5.2 Erfolgsquoten der Optimierungsstufen pro API. 48
5.3 Anzahl der verwendeten Zeichen für das vierte Szenario der Invoice Manager API pro Stufe oder Generator. 49
5.4 Dauer der Implementierung der einzelnen Stufen. 50
5.5 Dauer der Implementierung mit Generatoren. 51
5.6 RAM-Nutzung der verschiedenen Stufen und Generatoren. 52

Tabellenverzeichnis

4.1 Überblick über die definierten Nutzungsszenarien der vier ausgewählten Schnittstellen. 14
4.2 Unterstützte OAS Features der Generatoren 28
4.3 Unterstützte MCP-Server Capabilities der Generatoren 29

5.1 Unterstützte MCP-Server Capabilities der Generatoren und des manuellen MCP-Servers. Der Prefix Tool wurde bei den unteren sieben Punkten aufgrund der Formatierung weggelassen. 53

Listingverzeichnis

4.1 Minimalbeispiel für die Nutzung von FastMCP zur Generierung eines MCP-Servers aus einer OpenAPI Beschreibung der Invoice Manager API. 20
4.2 Beispielcode, welcher von dem GPT Modell im 'Code' Modus des Stainless MCP-Servers generiert wurde. 27
4.3 Befehl zur Generierung der API Module mit der OpenAPI CLI 29
4.4 Beispiel für Erstellung eines einfachen MCP-Server mit dem SDK. 31
4.5 Beispiel des JSON-Formats 37
4.6 Beispiel des YAML-Formats 37
4.7 Beispiel des TOON-Formats 38
4.8 Beispiel für die Kombination der Tools zur Abfrage der Labels und zur Erstellung eines Issues. 39
4.9 Auszug aus einem Tool des Discord MCP-Servers. Zu sehen ist die Abfrage der Server- und Kanalid 41
4.10 Tool zur Erstellung einer Rechnung mit Positionen 43

KI Nutzungserklärung

Diese Arbeit wurde unter Verwendung von KI-Technologien erstellt. Alle Inhalte wurden sorgfältig überprüft und bearbeitet, um sicherzustellen, dass sie den akademischen Standards entsprechen. Die KI-Technologien wurden als unterstützende Werkzeuge genutzt und haben keinen Einfluss auf die Originalität oder Authentizität der Arbeit. Alle Quellen und Referenzen wurden ordnungsgemäß zitiert, um die Integrität der Arbeit zu gewährleisten. Als KI-Werkzeuge wurde das GitHub Copilot Plugin für IntelliJ und PyCharm von JetBrains und Visual Studio Code verwendet, um Code oder Text zu vervollständigen und nicht als Hauptquelle für die Erstellung von Inhalten. Ausnahmen dazu sind in den entsprechenden Abschnitten der Arbeit angegeben.

21. März 2026

Tim Tomczek

Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und dabei keine anderen als die angegebenen Hilfsmittel benutzt habe. Sämtliche Stellen der Arbeit, die im Wortlaut oder dem Sinn nach Werken anderer Autoren entnommen sind, habe ich als solche kenntlich gemacht. Die Arbeit wurde bisher weder gesamt noch in Teilen einer anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

21. März 2026

Tim Tomczek